

STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

STAR Development Team
Locked Inc.

Contents

1 Glossary of Key Terms

This section defines key terminology used throughout this document.

1.1 Communication Protocols

gRPC **Google Remote Procedure Call** – A high-performance, open-source RPC framework developed by Google. Uses HTTP/2 for transport and Protocol Buffers (Protobuf) for serialization. Enables efficient binary communication between services with built-in support for streaming.

GraphQL A query language for APIs developed by Facebook. Unlike REST, clients specify exactly what data they need, reducing over-fetching. Supports queries (read), mutations (write), and subscriptions (real-time updates via WebSocket).

gRPC-Web A JavaScript implementation of gRPC for browser clients. Since browsers cannot use HTTP/2 trailers directly, gRPC-Web requires a proxy (like Envoy) to translate between gRPC-Web and native gRPC.

Protobuf **Protocol Buffers** – Google’s language-neutral, platform-neutral binary serialization format. Smaller and faster than JSON. Messages are defined in `.proto` files and compiled to language-specific code.

WebSocket A protocol providing full-duplex communication channels over a single TCP connection. Used by GraphQL subscriptions for real-time data streaming from server to client.

REST **Representational State Transfer** – An architectural style for web APIs using HTTP methods (GET, POST, PUT, DELETE). Uses JSON for data exchange. Simpler but less efficient than gRPC for high-frequency communication.

1.2 Middleware and Infrastructure

DDS **Data Distribution Service** – A middleware protocol and API standard for data-centric publish-subscribe communication. Used by ROS 2 for inter-node communication. Provides real-time, scalable, and reliable data exchange with Quality of Service (QoS) policies.

ROS 2 **Robot Operating System 2** – An open-source robotics middleware framework. Provides tools, libraries, and conventions for building robot applications. Uses DDS for communication between nodes.

SLAM **Simultaneous Localization and Mapping** – A computational technique where a robot builds a map of an unknown environment while simultaneously tracking its location within that map.

Nav2 **Navigation 2** – The ROS 2 navigation stack. Provides autonomous navigation capabilities including path planning (A*), obstacle avoidance (DWA), and localization (AMCL).

- DWA** **Dynamic Window Approach** – A local path planning algorithm used by Nav2. Samples velocities in a “dynamic window” (velocities reachable within one control cycle) and scores trajectories based on proximity to goal, obstacle clearance, and velocity. Runs at 20 Hz in the STAR robot.
- AMCL** **Adaptive Monte Carlo Localization** – A probabilistic localization algorithm that uses a particle filter to track the robot’s pose within a known map. Particles represent hypotheses about the robot’s position; sensor observations update particle weights. SLAM Toolbox provides similar functionality during mapping.
- mDNS** **Multicast DNS** – A protocol that resolves hostnames to IP addresses within small networks without a local DNS server. Enables `star-robot.local` hostname resolution. Implemented by Avahi on Linux.
- Avahi** A free, open-source implementation of mDNS/DNS-SD (Service Discovery) for Linux. Allows devices to discover each other on a local network without configuration.
- Envoy** A high-performance, open-source edge and service proxy. Used in this project to translate gRPC-Web requests from browsers to native gRPC for the Spring Boot backend.

1.3 Hardware Interfaces

- SPI** **Serial Peripheral Interface** – A synchronous serial communication interface for short-distance communication. Uses 4 wires: COPI (Controller Out Peripheral In), CPIO (Controller In Peripheral Out), CLK (Clock), and CS (Chip Select). Used for RPi5-to-ESP32 communication at 10 MHz.
- I2C** **Inter-Integrated Circuit** – A two-wire serial communication protocol (SDA for data, SCL for clock). Used for connecting sensors like IMUs. Supports multiple devices on the same bus with unique addresses.
- UART** **Universal Asynchronous Receiver-Transmitter** – A serial communication protocol using TX (transmit) and RX (receive) lines. Commonly used for debugging and some sensor connections.
- GPIO** **General Purpose Input/Output** – Configurable digital pins on microcontrollers and single-board computers. Can be set as input (read sensors/buttons) or output (control LEDs/relays).
- PWM** **Pulse Width Modulation** – A technique for controlling analog devices using digital signals. The duty cycle (percentage of time the signal is HIGH) controls the effective power. Used for motor speed control.
- Center-Aligned PWM**
A PWM mode where the counter counts up then down, centering pulses in the period. Produces cleaner signals with reduced electromagnetic interference (EMI) compared to edge-aligned PWM. Used by the ESP32 MCPWM peripheral for motor control.
- PID** **Proportional-Integral-Derivative** – A control loop mechanism for maintaining a desired setpoint. The ESP32 runs a 1 kHz PID loop to maintain precise motor velocities based on encoder feedback.

- SMBus** **System Management Bus** – A two-wire interface based on I2C but with stricter timing requirements and additional features like packet error checking (PEC) and timeouts. Used for battery fuel gauge ICs and power management chips.
- 1-Wire** A serial protocol using a single data wire (plus ground) for communication. Devices are powered parasitically from the data line or via a separate power pin. Used by DS18B20 temperature sensors for thermal monitoring.

1.4 Software Concepts

- PWA** **Progressive Web App** – A web application that uses modern web technologies to deliver app-like experiences. Can work offline, be installed on devices, and receive push notifications.

Service Worker

A JavaScript worker that runs in the background, separate from the web page. Enables offline functionality by intercepting network requests and serving cached responses.

IndexedDB

A low-level browser API for storing large amounts of structured data. Used for offline telemetry storage in the PWA.

Apollo Client

A comprehensive GraphQL client for JavaScript. Provides caching, state management, and offline support for GraphQL operations.

- Workbox** A set of JavaScript libraries from Google for adding offline support to web apps. Simplifies service worker creation and caching strategies.

Coroutines

A Kotlin feature for asynchronous programming. Allows writing non-blocking code in a sequential style. Used extensively in the Spring Boot gateway for handling concurrent requests.

- Pi4J** A Java library for accessing GPIO, I2C, SPI, and other hardware interfaces on Raspberry Pi. Used by the gateway to communicate with the ESP32 via SPI.

FreeRTOS

A real-time operating system kernel for embedded devices. Used by the ESP32 firmware for task scheduling, ensuring the motor control loop runs at exactly 1 kHz.

1.5 Protocols and Formats

SLAMTEC Protocol

The communication protocol used by RPLiDAR sensors (like the RPLiDAR C1). A proprietary serial protocol over UART for commanding the sensor and receiving scan data, supported by the `rplidar_ros2` driver.

- JSON** **JavaScript Object Notation** – A lightweight, human-readable data interchange format. Used by GraphQL and REST APIs. Larger than binary formats like Protobuf.

HTTP/2 The second major version of HTTP. Supports multiplexing (multiple requests over one connection), header compression, and server push. Required by gRPC.

TCP/IP **Transmission Control Protocol / Internet Protocol** – The foundational communication protocols of the internet. Provides reliable, ordered delivery of data packets.

1.6 Deployment and Operations

systemd The init system and service manager for modern Linux distributions. Manages service lifecycle, auto-restart on failure, and dependency ordering.

Buildroot

A tool for building embedded Linux systems. Used to create the minimal custom Linux image for the Raspberry Pi 5.

Prometheus

An open-source monitoring and alerting toolkit. Collects metrics from applications via HTTP endpoints. Used with Spring Boot Actuator for gateway monitoring.

Actuator A Spring Boot module that exposes operational information (health, metrics, info) via HTTP endpoints. Integrates with Prometheus for metrics collection.

JVM **Java Virtual Machine** – The runtime environment for Java and Kotlin applications. The Spring Boot gateway runs on the JVM.

2 Executive Summary

This document provides a comprehensive technical reference for implementing the STAR robot control system software stack. It covers:

- **Protocol Selection:** gRPC + GraphQL hybrid architecture with detailed justification
- **Backend Gateway:** Kotlin/Spring Boot 3 implementation for RPi5
- **Frontend PWA:** Vue 3 + TypeScript Progressive Web App with full offline capability
- **Hardware Integration:** SPI and TCP communication with ESP32
- **Deployment:** Systemd services, mDNS discovery, and metrics

2.1 Architecture Decisions Summary

Decision	Choice	Rationale
Communication Protocol	gRPC + GraphQL Hybrid	Low latency + flexible queries
Backend Framework	Kotlin/Spring Boot 3	JVM performance + coroutines
Frontend Framework	Vue 3 + TypeScript + Vite	Type-safe, reactive UI with Pinia state
Offline Strategy	Workbox + Apollo Cache	Full offline with IndexedDB
ESP32 Communication	SPI (primary) + TCP (fallback)	Redundancy + flexibility
Service Discovery	Avahi/mDNS	Zero-config networking
Metrics	Prometheus + Actuator	Industry standard

Table 1: Key Architecture Decisions

3 Communication Protocol Analysis

3.1 The Problem Statement

The STAR robot requires communication across three distinct paths:

1. **Controller App** → **RPi5 Gateway**: Over WiFi/Internet (this is the focus)
2. **RPi5 Gateway** → **ESP32**: Over SPI bus (already defined)
3. **RPi5 Gateway** → **LiDAR**: Over UART (already defined)

The critical question is: *What protocol should Path #1 use?*

3.2 Protocol Options Evaluated

Four options were evaluated for controller-to-robot communication:

1. gRPC + GraphQL Hybrid
2. ConnectRPC (modern gRPC alternative)
3. WebSocket + GraphQL
4. Pure gRPC with Envoy Proxy

3.3 Detailed Comparison

Aspect	gRPC + GraphQL	ConnectRPC	WebSocket + GraphQL	Pure gRPC
Latency	gRPC: 1-5ms, GraphQL: 10-30ms	5-15ms	WS: 5-10ms, GQL: 10-30ms	1-5ms
Packet Size	Binary (gRPC) + JSON (GQL)	Binary OR JSON	JSON only	Binary only
Browser Support	gRPC-Web (limited) + GQL (full)	Full (HTTP/1.1)	Full native	Requires Envoy
Bidirectional	gRPC-Web: NO, GQL Sub: YES	NO	YES (native)	gRPC-Web: NO
Infrastructure	2 servers	1 server	1 server + WS	1 server + Envoy
Learning Curve	High (2 systems)	Medium	Medium	Medium-High
Ecosystem	Very mature (both)	Newer (2022+)	Very mature	Very mature

Table 2: Protocol Comparison Matrix

3.4 Protocol Deep Dive

3.4.1 gRPC + GraphQL Hybrid (Selected)

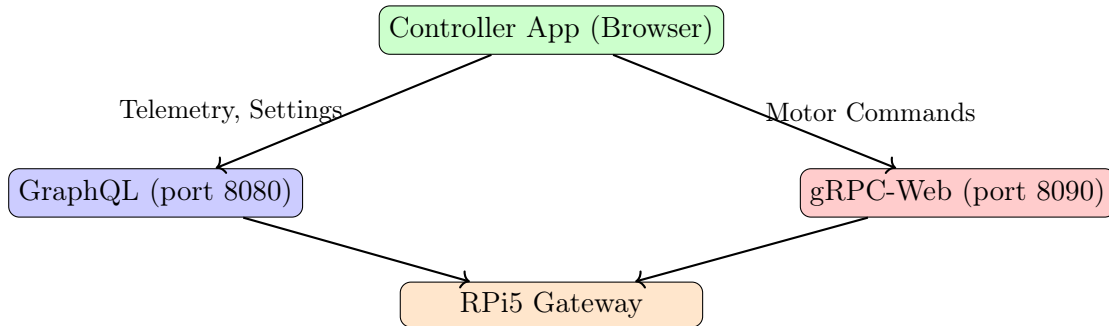


Figure 1: gRPC + GraphQL Hybrid Architecture

Advantages:

- gRPC binary format provides fastest motor command transmission
- GraphQL enables flexible queries for complex UI data needs
- Clear separation of concerns: real-time control vs. data queries
- Both protocols are industry standards with excellent tooling

Disadvantages:

- Two different APIs to maintain
- gRPC-Web requires Envoy proxy as translator
- More complex client code (two client libraries)

3.4.2 Why This Was Selected for STAR

1. **Low-Latency Motor Control:** gRPC's binary Protobuf format minimizes serialization overhead for time-critical motor commands.
2. **Flexible Telemetry Queries:** GraphQL subscriptions provide real-time telemetry streaming with selective field fetching.
3. **Offline Capability:** Apollo Client (GraphQL) has excellent caching and offline support built-in.
4. **Latency Acceptability:** For motor commands, even 10-30ms latency from browser to RPi5 is acceptable because:
 - The ESP32 maintains autonomous 1kHz PID control between commands
 - Human reaction time is approximately 200-300ms
 - The critical real-time path is RPi5 → ESP32 (SPI), not Controller → RPi5

3.5 Packet Format Comparison

3.5.1 Protobuf (gRPC) Binary Encoding

Protobuf uses a compact binary format that excludes field names:

```

1 message VelocityCommand {
2     float left_velocity = 1;    // 4 bytes + tag
3     float right_velocity = 2;   // 4 bytes + tag
4     uint64 timestamp = 3;       // 1-10 bytes (varint)
5 }
```

Listing 1: Protobuf Message Definition

Wire format example (VelocityCommand with left=1.0, right=0.5, timestamp=1702500000000):

```

0D 00 00 80 3F  // field 1 (float): 1.0
15 00 00 00 3F  // field 2 (float): 0.5
18 80 A0 CF EE E5 31  // field 3 (varint): timestamp
```

Total size: approximately 17 bytes

3.5.2 JSON (GraphQL) Text Encoding

Equivalent JSON payload:

```

1 {
2     "leftVelocity": 1.0,
3     "rightVelocity": 0.5,
4     "timestamp": 1702500000000
5 }
```

Listing 2: JSON Equivalent

Total size: approximately 65 bytes (3.8x larger)

3.5.3 Serialization Performance

Based on industry benchmarks¹:

Metric	Protobuf	JSON	Difference
Message Size	17 bytes	65 bytes	3.8x smaller
Serialization (JVM)	1.2 μ s	4.5 μ s	3.75x faster
Deserialization (JVM)	0.8 μ s	3.2 μ s	4x faster
Network Latency (1Mbps)	0.14 ms	0.52 ms	3.7x faster

Table 3: Protobuf vs JSON Performance Comparison

¹Source: Ambassador Labs, Protobuf vs JSON Performance Analysis, 2024

4 System Architecture

4.1 Network Topology

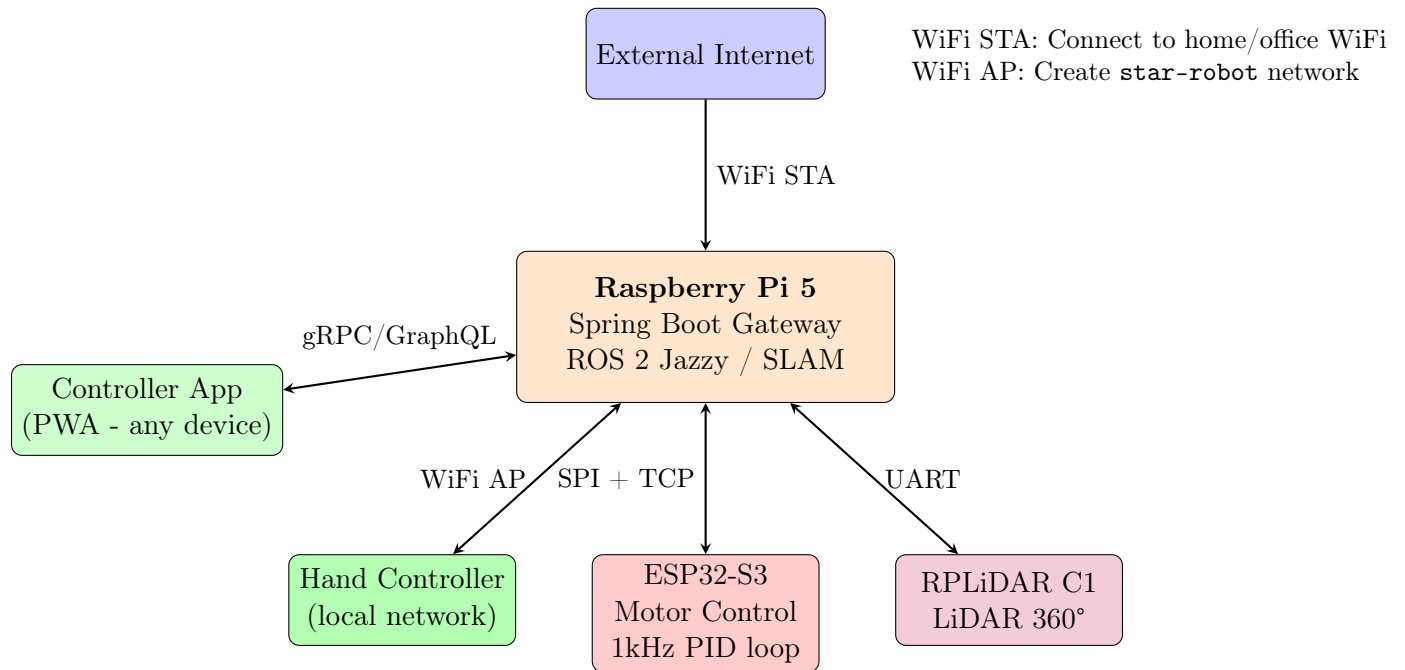


Figure 2: Complete Network Topology

4.2 Dual WiFi Mode Operation

The Raspberry Pi 5 operates in **simultaneous dual WiFi mode**:

1. **Station (STA) Mode:** Connects to external internet for remote access
 - Enables control from anywhere with internet access
 - Allows firmware updates and remote debugging
2. **Access Point (AP) Mode:** Creates local **star-robot** network
 - Provides low-latency local control
 - ESP32 connects to this network for TCP fallback
 - Hand controller connects for direct operation

4.3 Software Stack Overview

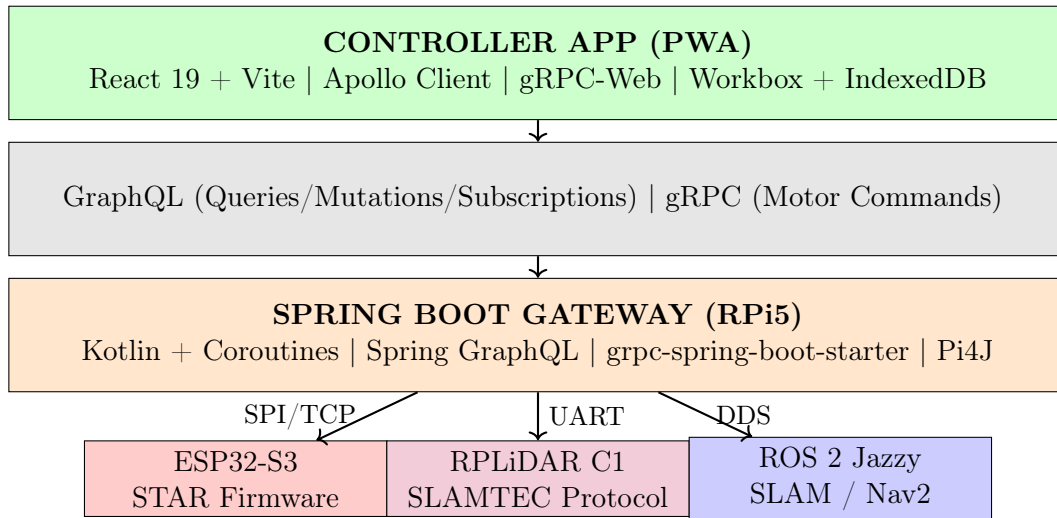


Figure 3: Software Stack Architecture

5 SLAM Algorithm Selection

The STAR robot uses 2D SLAM (Simultaneous Localization and Mapping) for indoor navigation. This section documents the algorithm selection process and rationale.

5.1 Algorithm Comparison

Four SLAM algorithms were evaluated for the STAR platform:

Table 4: SLAM Algorithm Comparison Matrix

Criterion (Weight)	SLAM Toolbox	Cartographer	Hector SLAM	GMapping
Compute Efficiency (0.25)	5	3	4	4
Loop Closure Quality (0.20)	4	5	2	3
Indoor Accuracy (0.25)	5	4	4	4
ROS 2 Support (0.15)	5	4	3	4
Memory Usage (0.15)	4	3	5	4
Weighted Score	4.55	3.75	3.55	3.80

Scoring: 1 = Poor, 2 = Below Average, 3 = Average, 4 = Good, 5 = Excellent

5.2 Selection: SLAM Toolbox

SLAM Toolbox was selected as the primary SLAM algorithm based on:

1. **Compute Efficiency** – Optimized for resource-constrained platforms like RPi5. Achieves 10+ Hz map updates with 360-degree LiDAR scans.
2. **Indoor Accuracy** – Sub-centimeter accuracy in structured indoor environments. Excellent performance with the RPLiDAR C1's 0.72-degree angular resolution.
3. **Native ROS 2 Support** – First-class ROS 2 Humble/Jazzy support. Developed specifically for Nav2 integration.
4. **Flexible Operation Modes:**
 - **mapping** – Create new maps (online SLAM)
 - **localization** – Localize in existing maps (AMCL-like)
 - **lifelong** – Continuous mapping with map updates
5. **Serialization** – Maps can be saved/loaded in multiple formats (PGM, YAML, binary).

5.3 SLAM Toolbox Configuration

```

1 slam_toolbox:
2   ros__parameters:
3     # Solver parameters
4     solver_plugin: solver_plugins::CeresSolver
5     ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
6     ceres_preconditioner: SCHUR_JACOBI

```

```

7   ceres_trust_strategy: LEVENBERG_MARQUARDT
8   ceres_dogleg_type: TRADITIONAL_DOGLEG
9   ceres_loss_function: None
10
11  # Frame IDs
12  odom_frame: odom
13  map_frame: map
14  base_frame: base_link
15  scan_topic: /scan
16  use_scan_matching: true
17  use_scan_barycenter: true
18
19  # Resolution and range
20  resolution: 0.05          # 5cm per cell (matches Nav2 costmap)
21  max_laser_range: 12.0     # RPLiDAR C1 max range
22  minimum_travel_distance: 0.3
23  minimum_travel_heading: 0.3
24
25  # Loop closure
26  do_loop_closing: true
27  loop_search_maximum_distance: 3.0
28  loop_match_minimum_chain_size: 3
29  loop_match_maximum_variance_coarse: 3.0
30  loop_match_minimum_response_coarse: 0.35
31  loop_match_minimum_response_fine: 0.45
32
33  # Performance tuning for RPi5
34  transform_publish_period: 0.05 # 20 Hz TF publish rate
35  map_update_interval: 0.5       # 2 Hz map updates (reduce CPU)
36  throttle_scans: 1              # Process every scan

```

Listing 3: slam_toolbox_params.yaml

5.4 Why Not Other Algorithms?

Cartographer Excellent loop closure but computationally expensive. Requires Lua configuration which adds complexity. Better suited for high-end compute platforms.

Hector SLAM Fast and lightweight but lacks loop closure entirely. Drift accumulates over time in longer sessions. Best for short-duration mapping only.

GMapping Particle filter approach with good accuracy but higher memory usage due to particle storage. ROS 2 port is community-maintained, not official.

5.5 Nav2 DWA Controller Parameters

The Dynamic Window Approach (DWA) local planner requires careful tuning for the STAR robot's physical characteristics:

Table 5: DWA Controller Parameters

Parameter	Value	Description
Velocity Limits		
max_vel_x	0.5 m/s	Maximum forward velocity
min_vel_x	-0.3 m/s	Maximum reverse velocity
max_vel_theta	1.0 rad/s	Maximum angular velocity
min_speed_xy	0.0 m/s	Minimum translational speed
Acceleration Limits		
acc_lim_x	1.0 m/s ²	Linear acceleration limit
acc_lim_theta	2.0 rad/s ²	Angular acceleration limit
decel_lim_x	-1.5 m/s ²	Linear deceleration limit
decel_lim_theta	-2.5 rad/s ²	Angular deceleration limit
Trajectory Scoring		
path_distance_bias	32.0	Weight for staying on global path
goal_distance_bias	24.0	Weight for approaching goal
occdist_scale	0.01	Weight for avoiding obstacles
Costmap		
inflation_radius	0.3 m	Obstacle inflation radius
cost_scaling_factor	3.0	Exponential decay rate

```

1 controller_server:
2   ros_parameters:
3     controller_frequency: 20.0   # Hz
4     FollowPath:
5       plugin: "dwb_core::DWBLocalPlanner"
6       # Velocity limits
7       max_vel_x: 0.5
8       min_vel_x: -0.3
9       max_vel_y: 0.0           # Differential drive - no lateral motion
10      min_vel_y: 0.0
11      max_vel_theta: 1.0
12      min_speed_xy: 0.0
13      max_speed_xy: 0.5
14      min_speed_theta: 0.0
15      # Acceleration limits
16      acc_lim_x: 1.0
17      acc_lim_y: 0.0
18      acc_lim_theta: 2.0
19      decel_lim_x: -1.5
20      decel_lim_y: 0.0
21      decel_lim_theta: -2.5
22      # Trajectory generation
23      vx_samples: 20
24      vy_samples: 1             # Differential drive
25      vtheta_samples: 40
26      sim_time: 1.5             # Trajectory simulation time
27      # Critics (trajectory scoring)
28      critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
29               "GoalAlign", "PathAlign", "PathDist", "GoalDist"]

```

```
30 PathAlign.scale: 32.0
31 GoalAlign.scale: 24.0
32 PathDist.scale: 32.0
33 GoalDist.scale: 24.0
34 BaseObstacle.scale: 0.01
```

Listing 4: nav2_params.yaml (DWA section)

6 ML Inference Pipeline

The STAR robot uses machine learning for person detection as part of its safety system. This section documents the model selection and inference pipeline.

6.1 Model Selection

Table 6: Person Detection Model Comparison

Model	Size (MB)	mAP@0.5	RPi5 FPS	Quantization
YOLOv8n INT8	3.2	0.37	15-18	INT8
YOLOv5n FP16	7.5	0.35	8-10	FP16
MobileNet-SSD	4.8	0.31	12-15	INT8
EfficientDet-Lite0	4.4	0.33	10-12	INT8

YOLOv8n INT8 was selected for its optimal balance of accuracy and performance on the Raspberry Pi 5.

6.2 Model Architecture

- **Base Model:** YOLOv8n (nano variant)
- **Input Resolution:** 320x320 (reduced from 640x640 for speed)
- **Quantization:** INT8 post-training quantization via TensorFlow Lite
- **Classes:** Single class (person) for focused detection
- **Inference Backend:** TensorFlow Lite with XNNPACK delegate

6.3 Inference Pipeline

```

1 import cv2
2 import numpy as np
3 from tflite_runtime.interpreter import Interpreter
4
5 class PersonDetector:
6     def __init__(self, model_path: str = "yolov8n_int8.tflite"):
7         # Load TFLite model with XNNPACK delegate for ARM optimization
8         self.interpreter = Interpreter(
9             model_path=model_path,
10            num_threads=4 # Use all 4 Cortex-A76 cores
11        )
12        self.interpreter.allocate_tensors()
13
14        self.input_details = self.interpreter.get_input_details()
15        self.output_details = self.interpreter.get_output_details()
16        self.input_shape = self.input_details[0]['shape'][1:3] # (320,
17            320)
18
19        # Detection thresholds

```

```

19     self.confidence_threshold = 0.5
20     self.nms_threshold = 0.4
21
22     def preprocess(self, frame: np.ndarray) -> np.ndarray:
23         """Resize and normalize input frame."""
24         resized = cv2.resize(frame, tuple(self.input_shape))
25         # INT8 quantization: scale to 0-255 range
26         normalized = resized.astype(np.uint8)
27         return np.expand_dims(normalized, axis=0)
28
29     def detect(self, frame: np.ndarray) -> list:
30         """Run inference and return person detections."""
31         input_data = self.preprocess(frame)
32         self.interpreter.set_tensor(
33             self.input_details[0]['index'],
34             input_data
35         )
36         self.interpreter.invoke()
37
38         # Parse YOLO output format
39         output = self.interpreter.get_tensor(
40             self.output_details[0]['index']
41         )
42         detections = self._parse_output(output, frame.shape)
43         return detections
44
45     def _parse_output(self, output: np.ndarray,
46                       original_shape: tuple) -> list:
47         """Convert YOLO output to bounding boxes."""
48         detections = []
49         h, w = original_shape[:2]
50         scale_x, scale_y = w / self.input_shape[1], h /
51             self.input_shape[0]
52
53         for detection in output[0]:
54             confidence = detection[4]
55             if confidence < self.confidence_threshold:
56                 continue
57
58             # Extract bounding box (center format to corner format)
59             cx, cy, bw, bh = detection[:4]
60             x1 = int((cx - bw / 2) * scale_x)
61             y1 = int((cy - bh / 2) * scale_y)
62             x2 = int((cx + bw / 2) * scale_x)
63             y2 = int((cy + bh / 2) * scale_y)
64
65             detections.append({
66                 'bbox': (x1, y1, x2, y2),
67                 'confidence': float(confidence),
68                 'class': 'person'
69             })
70
71         # Apply non-maximum suppression
72         return self._nms(detections)

```

```

72
73     def _nms(self, detections: list) -> list:
74         """Non-maximum suppression to remove overlapping boxes."""
75         if not detections:
76             return []
77
78         boxes = np.array([d['bbox'] for d in detections])
79         scores = np.array([d['confidence'] for d in detections])
80
81         indices = cv2.dnn.NMSBoxes(
82             boxes.tolist(),
83             scores.tolist(),
84             self.confidence_threshold,
85             self.nms_threshold
86         )
87
88         return [detections[i] for i in indices.flatten()]

```

Listing 5: person_detector.py

6.4 Performance Optimization

1. **XNNPACK Delegate** – ARM-optimized SIMD operations for 2-3x speedup over default TFLite.
2. **Multi-threading** – Utilize all 4 Cortex-A76 cores via `num_threads=4`.
3. **Reduced Resolution** – 320x320 input (vs 640x640) reduces computation by 4x with minimal accuracy loss for person detection.
4. **Frame Skipping** – Process every 2nd frame (15 FPS effective) to maintain system responsiveness.
5. **ROI Processing** – In localization mode, only process regions where persons were previously detected.

6.5 Integration with Safety System

The person detector feeds into the CV Safety System (Section ??):

```

1  # In main control loop
2  detector = PersonDetector()
3  camera = cv2.VideoCapture(0)
4
5  while True:
6      ret, frame = camera.read()
7      if not ret:
8          continue
9
10     detections = detector.detect(frame)
11
12     for det in detections:
13         # Calculate distance from bounding box size
14         # (calibrated for known person height)

```

```
15     bbox_height = det['bbox'][3] - det['bbox'][1]
16     estimated_distance = estimate_distance(bbox_height)
17
18     # Trigger safety response based on distance
19     if estimated_distance < EMERGENCY_STOP_DISTANCE:
20         robot.emergency_stop()
21     elif estimated_distance < SLOW_ZONE_DISTANCE:
22         robot.reduce_speed(0.5) # 50% max speed
```

Listing 6: Integration with safety pipeline

7 CV Safety System

The computer vision safety system provides an additional layer of protection beyond LiDAR-based obstacle detection. It uses the USB camera and YOLOv8n person detector to identify humans and adjust robot behavior accordingly.

7.1 Safety Zones

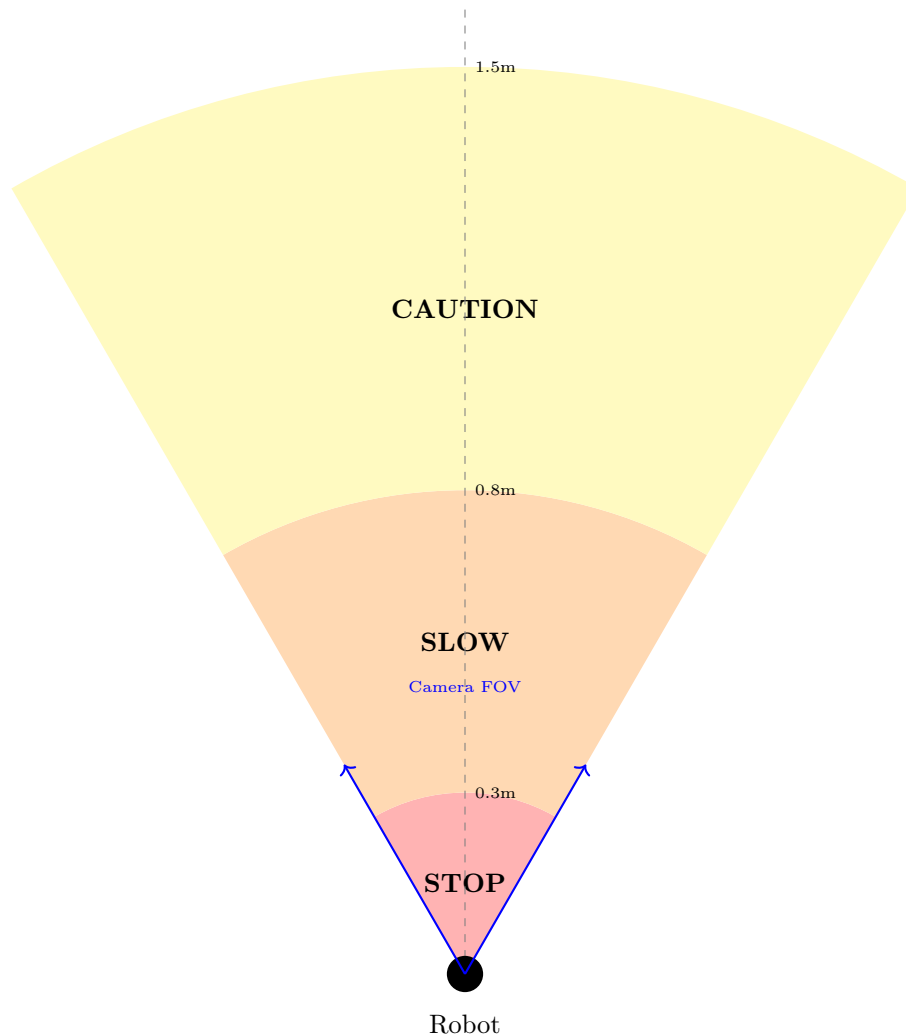


Figure 4: Safety Zone Distances (Top-Down View)

Table 7: Safety Zone Thresholds and Responses

Zone	Distance	Robot Response
Emergency Stop (Red)	< 0.3 m	Immediate stop, hold position
Slow Zone (Orange)	$0.3 - 0.8$ m	Reduce speed to 50% of max
Caution Zone (Yellow)	$0.8 - 1.5$ m	Continue at current speed, log warning
Clear (Green)	> 1.5 m	Normal operation

7.2 Distance Estimation

Since the USB camera provides 2D images without depth information, person distance is estimated using the detected bounding box height and a calibrated person height model:

```

1  # Calibration constants (adjust based on camera focal length and mounting)
2  FOCAL_LENGTH_PIXELS = 600 # Camera focal length in pixels
3  AVERAGE_PERSON_HEIGHT_M = 1.7 # Average adult height in meters
4  CAMERA_HEIGHT_M = 0.3 # Robot camera mounting height
5
6  def estimate_distance(bbox_height_pixels: int,
7                        frame_height: int) -> float:
8      """
9      Estimate distance to person using pinhole camera model.
10
11      distance = (real_height * focal_length) / image_height
12
13      Args:
14          bbox_height_pixels: Height of person bounding box in pixels
15          frame_height: Total frame height in pixels
16
17      Returns:
18          Estimated distance in meters
19      """
20      if bbox_height_pixels <= 0:
21          return float('inf')
22
23      # Pinhole camera model
24      distance = (AVERAGE_PERSON_HEIGHT_M * FOCAL_LENGTH_PIXELS) /
25                  bbox_height_pixels
26
27      # Apply height correction (camera is not at ground level)
28      # This is a simplification; more accurate models use full pose
29      estimation
30      distance = max(0.1, distance)
31
32      return distance
33
34  # Calibration procedure:
35  # 1. Place person at known distance (e.g., 1.0m, 2.0m, 3.0m)
36  # 2. Record bounding box heights
37  # 3. Calculate FOCAL_LENGTH_PIXELS = (bbox_height * distance) /
38  person_height
39  # 4. Average across multiple measurements

```

Listing 7: Distance estimation from bounding box

7.3 Safety State Machine

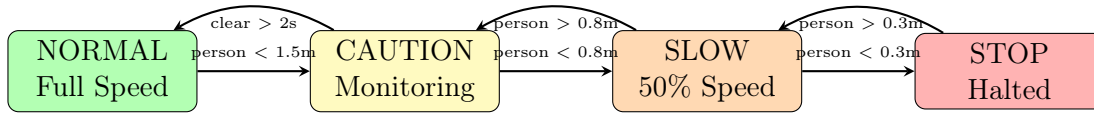


Figure 5: CV Safety State Machine

7.4 Integration with Navigation

The CV safety system overrides navigation commands when necessary:

```

1 from enum import Enum
2 from dataclasses import dataclass
3 from typing import Optional
4 import time
5
6 class SafetyState(Enum):
7     NORMAL = "normal"
8     CAUTION = "caution"
9     SLOW = "slow"
10    STOP = "stop"
11
12 @dataclass
13 class SafetyZones:
14     EMERGENCY_STOP: float = 0.3    # meters
15     SLOW_ZONE: float = 0.8          # meters
16     CAUTION_ZONE: float = 1.5       # meters
17     CLEAR_TIMEOUT: float = 2.0      # seconds before returning to normal
18
19 class SafetyController:
20     def __init__(self):
21         self.state = SafetyState.NORMAL
22         self.last_detection_time: Optional[float] = None
23         self.speed_multiplier = 1.0
24
25     def update(self, person_distance: Optional[float]) -> SafetyState:
26         """Update safety state based on nearest person distance."""
27         current_time = time.time()
28
29         if person_distance is None:
30             # No person detected
31             if self.last_detection_time is not None:
32                 elapsed = current_time - self.last_detection_time
33                 if elapsed > SafetyZones.CLEAR_TIMEOUT:
34                     self._transition_to(SafetyState.NORMAL)
35             return self.state
36
37         self.last_detection_time = current_time
38
39         # Determine appropriate state based on distance
40         if person_distance < SafetyZones.EMERGENCY_STOP:
41             self._transition_to(SafetyState.STOP)

```

```
42     elif person_distance < SafetyZones.SLOW_ZONE:
43         self._transition_to(SafetyState.SLOW)
44     elif person_distance < SafetyZones.CAUTION_ZONE:
45         self._transition_to(SafetyState.CAUTION)
46
47     return self.state
48
49     def _transition_to(self, new_state: SafetyState):
50         if new_state != self.state:
51             print(f"Safety state: {self.state.value} ->
52                   {new_state.value}")
53             self.state = new_state
54             self._update_speed_multiplier()
55
56     def _update_speed_multiplier(self):
57         multipliers = {
58             SafetyState.NORMAL: 1.0,
59             SafetyState.CAUTION: 1.0,
60             SafetyState.SLOW: 0.5,
61             SafetyState.STOP: 0.0,
62         }
63         self.speed_multiplier = multipliers[self.state]
64
65     def apply_to_velocity(self, cmd_vel: tuple) -> tuple:
66         """Apply safety multiplier to velocity command."""
67         linear, angular = cmd_vel
68         return (linear * self.speed_multiplier,
69               angular * self.speed_multiplier)
```

Listing 8: SafetyController class

8 Battery Management

The STAR robot implements a multi-threshold battery management system to ensure safe operation and prevent damage from deep discharge.

8.1 Battery State Thresholds

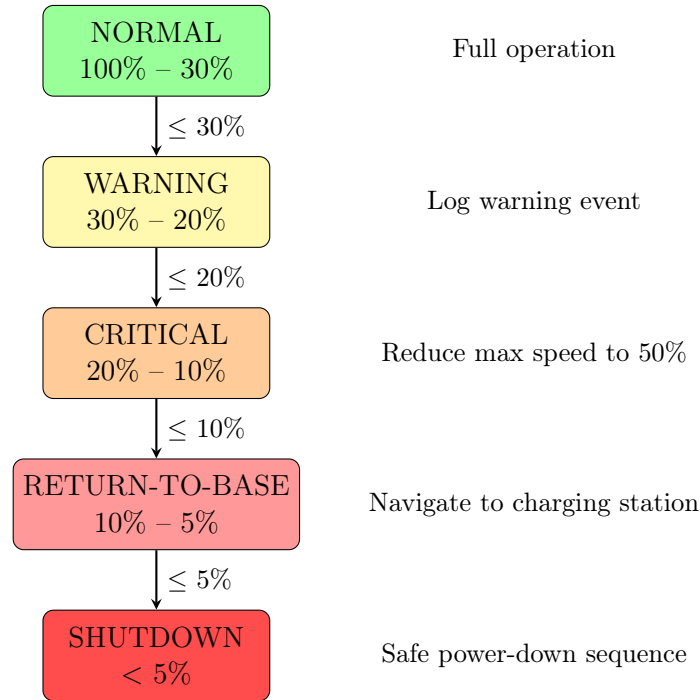


Figure 6: Battery State Machine

Table 8: Battery Management Thresholds

Level	State	Action
100% – 30%	Normal	Full operation, no restrictions
30%	Warning	Log battery warning event to telemetry
20%	Critical	Reduce maximum speed to 50%
10%	Return-to-Base	Abort current task, navigate to charging station
5%	Emergency	Initiate safe shutdown sequence

8.2 Battery Monitoring

Battery voltage and state-of-charge are monitored via the ESP32's ADC with filtering:

```

1 #include <esp_adc_cal.h>
2
3 #define BATTERY_ADC_CHANNEL    ADC1_CHANNEL_6    // GPIO34
4 #define BATTERY_SAMPLES        16                // Averaging samples
5 #define VOLTAGE_DIVIDER_RATIO  4.0                // R1=30k, R2=10k
6 #define BATTERY_MIN_V          10.0              // 3S LiPo empty

```

```

7  #define BATTERY_MAX_V          12.6          // 3S LiPo full
8
9  typedef enum {
10     BATTERY_NORMAL,
11     BATTERY_WARNING,
12     BATTERY_CRITICAL,
13     BATTERY_RTb,
14     BATTERY_SHUTDOWN
15 } battery_state_t;
16
17 static esp_adc_cal_characteristics_t adc_chars;
18 static battery_state_t current_state = BATTERY_NORMAL;
19
20 void battery_init(void) {
21     adc1_config_width(ADC_WIDTH_BIT_12);
22     adc1_config_channel_atten(BATTERY_ADC_CHANNEL, ADC_ATTEN_DB_11);
23     esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_11,
24                             ADC_WIDTH_BIT_12, 1100, &adc_chars);
25 }
26
27 float battery_read_voltage(void) {
28     uint32_t adc_sum = 0;
29
30     // Multi-sample averaging for noise reduction
31     for (int i = 0; i < BATTERY_SAMPLES; i++) {
32         adc_sum += adc1_get_raw(BATTERY_ADC_CHANNEL);
33     }
34     uint32_t adc_avg = adc_sum / BATTERY_SAMPLES;
35
36     // Convert to voltage
37     uint32_t voltage_mv = esp_adc_cal_raw_to_voltage(adc_avg, &adc_chars);
38
39     return (voltage_mv / 1000.0f) * VOLTAGE_DIVIDER_RATIO;
40 }
41
42 uint8_t battery_get_percentage(float voltage) {
43     // Linear approximation (for LiPo, use lookup table for accuracy)
44     float pct = (voltage - BATTERY_MIN_V) / (BATTERY_MAX_V -
45         BATTERY_MIN_V);
46     pct = fmaxf(0.0f, fminf(1.0f, pct));
47     return (uint8_t)(pct * 100.0f);
48 }
49
50 battery_state_t battery_update_state(uint8_t percentage) {
51     if (percentage <= 5) {
52         current_state = BATTERY_SHUTDOWN;
53     } else if (percentage <= 10) {
54         current_state = BATTERY_RTb;
55     } else if (percentage <= 20) {
56         current_state = BATTERY_CRITICAL;
57     } else if (percentage <= 30) {
58         current_state = BATTERY_WARNING;
59     } else {
60         current_state = BATTERY_NORMAL;
61     }
62 }

```

```
60     }  
61     return current_state;  
62 }
```

Listing 9: Battery monitoring (ESP32)

8.3 Safe Shutdown Sequence

When battery reaches critical level (5%), the system initiates a controlled shutdown:

1. **Stop Motors** – Immediate motor disable, engage brakes if available
2. **Log Position** – Save current map position to persistent storage
3. **Notify User** – Send critical battery notification via GraphQL subscription
4. **Save State** – Persist navigation state, waypoints, and settings to flash
5. **Shutdown Sequence** – Graceful ROS 2 node shutdown, then Linux poweroff

9 Repository Structure

9.1 Directory Layout

```

1 /STAR/
2 |-- ESP32-Firmware/           # Existing - keep as-is
3 |-- star-pi5-os/             # Existing - add gateway to Buildroot
4 |-- Schematic/               # Existing - hardware designs
5 |-- Locked_Inc_FTP/          # Existing - documentation
6 |-- docs/                    # Existing - documentation
7 |
8 |-- star-gateway/            # NEW: Spring Boot Gateway
9 |   |-- build.gradle.kts
10 |   |-- settings.gradle.kts
11 |   |-- src/
12 |   |   +-- main/
13 |   |       |-- kotlin/com/star/gateway/
14 |   |           |-- StarGatewayApplication.kt
15 |   |           |-- config/
16 |   |               |-- GraphQLConfig.kt
17 |   |               |-- GrpcConfig.kt
18 |   |               +-- CorsConfig.kt
19 |   |               |-- graphql/
20 |   |                   |-- TelemetryController.kt
21 |   |                   |-- SettingsController.kt
22 |   |                   +-- NavigationController.kt
23 |   |                   |-- grpc/
24 |   |                       +-- MotorControlGrpcService.kt
25 |   |                       |-- service/
26 |   |                           |-- MotorControlService.kt
27 |   |                           |-- TelemetryService.kt
28 |   |                           |-- NavigationService.kt
29 |   |                           +-- SensorFusionService.kt
30 |   |                           |-- hardware/
31 |   |                               |-- Esp32SpiAdapter.kt
32 |   |                               |-- Esp32TcpAdapter.kt
33 |   |                               |-- RplidarAdapter.kt
34 |   |                               +-- Ros2Bridge.kt
35 |   |                               +-- model/
36 |   |                                   |-- TelemetryData.kt
37 |   |                                   |-- MotorCommand.kt
38 |   |                                   +-- NavigationState.kt
39 |   |                                   |-- proto/
40 |   |                                       +-- motor_control.proto
41 |   |                                   +-- resources/
42 |   |                                       |-- application.yml
43 |   |                                       +-- graphql/
44 |   |                                           +-- schema.graphqls
45 |   +-- deployment/
46 |       |-- star-gateway.service    # Systemd unit
47 |       +-- avahi/
48 |           +-- star-robot.service  # mDNS config
49 |

```

```

50 |-- star-ui/                                # NEW: PWA (moved from UI-Demo)
51 |   |-- package.json
52 |   |-- vite.config.js
53 |   |-- public/
54 |       |-- manifest.json
55 |       +-- sw.js
56 |   |-- src/
57 |       |-- main.jsx
58 |       |-- App.jsx
59 |       |-- components/                    # From UI-Demo
60 |       |-- pages/                        # From UI-Demo
61 |       |-- hooks/
62 |           |-- useRobotConnection.js
63 |           |-- useTelemetry.js
64 |           +-- useMotorControl.js
65 |       |-- graphql/
66 |           |-- client.js
67 |           |-- queries.js
68 |           |-- mutations.js
69 |           +-- subscriptions.js
70 |       |-- grpc/
71 |           |-- client.js
72 |           +-- motor_control_pb.js
73 |       |-- service-worker/
74 |           |-- sw.js
75 |           +-- offline-cache.js
76 |       +-- storage/
77 |           +-- indexedDB.js
78 |       +-- workbox-config.js
79 |
80 +-- proto/                                # SHARED: Protobuf definitions
81 |   +-- star/
82 |       |-- motor_control.proto
83 |       |-- telemetry.proto
84 |       +-- navigation.proto

```

Listing 10: Complete Repository Structure

10 Spring Boot Gateway Implementation

10.1 Build Configuration

```
1 plugins {
2     kotlin("jvm") version "1.9.22"
3     kotlin("plugin.spring") version "1.9.22"
4     id("org.springframework.boot") version "3.2.4"
5     id("io.spring.dependency-management") version "1.1.4"
6     id("com.google.protobuf") version "0.9.4"
7 }
8
9 java {
10     sourceCompatibility = JavaVersion.VERSION_17
11 }
12
13 repositories {
14     mavenCentral()
15 }
16
17 dependencies {
18     // Spring Boot Core
19     implementation("org.springframework.boot:spring-boot-starter-webflux")
20     implementation("org.springframework.boot:spring-boot-starter-actuator")
21
22     // GraphQL
23     implementation("org.springframework.boot:spring-boot-starter-graphql")
24
25     // gRPC
26     implementation("net.devh:grpc-spring-boot-starter:3.1.0.RELEASE")
27     implementation("io.grpc:grpc-kotlin-stub:1.4.1")
28     implementation("io.grpc:grpc-protobuf:1.60.1")
29     implementation("com.google.protobuf:protobuf-kotlin:3.25.2")
30
31     // Kotlin Coroutines
32     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.8.0")
33     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:1.8.0")
34     implementation("io.projectreactor.kotlin:reactor-kotlin-extensions")
35
36     // Hardware Access (Pi4J for SPI/GPIO)
37     // Pi4J 2.6+ required for Raspberry Pi 5 (RP1 chip support via GpioD)
38     implementation("com.pi4j:pi4j-core:2.6.1")
39     implementation("com.pi4j:pi4j-plugin-raspberrypi:2.6.1")
40     implementation("com.pi4j:pi4j-plugin-gpiod:2.6.1") // Required for
41     // RPi5
42     implementation("com.pi4j:pi4j-plugin-linuxfs:2.6.1")
43
44     // Metrics
45     implementation("io.micrometer:micrometer-registry-prometheus")
46
47     // JSON Processing
48     implementation("com.fasterxml.jackson.module:jackson-module-kotlin")
```

```
49 // Kotlin Reflection
50 implementation("org.jetbrains.kotlin:kotlin-reflect")
51
52 // Testing
53 testImplementation("org.springframework.boot:spring-boot-starter-test")
54 testImplementation("io.projectreactor:reactor-test")
55 }
56
57 protobuf {
58     protoc {
59         artifact = "com.google.protobuf:protoc:3.25.2"
60     }
61     plugins {
62         id("grpc") {
63             artifact = "io.grpc:protoc-gen-grpc-java:1.60.1"
64         }
65         id("grpc_kotlin") {
66             artifact = "io.grpc:protoc-gen-grpc-kotlin:1.4.1:jdk8@jar"
67         }
68     }
69     generateProtoTasks {
70         all().forEach {
71             it.plugins {
72                 id("grpc")
73                 id("grpc_kotlin")
74             }
75             it.builtins {
76                 id("kotlin")
77             }
78         }
79     }
80 }
81
82 tasks.withType<org.jetbrains.kotlin.gradle.tasks.KotlinCompile> {
83     kotlinOptions {
84         freeCompilerArgs += "-Xjsr305=strict"
85         jvmTarget = "17"
86     }
87 }
```

Listing 11: build.gradle.kts

10.2 Application Configuration

```
1 spring:
2   application:
3     name: star-gateway
4   graphql:
5     graphiql:
6       enabled: true
7       path: /graphiql
8   websocket:
9     path: /graphql-ws
```

```
10     schema:
11       locations: classpath:graphql/
12     main:
13       banner-mode: off
14
15     server:
16       port: 8080
17
18     grpc:
19       server:
20         port: 9090
21
22     # Hardware configuration
23     star:
24       esp32:
25         spi:
26           enabled: true
27           bus: 0
28           chip-select: 0
29           speed-hz: 10000000 # 10 MHz for motor control
30           mode: 0
31         tcp:
32           enabled: true
33           host: 192.168.4.2 # ESP32 IP on AP network
34           port: 5000
35           timeout-ms: 100
36           retry-attempts: 3
37       lidar:
38         serial-port: /dev/ttyAMA0
39         baud-rate: 460800
40         protocol: SLAMTEC
41         scan-rate-hz: 10
42
43     # Actuator endpoints for health checks
44     management:
45       endpoints:
46         web:
47           exposure:
48             include: health,prometheus,info,metrics
49       endpoint:
50         health:
51           show-details: always
52       metrics:
53         tags:
54           application: star-gateway
55
56     logging:
57       level:
58         com.star.gateway: DEBUG
59         org.springframework.graphql: INFO
```

Listing 12: application.yml

10.3 Six-Layer Software Architecture

The STAR system follows a strict six-layer architecture spanning from the remote UI to embedded firmware:

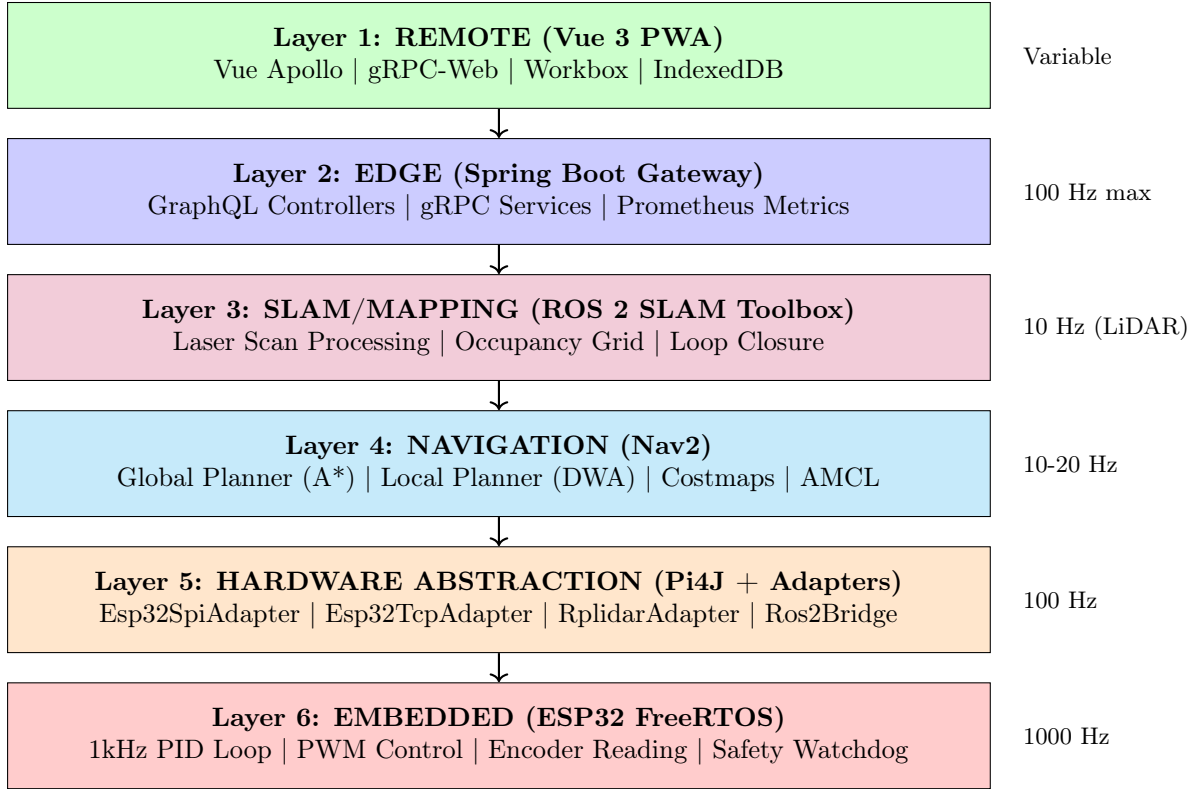


Figure 7: Six-Layer Software Architecture with Update Frequencies

10.3.1 Layer Responsibilities

Layer 1 - Remote (Vue 3 PWA)

User interface running in browser or as installed PWA. Handles user input, displays telemetry, provides offline capability via service workers and IndexedDB.

Layer 2 - Edge (Spring Boot Gateway)

API gateway running on RPi5. Translates between web protocols (GraphQL/gRPC) and robot internals. Manages authentication, rate limiting, and metrics.

Layer 3 - SLAM/Mapping (ROS 2 SLAM Toolbox)

Processes LiDAR scans to build occupancy grid maps. Handles loop closure detection and map optimization. Updates at LiDAR scan rate (10 Hz).

Layer 4 - Navigation (Nav2)

Path planning and obstacle avoidance. Global planner (A*) runs at 10 Hz, local planner (DWA) runs at 20 Hz. Publishes velocity commands to Layer 5.

Layer 5 - Hardware Abstraction (Pi4J + Adapters)

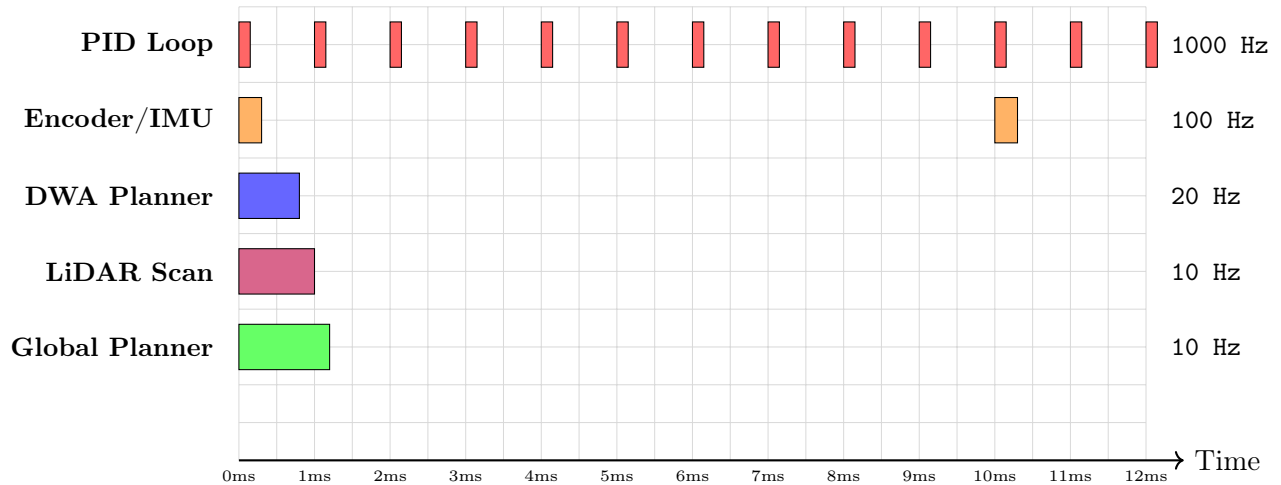
Interfaces between ROS 2 and physical hardware. Communicates with ESP32 via SPI (primary) and TCP (fallback). Reads LiDAR via UART (rplidar_ros2 driver).

Layer 6 - Embedded (ESP32 FreeRTOS)

Real-time motor control at 1 kHz. Runs PID loops, reads encoders, generates PWM signals. Maintains safe operation even if higher layers fail.

10.3.2 Control Timing Hierarchy

The STAR robot operates with a multi-rate control hierarchy, where each layer runs at a frequency appropriate to its function:



Note: Diagram shows 12ms window. Lower frequency tasks extend beyond visible range.

Figure 8: Control Timing Hierarchy (12ms Window)

Table 9: Control Loop Frequencies and Responsibilities

Control Loop	Frequency	Period	Processor
PID Motor Control	1000 Hz	1 ms	ESP32 (FreeRTOS)
Encoder/IMU Sampling	100 Hz	10 ms	ESP32 (FreeRTOS)
DWA Local Planner	20 Hz	50 ms	RPi5 (Nav2)
LiDAR Scan Processing	10 Hz	100 ms	RPi5 (ROS 2)
Global Path Planner	10 Hz	100 ms	RPi5 (Nav2)
Telemetry Publishing	10 Hz	100 ms	RPi5 (Spring Boot)
Map Updates	2 Hz	500 ms	RPi5 (SLAM Toolbox)

Timing Design Rationale:

- **1000 Hz PID** – Required for smooth motor control and quick response to disturbances. FreeRTOS ensures deterministic timing.
- **100 Hz Sensor Sampling** – Captures high-frequency vibrations and provides accurate velocity estimates via differentiation.
- **20 Hz DWA** – Fast enough for obstacle avoidance at max velocity ($0.5 \text{ m/s} = 2.5\text{cm per cycle}$), slow enough to not overload CPU.

- **10 Hz LiDAR** – Fixed by RPLiDAR C1 hardware. Sufficient for indoor navigation at walking speeds.
- **10 Hz Global Planner** – Path replanning happens infrequently; 10 Hz allows quick re-routing while minimizing CPU load.

10.4 GraphQL Schema

```
1  # Query operations - read-only data fetching
2  type Query {
3    telemetry: Telemetry!
4    systemStatus: SystemStatus!
5    settings: Settings!
6    navigationState: NavigationState!
7    telemetryHistory(limit: Int = 100): [TelemetrySnapshot!]!
8  }
9
10 # Mutation operations - state changes
11 type Mutation {
12   setMode(mode: RobotMode!): Boolean!
13   emergencyStop: Boolean!
14   setSettings(input: SettingsInput!): Settings!
15   addWaypoint(x: Float!, y: Float!): Waypoint!
16   clearWaypoints: Boolean!
17   startNavigation: Boolean!
18   stopNavigation: Boolean!
19 }
20
21 # Subscription operations - real-time streams
22 type Subscription {
23   telemetryStream: Telemetry!
24   lidarScan: LidarScan!
25   eventLog: Event!
26 }
27
28 # Core telemetry data structure
29 type Telemetry {
30   imu: ImuData!
31   gps: GpsData!
32   battery: Float!
33   wifiSignal: Int!
34   cpuUsage: Float!
35   temperature: Float!
36   motorLoad: Float!
37   timestamp: String!
38 }
39
40 type ImuData {
41   pitch: Float!
42   roll: Float!
43   yaw: Float!
44   velocity: Float!
45   accelerationX: Float!
```

```
46     accelerationY: Float!
47     accelerationZ: Float!
48 }
49
50 type GpsData {
51     latitude: Float!
52     longitude: Float!
53     altitude: Float!
54 }
55
56 type LidarScan {
57     points: [LidarPoint!]!
58     timestamp: String!
59 }
60
61 type LidarPoint {
62     angle: Float!
63     distance: Float!
64     intensity: Float!
65 }
66
67 type SystemStatus {
68     connectionStatus: ConnectionStatus!
69     mode: RobotMode!
70     esp32Connected: Boolean!
71     lidarConnected: Boolean!
72     rosConnected: Boolean!
73 }
74
75 type NavigationState {
76     currentPosition: Position!
77     path: [Position!]!
78     waypoints: [Waypoint!]!
79     isNavigating: Boolean!
80 }
81
82 type Position {
83     x: Float!
84     y: Float!
85     heading: Float!
86 }
87
88 type Waypoint {
89     id: ID!
90     x: Float!
91     y: Float!
92     label: String
93 }
94
95 type Event {
96     timestamp: String!
97     message: String!
98     type: EventType!
99 }
```

```
100
101 type Settings {
102     maxSpeed: Float!
103     safetyLockEnabled: Boolean!
104     autonomousEnabled: Boolean!
105     lidarEnabled: Boolean!
106 }
107
108 input SettingsInput {
109     maxSpeed: Float
110     safetyLockEnabled: Boolean
111     autonomousEnabled: Boolean
112     lidarEnabled: Boolean
113 }
114
115 enum RobotMode {
116     IDLE
117     MANUAL
118     AUTONOMOUS
119     MAPPING
120     EMERGENCY_STOP
121 }
122
123 enum ConnectionStatus {
124     CONNECTED
125     DISCONNECTED
126     CONNECTING
127     ERROR
128 }
129
130 enum EventType {
131     INFO
132     SUCCESS
133     WARNING
134     ERROR
135 }
```

Listing 13: schema.graphqls

10.5 gRPC Protobuf Definition

```
1 syntax = "proto3";
2
3 package star.motor;
4
5 option java_package = "com.star.gateway.proto";
6 option java_outer_classname = "MotorControlProto";
7 option java_multiple_files = true;
8
9 // Motor control service for low-latency commands
10 service MotorControl {
11     // Unary RPCs (single request-response)
12     rpc SetVelocity(VelocityCommand) returns (CommandResponse);
```

```

13  rpc EmergencyStop(EmptyRequest) returns (CommandResponse);
14  rpc SetMotorPower(MotorPowerCommand) returns (CommandResponse);
15
16  // Server streaming (continuous encoder feedback)
17  rpc StreamEncoderData(EmptyRequest) returns (stream EncoderData);
18 }
19
20 // Velocity command for differential drive
21 message VelocityCommand {
22     float left_velocity = 1;    // m/s, range: -2.0 to 2.0
23     float right_velocity = 2;   // m/s, range: -2.0 to 2.0
24     uint64 timestamp = 3;       // Unix timestamp in milliseconds
25 }
26
27 // Direct motor power control
28 message MotorPowerCommand {
29     int32 motor_id = 1;         // Motor index: 0-3
30     float power = 2;            // Power level: -1.0 to 1.0
31 }
32
33 // Standard command response
34 message CommandResponse {
35     bool success = 1;           // True if command executed
36     string message = 2;         // Human-readable status
37     uint64 latency_us = 3;      // Round-trip latency in microseconds
38 }
39
40 // Real-time encoder data
41 message EncoderData {
42     int32 motor_id = 1;         // Motor index: 0-3
43     int64 ticks = 2;            // Absolute encoder position
44     float velocity = 3;         // Current velocity in m/s
45     uint64 timestamp = 4;       // Unix timestamp in milliseconds
46 }
47
48 // Empty request placeholder
49 message EmptyRequest {}

```

Listing 14: motor_control.proto

10.6 Service Implementations

10.6.1 Motor Control Service

```

1  package com.star.gateway.service
2
3  import com.star.gateway.hardware.Esp32SpiAdapter
4  import com.star.gateway.hardware.Esp32TcpAdapter
5  import com.star.gateway.model.CommandResult
6  import io.micrometer.core.instrument.MeterRegistry
7  import io.micrometer.core.instrument.Timer
8  import kotlinx.coroutines.async
9  import kotlinx.coroutines.coroutineScope

```

```

10 import org.slf4j.LoggerFactory
11 import org.springframework.stereotype.Service
12 import java.util.concurrent.TimeUnit
13
14 @Service
15 class MotorControlService(
16     private val esp32SpiAdapter: Esp32SpiAdapter,
17     private val esp32TcpAdapter: Esp32TcpAdapter,
18     private val meterRegistry: MeterRegistry
19 ) {
20     private val logger = LoggerFactory.getLogger(javaClass)
21
22     // Metrics
23     private val commandCounter =
24         meterRegistry.counter("motor.commands.total")
25     private val spiErrorCounter =
26         meterRegistry.counter("motor.spi.errors")
27     private val tcpFallbackCounter =
28         meterRegistry.counter("motor.tcp.fallbacks")
29     private val latencyTimer: Timer =
30         Timer.builder("motor.command.latency")
31             .description("Motor command round-trip latency")
32             .register(meterRegistry)
33
34     /**
35      * Send velocity command to ESP32.
36      * Primary: SPI (lowest latency)
37      * Fallback: TCP (if SPI fails)
38      */
39     suspend fun setVelocity(left: Float, right: Float): CommandResult {
40         val startTime = System.nanoTime()
41
42         return try {
43             val result = esp32SpiAdapter.sendVelocityCommand(left, right)
44             commandCounter.increment()
45             latencyTimer.record(System.nanoTime() - startTime,
46                 TimeUnit.NANOSECONDS)
47             result
48         } catch (e: Exception) {
49             logger.warn("SPI failed, falling back to TCP: ${e.message}")
50             spiErrorCounter.increment()
51             tcpFallbackCounter.increment()
52
53             try {
54                 esp32TcpAdapter.sendVelocityCommand(left, right)
55             } catch (tcpException: Exception) {
56                 logger.error("TCP fallback also failed:
57                     ${tcpException.message}")
58                 CommandResult.failure("Both SPI and TCP communication
59                     failed")
60             }
61         }
62     }
63 }

```

```

57  /**
58   * Emergency stop - send to BOTH channels simultaneously for
59   * redundancy.
60   * Returns success if either channel succeeds.
61   */
62  suspend fun emergencyStop(): CommandResult {
63      logger.warn("EMERGENCY STOP triggered")
64
65      return coroutineScope {
66          val spiDeferred = async {
67              runCatching { esp32SpiAdapter.emergencyStop() }
68          }
69          val tcpDeferred = async {
70              runCatching { esp32TcpAdapter.emergencyStop() }
71          }
72
73          val spiResult = spiDeferred.await()
74          val tcpResult = tcpDeferred.await()
75
76          when {
77              spiResult.isSuccess && tcpResult.isSuccess -> {
78                  logger.info("Emergency stop confirmed on both
79                      channels")
80                  CommandResult.success("Emergency stop executed on
81                      both channels")
82              }
83              spiResult.isSuccess || tcpResult.isSuccess -> {
84                  logger.warn("Emergency stop partially successful")
85                  CommandResult.success("Emergency stop executed
86                      (partial)")
87              }
88              else -> {
89                  logger.error("Emergency stop FAILED on both
90                      channels!")
91                  CommandResult.failure("Emergency stop failed on both
92                      channels")
93              }
94          }
95      }
96  }
97
98  /**
99   * Get current connection status for both channels.
100  */
101  fun getConnectionStatus(): Map<String, Boolean> {
102      return mapOf(
103          "spi" to esp32SpiAdapter.isConnected(),
104          "tcp" to esp32TcpAdapter.isConnected()
105      )
106  }

```

Listing 15: MotorControlService.kt

10.6.2 ESP32 SPI Adapter

```

1 package com.star.gateway.hardware
2
3 import com.pi4j.Pi4J
4 import com.pi4j.context.Context
5 import com.pi4j.io.spi.Spi
6 import com.pi4j.io.spi.SpiMode
7 import com.star.gateway.model.CommandResult
8 import org.slf4j.LoggerFactory
9 import org.springframework.beans.factory.annotation.Value
10 import org.springframework.stereotype.Component
11 import java.nio.ByteBuffer
12 import java.nio.ByteOrder
13 import javax.annotation.PostConstruct
14 import javax.annotation.PreDestroy
15
16 @Component
17 class Esp32SpiAdapter(
18     @Value("${star.esp32.spi.enabled:true}")
19     private val enabled: Boolean,
20
21     @Value("${star.esp32.spi.bus:0}")
22     private val spiBus: Int,
23
24     @Value("${star.esp32.spi.chip-select:0}")
25     private val chipSelect: Int,
26
27     @Value("${star.esp32.spi.speed-hz:10000000}") // 10 MHz default
28     private val speedHz: Int,
29
30     @Value("${star.esp32.spi.mode:0}")
31     private val spiModeValue: Int
32 ) {
33     private val logger = LoggerFactory.getLogger(javaClass)
34
35     private var pi4j: Context? = null
36     private var spi: Spi? = null
37     private var connected = false
38
39     // SPI Command bytes
40     companion object {
41         const val CMD_SET_VELOCITY: Byte = 0x01
42         const val CMD_EMERGENCY_STOP: Byte = 0x02
43         const val CMD_GET_ENCODERS: Byte = 0x03
44         const val CMD_SET_MOTOR: Byte = 0x04
45         const val CMD_PING: Byte = 0x05
46
47         const val RESPONSE_OK: Byte = 0x00
48         const val RESPONSE_ERROR: Byte = 0x01
49     }
50
51     @PostConstruct
52     fun initialize() {

```

```

53     if (!enabled) {
54         logger.info("SPI adapter disabled by configuration")
55         return
56     }
57
58     try {
59         pi4j = Pi4J.newAutoContext()
60
61         val spiConfig = Spi.newConfigBuilder(pi4j)
62             .id("esp32-spi")
63             .name("ESP32 Motor Controller")
64             .bus(spiBus)
65             .chipSelect(chipSelect)
66             .baud(speedHz)
67             .mode(SpiMode.entries[spiModeValue])
68             .build()
69
70         spi = pi4j?.create(spiConfig)
71         connected = testConnection()
72
73         logger.info("SPI adapter initialized: bus=$spiBus,
74             cs=$chipSelect, speed=$speedHz Hz")
75     } catch (e: Exception) {
76         logger.error("Failed to initialize SPI adapter: ${e.message}")
77         connected = false
78     }
79
80     @PreDestroy
81     fun shutdown() {
82         spi?.close()
83         pi4j?.shutdown()
84         logger.info("SPI adapter shutdown complete")
85     }
86
87     /**
88     * Send velocity command.
89     * Packet format: [CMD_SET_VELOCITY (1), left_velocity (4),
90     * right_velocity (4)]
91     * Response: [status (1)]
92     */
93     suspend fun sendVelocityCommand(left: Float, right: Float):
94         CommandResult {
95         CommandResult {
96             if (!enabled || spi == null) {
97                 throw IllegalStateException("SPI not initialized")
98             }
99
100             val buffer = ByteBuffer.allocate(9)
101                 .order(ByteOrder.LITTLE_ENDIAN)
102                 .put(CMD_SET_VELOCITY)
103                 .putFloat(left)
104                 .putFloat(right)
105
106             val txData = buffer.array()

```

```

104     val rxData = ByteArray(2)
105
106     synchronized(this) {
107         spi!!.transfer(txData, rxData)
108     }
109
110     return if (rxData[0] == RESPONSE_OK) {
111         CommandResult.success("Velocity set: left=$left,
112                                right=$right")
113     } else {
114         CommandResult.failure("ESP32 error code: ${rxData[0]}")
115     }
116
117     /**
118     * Send emergency stop command.
119     * Packet format: [CMD_EMERGENCY_STOP (1)]
120     * Response: [status (1)]
121     */
122     suspend fun emergencyStop(): CommandResult {
123         if (!enabled || spi == null) {
124             throw IllegalStateException("SPI not initialized")
125         }
126
127         val txData = byteArrayOf(CMD_EMERGENCY_STOP)
128         val rxData = ByteArray(2)
129
130         synchronized(this) {
131             spi!!.transfer(txData, rxData)
132         }
133
134         return if (rxData[0] == RESPONSE_OK) {
135             CommandResult.success("Emergency stop executed")
136         } else {
137             CommandResult.failure("Emergency stop failed: ${rxData[0]}")
138         }
139     }
140
141     /**
142     * Test SPI connection with ping command.
143     */
144     private fun testConnection(): Boolean {
145         return try {
146             val txData = byteArrayOf(CMD_PING)
147             val rxData = ByteArray(2)
148             spi?.transfer(txData, rxData)
149             rxData[0] == RESPONSE_OK
150         } catch (e: Exception) {
151             logger.warn("SPI connection test failed: ${e.message}")
152             false
153         }
154     }
155
156     fun isConnected(): Boolean = connected && enabled

```

157 }

Listing 16: Esp32SpiAdapter.kt

10.6.3 ESP32 TCP Adapter

```

1 package com.star.gateway.hardware
2
3 import com.fasterxml.jackson.databind.ObjectMapper
4 import com.star.gateway.model.CommandResult
5 import kotlinx.coroutines.Dispatchers
6 import kotlinx.coroutines.withContext
7 import kotlinx.coroutines.withTimeoutOrNull
8 import org.slf4j.LoggerFactory
9 import org.springframework.beans.factory.annotation.Value
10 import org.springframework.stereotype.Component
11 import java.io.BufferedReader
12 import java.io.InputStreamReader
13 import java.io.PrintWriter
14 import java.net.InetSocketAddress
15 import java.net.Socket
16
17 @Component
18 class Esp32TcpAdapter(
19     @Value("\${star.esp32.tcp.enabled:true}")
20     private val enabled: Boolean,
21
22     @Value("\${star.esp32.tcp.host:192.168.4.2}")
23     private val host: String,
24
25     @Value("\${star.esp32.tcp.port:5000}")
26     private val port: Int,
27
28     @Value("\${star.esp32.tcp.timeout-ms:100}")
29     private val timeoutMs: Long,
30
31     @Value("\${star.esp32.tcp.retry-attempts:3}")
32     private val retryAttempts: Int,
33
34     private val objectMapper: ObjectMapper
35 ) {
36     private val logger = LoggerFactory.getLogger(javaClass)
37
38     data class TcpCommand(
39         val command: String,
40         val params: Map<String, Any> = emptyMap(),
41         val timestamp: Long = System.currentTimeMillis()
42     )
43
44     data class TcpResponse(
45         val success: Boolean,
46         val message: String,
47         val data: Map<String, Any>? = null

```

```

48     )
49
50     /**
51      * Send velocity command via TCP.
52      */
53     suspend fun sendVelocityCommand(left: Float, right: Float):
54         CommandResult {
55         val command = TcpCommand(
56             command = "velocity",
57             params = mapOf(
58                 "left" to left,
59                 "right" to right
60             )
61         )
62         return sendCommand(command)
63     }
64
65     /**
66      * Send emergency stop via TCP.
67      */
68     suspend fun emergencyStop(): CommandResult {
69         val command = TcpCommand(command = "emergency_stop")
70         return sendCommand(command)
71     }
72
73     /**
74      * Generic command sender with retry logic.
75      */
76     private suspend fun sendCommand(command: TcpCommand): CommandResult {
77         if (!enabled) {
78             return CommandResult.failure("TCP adapter disabled")
79         }
80
81         var lastException: Exception? = null
82
83         repeat(retryAttempts) { attempt ->
84             try {
85                 return withContext(Dispatchers.IO) {
86                     withTimeoutOrNull(timeoutMs) {
87                         executeCommand(command)
88                     } ?: CommandResult.failure("TCP timeout after
89                         ${timeoutMs}ms")
90                 }
91             } catch (e: Exception) {
92                 lastException = e
93                 logger.warn("TCP attempt ${attempt + 1}/${retryAttempts}
94                     failed: ${e.message}")
95             }
96         }
97
98         return CommandResult.failure(
99             "TCP failed after $retryAttempts attempts:
100                 ${lastException?.message}"
101         )

```

```

98     }
99
100    private fun executeCommand(command: TcpCommand): CommandResult {
101        Socket().use { socket ->
102            socket.connect(InetSocketAddress(host, port),
103                timeoutMs.toInt())
104            socket.soTimeout = timeoutMs.toInt()
105
106            val writer = PrintWriter(socket.getOutputStream(), true)
107            val reader =
108                BufferedReader(InputStreamReader(socket.getInputStream()))
109
110            // Send JSON command
111            val json = objectMapper.writeValueAsString(command)
112            writer.println(json)
113
114            // Read response
115            val responseLine = reader.readLine()
116            val response = objectMapper.readValue(responseLine,
117                TcpResponse::class.java)
118
119            return if (response.success) {
120                CommandResult.success(response.message)
121            } else {
122                CommandResult.failure(response.message)
123            }
124        }
125    }
126
127    /**
128     * Check TCP connectivity.
129     */
130    fun isConnected(): Boolean {
131        if (!enabled) return false
132
133        return try {
134            Socket().use { socket ->
135                socket.connect(InetSocketAddress(host, port),
136                    timeoutMs.toInt())
137                true
138            }
139        } catch (e: Exception) {
140            false
141        }
142    }
143 }

```

Listing 17: Esp32TcpAdapter.kt

10.6.4 GraphQL Controller

```

1 package com.star.gateway.graphql
2

```

```

3 import com.star.gateway.model.*
4 import com.star.gateway.service.TelemetryService
5 import com.star.gateway.service.MotorControlService
6 import kotlinx.coroutines.flow.Flow
7 import org.springframework.graphql.data.method.annotation.Argument
8 import org.springframework.graphql.data.method.annotation.MutationMapping
9 import org.springframework.graphql.data.method.annotation.QueryMapping
10 import
    org.springframework.graphql.data.method.annotation.SubscriptionMapping
11 import org.springframework.stereotype.Controller
12 import reactor.core.publisher.Flux
13
14 @Controller
15 class TelemetryController(
16     private val telemetryService: TelemetryService,
17     private val motorControlService: MotorControlService
18 ) {
19     // ===== QUERIES =====
20
21     @QueryMapping
22     suspend fun telemetry(): Telemetry {
23         return telemetryService.getCurrentTelemetry()
24     }
25
26     @QueryMapping
27     suspend fun systemStatus(): SystemStatus {
28         return telemetryService.getSystemStatus()
29     }
30
31     @QueryMapping
32     suspend fun telemetryHistory(
33         @Argument limit: Int?
34     ): List<TelemetrySnapshot> {
35         return telemetryService.getHistory(limit ?: 100)
36     }
37
38     // ===== MUTATIONS =====
39
40     @MutationMapping
41     suspend fun emergencyStop(): Boolean {
42         val result = motorControlService.emergencyStop()
43         return result.success
44     }
45
46     @MutationMapping
47     suspend fun setMode(@Argument mode: RobotMode): Boolean {
48         return telemetryService.setMode(mode)
49     }
50
51     // ===== SUBSCRIPTIONS =====
52
53     @SubscriptionMapping
54     fun telemetryStream(): Flux<Telemetry> {
55         return telemetryService.telemetryFlow()

```

```

56         .asFlux()
57     }
58
59     @SubscriptionMapping
60     fun eventLog(): Flux<Event> {
61         return telemetryService.eventFlow()
62             .asFlux()
63     }
64 }
65
66 // Extension function to convert Flow to Flux
67 private fun <T> Flow<T>.asFlux(): Flux<T> {
68     return kotlinox.coroutines.reactor.asFlux(this)
69 }

```

Listing 18: TelemetryController.kt

10.6.5 gRPC Service

```

1 package com.star.gateway.grpc
2
3 import com.star.gateway.proto.*
4 import com.star.gateway.service.MotorControlService
5 import io.grpc.stub.StreamObserver
6 import kotlinox.coroutines.flow.Flow
7 import kotlinox.coroutines.flow.flow
8 import kotlinox.coroutines.runBlocking
9 import net.devh.boot.grpc.server.service.GrpcService
10 import org.slf4j.LoggerFactory
11
12 @GrpcService
13 class MotorControlGrpcService(
14     private val motorControlService: MotorControlService
15 ) : MotorControlGrpcKt.MotorControlCoroutineImplBase() {
16
17     private val logger = LoggerFactory.getLogger(javaClass)
18
19     /**
20      * Handle velocity command via gRPC.
21      */
22     override suspend fun setVelocity(request: VelocityCommand):
23         CommandResponse {
24         val startTime = System.nanoTime()
25
26         logger.debug("gRPC SetVelocity: left=${request.leftVelocity},
27             right=${request.rightVelocity}")
28
29         val result = motorControlService.setVelocity(
30             left = request.leftVelocity,
31             right = request.rightVelocity
32         )
33
34         val latencyUs = (System.nanoTime() - startTime) / 1000

```



```
33
34     return CommandResponse.newBuilder()
35         .setSuccess(result.success)
36         .setMessage(result.message)
37         .setLatencyUs(latencyUs)
38         .build()
39 }
40
41 /**
42  * Handle emergency stop via gRPC.
43  */
44 override suspend fun emergencyStop(request: EmptyRequest):
45     CommandResponse {
46     logger.warn("gRPC EmergencyStop received")
47
48     val result = motorControlService.emergencyStop()
49
50     return CommandResponse.newBuilder()
51         .setSuccess(result.success)
52         .setMessage(result.message)
53         .build()
54 }
55
56 /**
57  * Stream encoder data to client.
58  */
59 override fun streamEncoderData(request: EmptyRequest):
60     Flow<EncoderData> = flow {
61     // Emit encoder data at 100Hz
62     while (true) {
63         val encoders = motorControlService.getEncoderData()
64
65         encoders.forEach { (motorId, data) ->
66             emit(
67                 EncoderData.newBuilder()
68                     .setMotorId(motorId)
69                     .setTicks(data.ticks)
70                     .setVelocity(data.velocity)
71                     .setTimestamp(System.currentTimeMillis())
72                     .build()
73             )
74         }
75
76         kotlinx.coroutines.delay(10) // 100Hz
77     }
78 }
```

Listing 19: MotorControlGrpcService.kt

11 PWA Frontend Implementation

11.1 Package Configuration

The PWA builds upon the existing UI-Demo codebase with additional dependencies:

```

1 {
2   "name": "star-ui",
3   "version": "1.0.0",
4   "type": "module",
5   "scripts": {
6     "dev": "vite",
7     "build": "vue-tsc && vite build",
8     "preview": "vite preview",
9     "type-check": "vue-tsc --noEmit",
10    "generate:proto": "grpc_tools_node_protoc
        --js_out=import_style=commonjs,binary:./src/grpc/generated
        --grpc-web_out=import_style=typescript,mode=grpcwebtext:./src/grpc/generated
        -I ../proto ../proto/star/*.proto"
11  },
12  "dependencies": {
13    // Vue 3 Core
14    "vue": "^3.5.13",
15    "vue-router": "^4.5.0",
16    "pinia": "^2.3.0",
17
18    // UI Libraries
19    "chart.js": "^4.4.7",
20    "vue-chartjs": "^5.3.2",
21    "@vueuse/core": "^12.0.0",
22    "lucide-vue-next": "^0.460.0",
23    "clsx": "^2.1.1",
24    "tailwind-merge": "^3.4.0",
25
26    // GraphQL (Vue Apollo)
27    "@vue/apollo-composable": "^4.3.0",
28    "@apollo/client": "^3.12.0",
29    "graphql": "^16.10.0",
30    "graphql-ws": "^5.16.0",
31    "apollo3-cache-persist": "^0.15.0",
32
33    // gRPC-Web
34    "@improbable-eng/grpc-web": "^0.15.0",
35    "google-protobuf": "^3.21.4",
36
37    // Offline storage
38    "idb": "^8.0.1",
39
40    // Service Worker
41    "workbox-core": "^7.3.0",
42    "workbox-precaching": "^7.3.0",
43    "workbox-routing": "^7.3.0",
44    "workbox-strategies": "^7.3.0"
45  },

```

```

46  "devDependencies": {
47    // TypeScript
48    "typescript": "~5.7.0",
49    "vue-tsc": "^2.2.0",
50    "@types/node": "^22.10.0",
51
52    // Vite + Vue
53    "@vitejs/plugin-vue": "^5.2.1",
54    "vite": "^6.0.0",
55
56    // Styling
57    "tailwindcss": "^4.1.17",
58    "autoprefixer": "^10.4.20",
59
60    // PWA plugin
61    "vite-plugin-pwa": "^0.21.1",
62
63    // Proto compilation
64    "grpc_tools_node_protoc_ts": "^5.3.3",
65    "grpc-tools": "^1.12.4"
66  }
67 }

```

Listing 20: package.json (Vue 3 + TypeScript)

11.2 Vite PWA Configuration

```

1  import { defineConfig } from 'vite'
2  import vue from '@vitejs/plugin-vue'
3  import { VitePWA } from 'vite-plugin-pwa'
4
5  export default defineConfig({
6    plugins: [
7      vue(),
8      VitePWA({
9        registerType: 'autoUpdate',
10       includeAssets: [
11         'favicon.ico',
12         'robots.txt',
13         'apple-touch-icon.png'
14       ],
15       manifest: {
16         name: 'STAR Robot Controller',
17         short_name: 'STAR',
18         description: 'Control interface for STAR LiDAR SLAM robot',
19         theme_color: '#500000', // Texas A&M maroon
20         background_color: '#1a1a1a',
21         display: 'standalone',
22         orientation: 'any',
23         start_url: '/',
24         scope: '/',
25         icons: [
26

```

```

27         src: 'pwa-192x192.png',
28         sizes: '192x192',
29         type: 'image/png'
30     },
31     {
32         src: 'pwa-512x512.png',
33         sizes: '512x512',
34         type: 'image/png',
35         purpose: 'any maskable'
36     }
37 ]
38 },
39 workbox: {
40     // Precache static assets
41     globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],
42
43     // Runtime caching strategies
44     runtimeCaching: [
45         {
46             // GraphQL API - Network First with fallback
47             urlPattern: /^https?:\/\/\/star-robot\.local(:8080)?\/graphql/,
48             handler: 'NetworkFirst',
49             options: {
50                 cacheName: 'graphql-cache',
51                 expiration: {
52                     maxEntries: 100,
53                     maxAgeSeconds: 60 * 60 // 1 hour
54                 },
55                 networkTimeoutSeconds: 3,
56                 cacheableResponse: {
57                     statuses: [0, 200]
58                 }
59             }
60         },
61         {
62             // Static assets from gateway
63             urlPattern: /^https?:\/\/\/star-robot\.local(:8080)?\/static/,
64             handler: 'CacheFirst',
65             options: {
66                 cacheName: 'static-assets',
67                 expiration: {
68                     maxEntries: 60,
69                     maxAgeSeconds: 60 * 60 * 24 * 30 // 30 days
70                 }
71             }
72         }
73     ]
74 },
75 devOptions: {
76     enabled: true // Enable PWA in dev mode for testing
77 }
78 })
79 ],
80 server: {

```

```

81   port: 3000,
82   proxy: {
83     '/graphql': {
84       target: 'http://localhost:8080',
85       ws: true // WebSocket proxy for subscriptions
86     },
87     '/grpc': {
88       target: 'http://localhost:8090'
89     }
90   }
91 }
92 })

```

Listing 21: vite.config.ts

11.3 Apollo Client with Offline Support (Vue 3)

```

1  import {
2    ApolloClient,
3    InMemoryCache,
4    HttpLink,
5    split,
6    ApolloLink,
7    type NormalizedCacheObject
8  } from '@apollo/client/core'
9  import { GraphQLWsLink } from '@apollo/client/link/subscriptions'
10 import { getMainDefinition } from '@apollo/client/utilities'
11 import { createClient } from 'graphql-ws'
12 import { persistCache, LocalStorageWrapper } from 'apollo3-cache-persist'
13 import { onError } from '@apollo/client/link/error'
14 import { provideApolloClient } from '@vue/apollo-composable'
15
16 // Gateway URL - defaults to mDNS hostname
17 const GATEWAY_URL = import.meta.env.VITE_GATEWAY_URL
18   || 'http://star-robot.local:8080';
19
20 // HTTP link for queries and mutations
21 const httpLink = new HttpLink({
22   uri: `${GATEWAY_URL}/graphql`,
23   credentials: 'include'
24 });
25
26 // WebSocket link for subscriptions
27 const wsLink = new GraphQLWsLink(createClient({
28   url: `ws://${GATEWAY_URL.replace(/^https?:\/\//, '')}/graphql-ws`,
29   connectionParams: () => ({
30     // Add auth tokens here if needed
31     timestamp: Date.now()
32   }),
33   retryAttempts: Infinity,
34   shouldRetry: () => true,
35   on: {
36     connected: () => console.log('WebSocket connected'),

```

```

37     closed: () => console.log('WebSocket closed'),
38     error: (err) => console.error('WebSocket error:', err)
39   }
40 }));
41
42 // Error handling link
43 const errorLink = onError(({ graphqlErrors, networkError, operation }) =>
44   {
45     if (graphqlErrors) {
46       graphqlErrors.forEach(({ message, locations, path }) => {
47         console.error(
48           '[GraphQL error]: Message: ${message}, ' +
49           'Location: ${locations}, Path: ${path}'
50         );
51       });
52     }
53     if (networkError) {
54       console.error('[Network error]: ${networkError}');
55       // Could dispatch to offline queue here
56     }
57   });
58
59 // Split link based on operation type
60 const splitLink = split(
61   ({ query }) => {
62     const definition = getMainDefinition(query);
63     return (
64       definition.kind === 'OperationDefinition' &&
65       definition.operation === 'subscription'
66     );
67   },
68   wsLink,
69   httpLink
70 );
71
72 // Create cache with custom type policies
73 const cache = new InMemoryCache({
74   typePolicies: {
75     Query: {
76       fields: {
77         telemetryHistory: {
78           // Merge incoming with existing, keep last 1000
79           merge(existing = [], incoming) {
80             return [...incoming].slice(-1000);
81           }
82         }
83       }
84     },
85     Telemetry: {
86       // Use timestamp as unique identifier
87       keyFields: ['timestamp']
88     },
89     Waypoint: {

```

```

90     keyFields: ['id']
91   }
92 }
93 });
94
95 // Initialize cache persistence
96 let apolloClient;
97
98 export async function initializeApollo() {
99   // Persist cache to localStorage
100   await persistCache({
101     cache,
102     storage: new LocalStorageWrapper(window.localStorage),
103     maxSize: 1048576 * 10, // 10MB
104     debug: import.meta.env.DEV
105   });
106
107   apolloClient = new ApolloClient({
108     link: ApolloLink.from([errorLink, splitLink]),
109     cache,
110     defaultOptions: {
111       watchQuery: {
112         fetchPolicy: 'cache-and-network',
113         errorPolicy: 'all'
114       },
115       query: {
116         fetchPolicy: 'cache-first',
117         errorPolicy: 'all'
118       },
119       mutate: {
120         errorPolicy: 'all'
121       }
122     }
123   });
124
125   return apolloClient;
126 }
127
128 export function getApolloClient() {
129   if (!apolloClient) {
130     throw new Error('Apollo Client not initialized. Call
131       initializeApollo() first.');
```

Listing 22: src/graphql/client.ts

11.4 GraphQL Operations

```

1 import { gql } from '@apollo/client';
2
3 // ===== QUERIES =====
```

```
4
5 export const TELEMETRY_QUERY = gql`
6   query GetTelemetry {
7     telemetry {
8       imu {
9         pitch
10        roll
11        yaw
12        velocity
13        accelerationX
14        accelerationY
15        accelerationZ
16      }
17      gps {
18        latitude
19        longitude
20        altitude
21      }
22      battery
23      wifiSignal
24      cpuUsage
25      temperature
26      motorLoad
27      timestamp
28    }
29  }
30 `;
31
32 export const SYSTEM_STATUS_QUERY = gql`
33   query GetSystemStatus {
34     systemStatus {
35       connectionStatus
36       mode
37       esp32Connected
38       lidarConnected
39       rosConnected
40     }
41   }
42 `;
43
44 export const NAVIGATION_STATE_QUERY = gql`
45   query GetNavigationState {
46     navigationState {
47       currentPosition {
48         x
49         y
50         heading
51       }
52       path {
53         x
54         y
55         heading
56       }
57       waypoints {
```



```

58         id
59         x
60         y
61         label
62     }
63     isNavigating
64 }
65 }
66 ';
67
68 export const TELEMETRY_HISTORY_QUERY = gql`
69     query GetTelemetryHistory($limit: Int) {
70         telemetryHistory(limit: $limit) {
71             imu {
72                 pitch
73                 roll
74                 yaw
75                 velocity
76             }
77             battery
78             timestamp
79         }
80     }
81 `;
82
83 // ===== MUTATIONS =====
84
85 export const EMERGENCY_STOP_MUTATION = gql`
86     mutation EmergencyStop {
87         emergencyStop
88     }
89 `;
90
91 export const SET_MODE_MUTATION = gql`
92     mutation SetMode($mode: RobotMode!) {
93         setMode(mode: $mode)
94     }
95 `;
96
97 export const SET_SETTINGS_MUTATION = gql`
98     mutation SetSettings($input: SettingsInput!) {
99         setSettings(input: $input) {
100             maxSpeed
101             safetyLockEnabled
102             autonomousEnabled
103             lidarEnabled
104         }
105     }
106 `;
107
108 export const ADD_WAYPOINT_MUTATION = gql`
109     mutation AddWaypoint($x: Float!, $y: Float!) {
110         addWaypoint(x: $x, y: $y) {
111             id

```

```
112     x
113     y
114     label
115   }
116 }
117 ‘;
118
119 // ===== SUBSCRIPTIONS =====
120
121 export const TELEMETRY_SUBSCRIPTION = gql‘
122   subscription TelemetryStream {
123     telemetryStream {
124       imu {
125         pitch
126         roll
127         yaw
128         velocity
129       }
130       battery
131       wifiSignal
132       cpuUsage
133       temperature
134       timestamp
135     }
136   }
137 ‘;
138
139 export const LIDAR_SCAN_SUBSCRIPTION = gql‘
140   subscription LidarScan {
141     lidarScan {
142       points {
143         angle
144         distance
145         intensity
146       }
147       timestamp
148     }
149   }
150 ‘;
151
152 export const EVENT_LOG_SUBSCRIPTION = gql‘
153   subscription EventLog {
154     eventLog {
155       timestamp
156       message
157       type
158     }
159   }
160 ‘;
```

Listing 23: src/graphql/operations.js

11.5 gRPC-Web Client

```

1 import { grpc } from '@improbable-eng/grpc-web';
2 import {
3   MotorControlClient
4 } from '../generated/motor_control_pb_service';
5 import {
6   VelocityCommand,
7   EmptyRequest,
8   MotorPowerCommand
9 } from '../generated/motor_control_pb';
10
11 // gRPC endpoint (through Envoy proxy)
12 const GRPC_URL = import.meta.env.VITE_GRPC_URL
13   || 'http://star-robot.local:8090';
14
15 // Create gRPC-Web client
16 const client = new MotorControlClient(GRPC_URL);
17
18 /**
19  * Send velocity command via gRPC.
20  * @param {number} leftVelocity - Left wheel velocity in m/s
21  * @param {number} rightVelocity - Right wheel velocity in m/s
22  * @returns {Promise<{success: boolean, message: string, latencyUs:
23    number}>}
24  */
25 export function setVelocity(leftVelocity, rightVelocity) {
26   return new Promise((resolve, reject) => {
27     const request = new VelocityCommand();
28     request.setLeftVelocity(leftVelocity);
29     request.setRightVelocity(rightVelocity);
30     request.setTimestamp(Date.now());
31
32     client.setVelocity(request, {}, (err, response) => {
33       if (err) {
34         console.error('gRPC setVelocity error:', err);
35         reject(err);
36       } else {
37         resolve({
38           success: response.getSuccess(),
39           message: response.getMessage(),
40           latencyUs: response.getLatencyUs()
41         });
42       }
43     });
44   });
45
46   /**
47    * Send emergency stop command via gRPC.
48    * @returns {Promise<{success: boolean, message: string}>}
49    */
50   export function emergencyStop() {
51     return new Promise((resolve, reject) => {

```

```

52     const request = new EmptyRequest();
53
54     client.emergencyStop(request, {}, (err, response) => {
55         if (err) {
56             console.error('gRPC emergencyStop error:', err);
57             reject(err);
58         } else {
59             resolve({
60                 success: response.getSuccess(),
61                 message: response.getMessage()
62             });
63         }
64     });
65 });
66 }
67
68 /**
69  * Set individual motor power via gRPC.
70  * @param {number} motorId - Motor index (0-3)
71  * @param {number} power - Power level (-1.0 to 1.0)
72  * @returns {Promise<{success: boolean, message: string}>}
73  */
74 export function setMotorPower(motorId, power) {
75     return new Promise((resolve, reject) => {
76         const request = new MotorPowerCommand();
77         request.setMotorId(motorId);
78         request.setPower(power);
79
80         client.setMotorPower(request, {}, (err, response) => {
81             if (err) {
82                 reject(err);
83             } else {
84                 resolve({
85                     success: response.getSuccess(),
86                     message: response.getMessage()
87                 });
88             }
89         });
90     });
91 }
92
93 /**
94  * Stream encoder data from all motors.
95  * @param {Function} onData - Callback for each encoder data packet
96  * @param {Function} onError - Callback for errors
97  * @param {Function} onEnd - Callback when stream ends
98  * @returns {Function} Cancel function to stop the stream
99  */
100 export function streamEncoderData(onData, onError, onEnd) {
101     const request = new EmptyRequest();
102     const stream = client.streamEncoderData(request);
103
104     stream.on('data', (response) => {
105         onData({

```

```

106     motorId: response.getMotorId(),
107     ticks: response.getTicks(),
108     velocity: response.getVelocity(),
109     timestamp: response.getTimestamp()
110   });
111 });
112
113 stream.on('error', (err) => {
114   console.error('Encoder stream error:', err);
115   onError(err);
116 });
117
118 stream.on('end', () => {
119   console.log('Encoder stream ended');
120   onEnd();
121 });
122
123 // Return cancel function
124 return () => {
125   stream.cancel();
126 };
127 }

```

Listing 24: src/grpc/client.js

11.6 Robot Connection Composable (Vue 3 - Replaces useMockData)

```

1 import { ref, computed, onMounted, onUnmounted, watch } from 'vue'
2 import { useQuery, useSubscription, useMutation } from
   '@vue/apollo-composable'
3 import {
4   TELEMETRY_QUERY,
5   SYSTEM_STATUS_QUERY,
6   NAVIGATION_STATE_QUERY,
7   TELEMETRY_SUBSCRIPTION,
8   LIDAR_SCAN_SUBSCRIPTION,
9   EVENT_LOG_SUBSCRIPTION,
10  EMERGENCY_STOP_MUTATION,
11  SET_MODE_MUTATION
12 } from '@graphql/operations'
13 import * as grpcClient from '@grpc/client'
14 import {
15   saveTelemetryToIndexedDB,
16   getTelemetryFromIndexedDB,
17   saveNavigationState,
18   getNavigationState as getStoredNavState
19 } from '@storage/indexedDB'
20 import type { Telemetry, SystemStatus, NavigationState, EncoderData }
   from '@types'
21
22 /**
23  * Main composable for robot connection and control.
24  * Replaces useMockData with real API integration.

```

```

25  */
26  export function useRobotConnection() {
27    // Connection state (Vue 3 refs)
28    const isOnline = ref(navigator.onLine)
29    const lastSyncTime = ref<Date | null>(null)
30    const connectionQuality = ref<'good' | 'fair' | 'poor'>('good')
31
32    // Encoder stream reference
33    let encoderStreamCancel: (() => void) | null = null
34    const encoderData = ref<Record<number, EncoderData>>({})
35
36    // ===== GraphQL Queries (Vue Apollo) =====
37
38    const {
39      result: telemetryResult,
40      loading: telemetryLoading,
41      error: telemetryError
42    } = useQuery<TELEMETRY_QUERY>(), () => ({
43      pollInterval: isOnline.value ? 0 : 5000,
44      fetchPolicy: 'cache-and-network'
45    })
46
47    const { result: statusResult } = useQuery<SYSTEM_STATUS_QUERY>(), {
48      pollInterval: 1000
49    }
50
51    const { result: navResult } = useQuery<NAVIGATION_STATE_QUERY>(), {
52      pollInterval: 500
53    }
54
55    // ===== GraphQL Subscriptions =====
56
57    const { result: liveTelemetry } = useSubscription<
58      TELEMETRY_SUBSCRIPTION,
59      null,
60      () => ({ enabled: isOnline.value })
61    >()
62
63    const { result: lidarResult } = useSubscription<
64      LIDAR_SCAN_SUBSCRIPTION,
65      null,
66      () => ({ enabled: isOnline.value })
67    >()
68
69    const { result: eventResult } = useSubscription<
70      EVENT_LOG_SUBSCRIPTION,
71      null,
72      () => ({ enabled: isOnline.value })
73    >()
74
75    // Watch for telemetry updates to persist to IndexedDB
76    watch(liveTelemetry, (newData) => {
77      if (newData?.telemetryStream) {
78        saveTelemetryToIndexedDB(newData.telemetryStream)

```

```

79     lastSyncTime.value = new Date()
80   }
81 })
82
83 // ===== GraphQL Mutations =====
84
85 const { mutate: emergencyStopMutation } =
86   useMutation(EMERGENCY_STOP_MUTATION)
87 const { mutate: setModeMutation } = useMutation(SET_MODE_MUTATION)
88
89 // ===== Network Status (Vue lifecycle) =====
90
91 onMounted(() => {
92   const handleOnline = () => {
93     isOnline.value = true
94     console.log('Network: Online')
95   }
96   const handleOffline = () => {
97     isOnline.value = false
98     console.log('Network: Offline')
99   }
100   window.addEventListener('online', handleOnline)
101   window.addEventListener('offline', handleOffline)
102 })
103
104 onUnmounted(() => {
105   window.removeEventListener('online', () => {})
106   window.removeEventListener('offline', () => {})
107 })
108
109 // ===== Encoder Stream (gRPC) =====
110
111 watch(isOnline, (online) => {
112   if (online && !encoderStreamCancel) {
113     encoderStreamCancel = grpcClient.streamEncoderData(
114       (data: EncoderData) => {
115         encoderData.value = {
116           ...encoderData.value,
117           [data.motorId]: data
118         }
119       },
120       (err: Error) => {
121         console.error('Encoder stream error:', err)
122         encoderStreamCancel = null
123       },
124       () => {
125         encoderStreamCancel = null
126       }
127     )
128   }
129 }, { immediate: true })
130
131 onUnmounted(() => {

```

```

132     if (encoderStreamCancel) {
133         encoderStreamCancel()
134         encoderStreamCancel = null
135     }
136 })
137
138 // ===== Motor Control (gRPC) =====
139
140 async function sendMotorCommand(left: number, right: number) {
141     if (!isOnline.value) {
142         throw new Error('Cannot send motor commands while offline')
143     }
144     return grpcClient.setVelocity(left, right)
145 }
146
147 async function emergencyStop() {
148     // Try both gRPC and GraphQL for redundancy
149     const results = await Promise.allSettled([
150         grpcClient.emergencyStop(),
151         emergencyStopMutation()
152     ])
153
154     const anySuccess = results.some(
155         r => r.status === 'fulfilled' && (r.value as any)?.success !== false
156     )
157
158     return { success: anySuccess }
159 }
160
161 async function setMode(mode: string) {
162     const result = await setModeMutation({ mode })
163     return result?.data?.setMode
164 }
165
166 // ===== Computed Properties =====
167
168 const telemetry = computed(() =>
169     liveTelemetry.value?.telemetryStream
170     || telemetryResult.value?.telemetry
171 )
172
173 const lidarScan = computed(() => lidarResult.value?.lidarScan)
174 const systemStatus = computed(() => statusResult.value?.systemStatus)
175 const navigationState = computed(() => navResult.value?.navigationState)
176 const latestEvent = computed(() => eventResult.value?.eventLog)
177
178 // ===== Return Values =====
179
180 return {
181     // Connection state
182     isOnline,
183     lastSyncTime,
184     connectionQuality,
185

```



```
186 // Telemetry
187 telemetry,
188 telemetryLoading,
189 telemetryError,
190
191 // LiDAR
192 lidarScan,
193
194 // System status
195 systemStatus,
196
197 // Navigation
198 navigationState,
199
200 // Events
201 latestEvent,
202
203 // Encoder data
204 encoderData,
205
206 // Motor control
207 sendMotorCommand,
208 emergencyStop,
209 setMode
210 }
211 }
```

Listing 25: src/composables/useRobotConnection.ts

11.7 IndexedDB Offline Storage

```
1 import { openDB } from 'idb';
2
3 const DB_NAME = 'star-robot-db';
4 const DB_VERSION = 1;
5
6 let dbPromise;
7
8 /**
9  * Get or create the IndexedDB database.
10  */
11 async function getDB() {
12   if (!dbPromise) {
13     dbPromise = openDB(DB_NAME, DB_VERSION, {
14       upgrade(db, oldVersion, newVersion, transaction) {
15         // Telemetry history store
16         if (!db.objectStoreNames.contains('telemetry')) {
17           const store = db.createObjectStore('telemetry', {
18             keyPath: 'id',
19             autoIncrement: true
20           });
21           store.createIndex('timestamp', 'timestamp');
22           store.createIndex('savedAt', 'savedAt');
```

```

23     }
24
25     // Settings store
26     if (!db.objectStoreNames.contains('settings')) {
27         db.createObjectStore('settings', { keyPath: 'key' });
28     }
29
30     // Navigation state store
31     if (!db.objectStoreNames.contains('navigation')) {
32         db.createObjectStore('navigation', { keyPath: 'key' });
33     }
34
35     // Event log store
36     if (!db.objectStoreNames.contains('events')) {
37         const store = db.createObjectStore('events', {
38             keyPath: 'id',
39             autoIncrement: true
40         });
41         store.createIndex('timestamp', 'timestamp');
42     }
43 }
44 });
45 }
46 return dbPromise;
47 }
48
49 /**
50  * Save telemetry snapshot to IndexedDB.
51  * Automatically prunes to keep only last 10,000 entries.
52  */
53 export async function saveTelemetryToIndexedDB(telemetry) {
54     const db = await getDB();
55
56     await db.add('telemetry', {
57         ...telemetry,
58         savedAt: Date.now()
59     });
60
61     // Prune old entries
62     const count = await db.count('telemetry');
63     if (count > 10000) {
64         const tx = db.transaction('telemetry', 'readwrite');
65         const store = tx.objectStore('telemetry');
66         const index = store.index('savedAt');
67
68         let cursor = await index.openCursor();
69         const toDelete = count - 10000;
70         let deleted = 0;
71
72         while (cursor && deleted < toDelete) {
73             await cursor.delete();
74             cursor = await cursor.continue();
75             deleted++;
76         }

```

```

77     await tx.done;
78   }
79 }
80
81
82 /**
83  * Retrieve telemetry history from IndexedDB.
84  * @param {number} limit - Maximum entries to return
85  * @returns {Promise<Array>} Telemetry entries, newest first
86  */
87 export async function getTelemetryFromIndexedDB(limit = 100) {
88   const db = await getDB();
89   const tx = db.transaction('telemetry', 'readonly');
90   const store = tx.objectStore('telemetry');
91   const index = store.index('timestamp');
92
93   const entries = [];
94   let cursor = await index.openCursor(null, 'prev');
95
96   while (cursor && entries.length < limit) {
97     entries.push(cursor.value);
98     cursor = await cursor.continue();
99   }
100
101   return entries;
102 }
103
104 /**
105  * Save user settings to IndexedDB.
106  */
107 export async function saveSettings(settings) {
108   const db = await getDB();
109   await db.put('settings', {
110     key: 'userSettings',
111     ...settings,
112     updatedAt: Date.now()
113   });
114 }
115
116 /**
117  * Retrieve user settings from IndexedDB.
118  */
119 export async function getSettings() {
120   const db = await getDB();
121   const result = await db.get('settings', 'userSettings');
122   return result || null;
123 }
124
125 /**
126  * Save navigation state to IndexedDB.
127  */
128 export async function saveNavigationState(state) {
129   const db = await getDB();
130   await db.put('navigation', {

```

```
131     key: 'currentState',
132     ...state,
133     savedAt: Date.now()
134   });
135 }
136
137 /**
138  * Retrieve navigation state from IndexedDB.
139  */
140 export async function getNavigationState() {
141   const db = await getDB();
142   const result = await db.get('navigation', 'currentState');
143   return result || null;
144 }
145
146 /**
147  * Save event to IndexedDB.
148  */
149 export async function saveEvent(event) {
150   const db = await getDB();
151   await db.add('events', {
152     ...event,
153     savedAt: Date.now()
154   });
155
156   // Keep only last 500 events
157   const count = await db.count('events');
158   if (count > 500) {
159     const tx = db.transaction('events', 'readwrite');
160     const store = tx.objectStore('events');
161     const index = store.index('timestamp');
162
163     let cursor = await index.openCursor();
164     const toDelete = count - 500;
165     let deleted = 0;
166
167     while (cursor && deleted < toDelete) {
168       await cursor.delete();
169       cursor = await cursor.continue();
170       deleted++;
171     }
172   }
173 }
174
175 /**
176  * Get recent events from IndexedDB.
177  */
178 export async function getEvents(limit = 50) {
179   const db = await getDB();
180   const tx = db.transaction('events', 'readonly');
181   const store = tx.objectStore('events');
182   const index = store.index('timestamp');
183
184   const entries = [];
```

```
185   let cursor = await index.openCursor(null, 'prev');
186
187   while (cursor && entries.length < limit) {
188     entries.push(cursor.value);
189     cursor = await cursor.continue();
190   }
191
192   return entries;
193 }
194
195 /**
196  * Clear all data from IndexedDB.
197  */
198 export async function clearAllData() {
199   const db = await getDB();
200
201   const tx = db.transaction(
202     ['telemetry', 'settings', 'navigation', 'events'],
203     'readwrite'
204   );
205
206   await Promise.all([
207     tx.objectStore('telemetry').clear(),
208     tx.objectStore('settings').clear(),
209     tx.objectStore('navigation').clear(),
210     tx.objectStore('events').clear()
211   ]);
212
213   await tx.done;
214 }
```

Listing 26: src/storage/indexedDB.js

12 ESP32 Firmware Updates

The existing STAR firmware requires additions for TCP communication as a fallback to SPI.

12.1 TCP Server Module

```

1  /**
2   * @file star_tcp_server.h
3   * @brief TCP server for command reception from RPi5
4   *
5   * Provides fallback communication when SPI is unavailable.
6   */
7
8  #ifndef STAR_TCP_SERVER_H
9  #define STAR_TCP_SERVER_H
10
11 #include "esp_err.h"
12
13 #ifdef __cplusplus
14 extern "C" {
15 #endif
16
17 #define TCP_SERVER_PORT 5000
18 #define TCP_MAX_CLIENTS 2
19 #define TCP_RX_BUFFER_SIZE 256
20 #define TCP_TX_BUFFER_SIZE 256
21
22 /**
23  * @brief Motor command callback type
24  */
25 typedef struct {
26     float left_velocity;
27     float right_velocity;
28     bool emergency_stop;
29     uint64_t timestamp;
30 } tcp_motor_command_t;
31
32 typedef void (*tcp_command_callback_t)(const tcp_motor_command_t*
33     command);
34
35 /**
36  * @brief TCP server configuration
37  */
38 typedef struct {
39     uint16_t port;
40     tcp_command_callback_t on_command;
41     void* user_context;
42 } tcp_server_config_t;
43
44 /**
45  * @brief TCP server handle
46  */
47 typedef struct tcp_server_handle* tcp_server_handle_t;

```

```

47
48 /**
49  * @brief Initialize and start TCP server
50  *
51  * @param config Server configuration
52  * @param out_handle Output handle
53  * @return esp_err_t ESP_OK on success
54  */
55 esp_err_t star_tcp_server_init(
56     const tcp_server_config_t* config,
57     tcp_server_handle_t* out_handle
58 );
59
60 /**
61  * @brief Stop and cleanup TCP server
62  */
63 esp_err_t star_tcp_server_deinit(tcp_server_handle_t handle);
64
65 /**
66  * @brief Get number of connected clients
67  */
68 int star_tcp_server_get_client_count(tcp_server_handle_t handle);
69
70 #ifdef __cplusplus
71 }
72 #endif
73
74 #endif // STAR_TCP_SERVER_H

```

Listing 27: lib/star_module_tcp/include/star_tcp_server.h

```

1 /**
2  * @file star_tcp_server.c
3  * @brief TCP server implementation
4  */
5
6 #include "star_tcp_server.h"
7 #include "esp_log.h"
8 #include "esp_netif.h"
9 #include "lwip/sockets.h"
10 #include "cJSON.h"
11 #include "freertos/FreeRTOS.h"
12 #include "freertos/task.h"
13
14 static const char* TAG = "tcp_server";
15
16 struct tcp_server_handle {
17     int listen_socket;
18     tcp_command_callback_t callback;
19     void* user_context;
20     TaskHandle_t server_task;
21     bool running;
22     int client_count;
23 };

```

```

24
25 /**
26  * @brief Parse JSON command from client
27  */
28 static esp_err_t parse_json_command(
29     const char* json_str,
30     tcp_motor_command_t* out_cmd
31 ) {
32     cJSON* root = cJSON_Parse(json_str);
33     if (root == NULL) {
34         ESP_LOGE(TAG, "Failed to parse JSON");
35         return ESP_ERR_INVALID_ARG;
36     }
37
38     memset(out_cmd, 0, sizeof(tcp_motor_command_t));
39
40     cJSON* cmd = cJSON_GetObjectItem(root, "command");
41     if (cmd && cJSON_IsString(cmd)) {
42         if (strcmp(cmd->valuestring, "velocity") == 0) {
43             cJSON* params = cJSON_GetObjectItem(root, "params");
44             if (params) {
45                 cJSON* left = cJSON_GetObjectItem(params, "left");
46                 cJSON* right = cJSON_GetObjectItem(params, "right");
47
48                 if (left && cJSON_IsNumber(left)) {
49                     out_cmd->left_velocity = (float)left->valuedouble;
50                 }
51                 if (right && cJSON_IsNumber(right)) {
52                     out_cmd->right_velocity = (float)right->valuedouble;
53                 }
54             }
55         } else if (strcmp(cmd->valuestring, "emergency_stop") == 0) {
56             out_cmd->emergency_stop = true;
57         }
58     }
59
60     cJSON* ts = cJSON_GetObjectItem(root, "timestamp");
61     if (ts && cJSON_IsNumber(ts)) {
62         out_cmd->timestamp = (uint64_t)ts->valuedouble;
63     }
64
65     cJSON_Delete(root);
66     return ESP_OK;
67 }
68
69 /**
70  * @brief Build JSON response
71  */
72 static void build_json_response(
73     char* buffer,
74     size_t size,
75     bool success,
76     const char* message
77 ) {

```



```

78     snprintf(buffer, size,
79         "{ \"success\":%s, \"message\": \"%s\" }",
80         success ? "true" : "false",
81         message
82     );
83 }
84
85 /**
86  * @brief Handle single client connection
87  */
88 static void handle_client(
89     tcp_server_handle_t handle,
90     int client_socket
91 ) {
92     char rx_buffer[TCP_RX_BUFFER_SIZE];
93     char tx_buffer[TCP_TX_BUFFER_SIZE];
94
95     int len = recv(client_socket, rx_buffer, sizeof(rx_buffer) - 1, 0);
96
97     if (len > 0) {
98         rx_buffer[len] = '\0';
99         ESP_LOGD(TAG, "Received: %s", rx_buffer);
100
101         tcp_motor_command_t cmd;
102         esp_err_t err = parse_json_command(rx_buffer, &cmd);
103
104         if (err == ESP_OK && handle->callback) {
105             handle->callback(&cmd);
106             build_json_response(tx_buffer, sizeof(tx_buffer), true, "OK");
107         } else {
108             build_json_response(tx_buffer, sizeof(tx_buffer), false,
109                 "Parse error");
110         }
111
112         send(client_socket, tx_buffer, strlen(tx_buffer), 0);
113     }
114 }
115
116 /**
117  * @brief TCP server task
118  */
119 static void tcp_server_task(void* pvParameters) {
120     tcp_server_handle_t handle = (tcp_server_handle_t)pvParameters;
121
122     struct sockaddr_in server_addr = {
123         .sin_family = AF_INET,
124         .sin_port = htons(TCP_SERVER_PORT),
125         .sin_addr.s_addr = htonl(INADDR_ANY)
126     };
127
128     handle->listen_socket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
129     if (handle->listen_socket < 0) {
130         ESP_LOGE(TAG, "Failed to create socket");
131         vTaskDelete(NULL);
132     }

```

```

131     return;
132 }
133
134 int opt = 1;
135 setsockopt(handle->listen_socket, SOL_SOCKET, SO_REUSEADDR, &opt,
136           sizeof(opt));
137
138 if (bind(handle->listen_socket, (struct sockaddr*)&server_addr,
139       sizeof(server_addr)) < 0) {
140     ESP_LOGE(TAG, "Failed to bind socket");
141     close(handle->listen_socket);
142     vTaskDelete(NULL);
143     return;
144 }
145
146 if (listen(handle->listen_socket, TCP_MAX_CLIENTS) < 0) {
147     ESP_LOGE(TAG, "Failed to listen");
148     close(handle->listen_socket);
149     vTaskDelete(NULL);
150     return;
151 }
152
153 ESP_LOGI(TAG, "TCP server listening on port %d", TCP_SERVER_PORT);
154
155 while (handle->running) {
156     struct sockaddr_in client_addr;
157     socklen_t addr_len = sizeof(client_addr);
158
159     int client_socket = accept(
160         handle->listen_socket,
161         (struct sockaddr*)&client_addr,
162         &addr_len
163     );
164
165     if (client_socket >= 0) {
166         handle->client_count++;
167         ESP_LOGI(TAG, "Client connected from %s",
168             inet_ntoa(client_addr.sin_addr));
169
170         handle_client(handle, client_socket);
171
172         close(client_socket);
173         handle->client_count--;
174     }
175 }
176
177 close(handle->listen_socket);
178 vTaskDelete(NULL);
179 }
180
181 esp_err_t star_tcp_server_init(
182     const tcp_server_config_t* config,
183     tcp_server_handle_t* out_handle
184 ) {

```

```

184     if (config == NULL || out_handle == NULL) {
185         return ESP_ERR_INVALID_ARG;
186     }
187
188     tcp_server_handle_t handle = calloc(1, sizeof(struct
189         tcp_server_handle));
189     if (handle == NULL) {
190         return ESP_ERR_NO_MEM;
191     }
192
193     handle->callback = config->on_command;
194     handle->user_context = config->user_context;
195     handle->running = true;
196     handle->client_count = 0;
197
198     BaseType_t ret = xTaskCreate(
199         tcp_server_task,
200         "tcp_server",
201         4096,
202         handle,
203         5,
204         &handle->server_task
205     );
206
207     if (ret != pdPASS) {
208         free(handle);
209         return ESP_FAIL;
210     }
211
212     *out_handle = handle;
213     return ESP_OK;
214 }
215
216 esp_err_t star_tcp_server_deinit(tcp_server_handle_t handle) {
217     if (handle == NULL) {
218         return ESP_ERR_INVALID_ARG;
219     }
220
221     handle->running = false;
222
223     // Wake up the accept() call
224     if (handle->listen_socket >= 0) {
225         shutdown(handle->listen_socket, SHUT_RDWR);
226     }
227
228     // Wait for task to finish
229     vTaskDelay(pdMS_TO_TICKS(100));
230
231     free(handle);
232     return ESP_OK;
233 }
234
235 int star_tcp_server_get_client_count(tcp_server_handle_t handle) {
236     return handle ? handle->client_count : 0;

```

237 }

Listing 28: lib/star_module_tcp/src/star_tcp_server.c

12.2 Main Application Integration

Add TCP server to the main application:

```

1 // Add to includes
2 #include "star_tcp_server.h"
3
4 // Add global handle
5 static tcp_server_handle_t g_tcp_server = NULL;
6
7 // Add command callback
8 static void on_tcp_command(const tcp_motor_command_t* cmd) {
9     if (cmd->emergency_stop) {
10         // Execute emergency stop
11         motor_control_emergency_stop();
12     } else {
13         // Set motor velocities
14         motor_control_set_velocity(cmd->left_velocity,
15                                   cmd->right_velocity);
16     }
17 }
18
19 // Add to app_main() after WiFi is connected
20 void app_main(void) {
21     // ... existing initialization ...
22
23     // Initialize TCP server for RPi5 communication
24     tcp_server_config_t tcp_config = {
25         .port = TCP_SERVER_PORT,
26         .on_command = on_tcp_command,
27         .user_context = NULL
28     };
29
30     esp_err_t err = star_tcp_server_init(&tcp_config, &g_tcp_server);
31     if (err == ESP_OK) {
32         ESP_LOGI(TAG, "TCP server started on port %d", TCP_SERVER_PORT);
33     } else {
34         ESP_LOGE(TAG, "Failed to start TCP server");
35     }
36
37     // ... rest of initialization ...
38 }
```

Listing 29: Diff for ESP32-Firmware/src/main.c

13 Process Flow Diagrams

This section illustrates the data flow through the STAR robot system for key operations.

13.1 Motor Command Flow

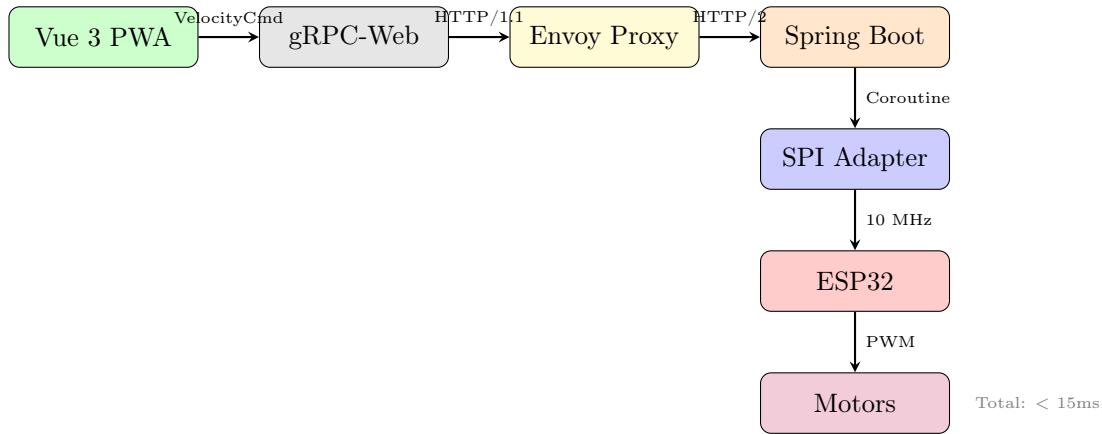


Figure 9: Motor Command Data Flow

13.2 SLAM Data Flow

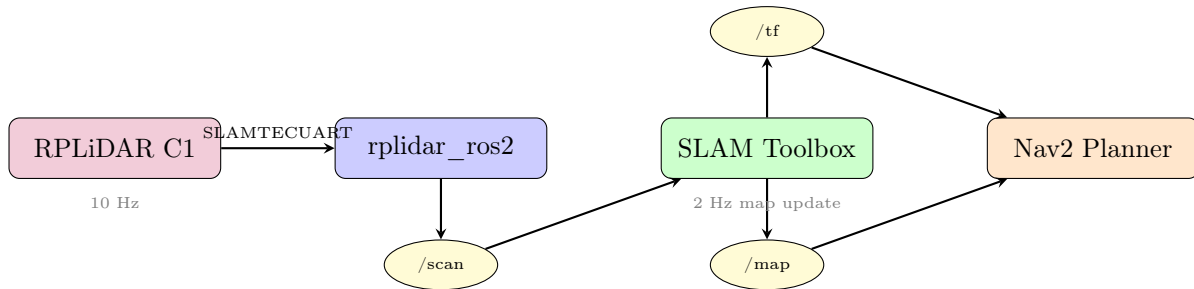


Figure 10: SLAM Data Flow (ROS 2 Topics)

13.3 Navigation Pipeline

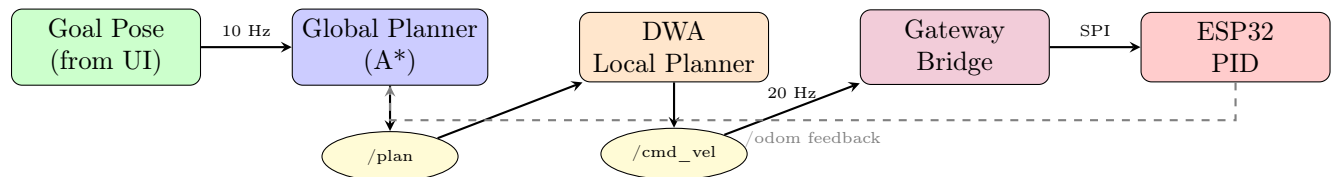


Figure 11: Navigation Pipeline Data Flow

13.4 Telemetry Aggregation Flow

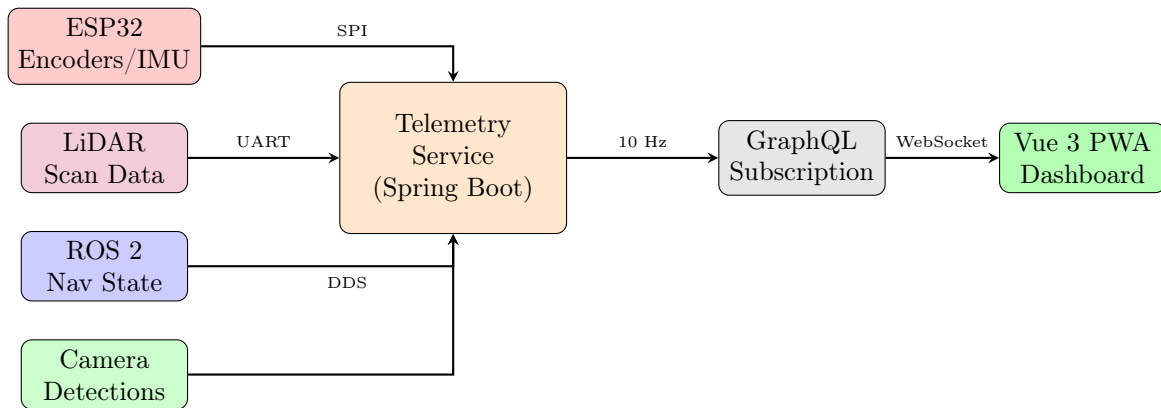


Figure 12: Telemetry Aggregation and Publishing

14 Deployment Configuration

14.1 Systemd Service

```

1 [Unit]
2 Description=STAR Robot Gateway
3 Documentation=https://github.com/locked-inc/star
4 After=network-online.target
5 Wants=network-online.target avahi-daemon.service
6
7 [Service]
8 Type=simple
9 User=star
10 Group=star
11 WorkingDirectory=/opt/star-gateway
12 ExecStart=/usr/bin/java \
13     -Xmx256m \
14     -Xms128m \
15     -XX:+UseG1GC \
16     -XX:MaxGCPauseMillis=50 \
17     -Djava.security.egd=file:/dev/./urandom \
18     -jar star-gateway.jar
19
20 # Restart policy
21 Restart=always
22 RestartSec=10
23 StartLimitInterval=300
24 StartLimitBurst=5
25
26 # Logging
27 StandardOutput=journal
28 StandardError=journal
29 SyslogIdentifier=star-gateway
30
31 # Environment
32 Environment="SPRING_PROFILES_ACTIVE=production"
33 Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"
34
35 # Security hardening
36 NoNewPrivileges=true
37 ProtectSystem=strict
38 ProtectHome=true
39 PrivateTmp=true
40 ReadWritePaths=/var/log/star-gateway /var/lib/star-gateway
41
42 # Resource limits
43 MemoryMax=512M
44 CPUQuota=80%
45
46 [Install]
47 WantedBy=multi-user.target

```

Listing 30: star-gateway/deployment/star-gateway.service

14.2 Avahi mDNS Service

```

1 <?xml version="1.0" standalone='no'?>
2 <!DOCTYPE service-group SYSTEM "avahi-service.dtd">
3 <service-group>
4   <name replace-wildcards="yes">STAR Robot Gateway on %h</name>
5
6   <!-- GraphQL HTTP endpoint -->
7   <service>
8     <type>_http._tcp</type>
9     <port>8080</port>
10    <txt-record>path=/graphql</txt-record>
11    <txt-record>api=graphql</txt-record>
12  </service>
13
14  <!-- GraphQL WebSocket endpoint -->
15  <service>
16    <type>_ws._tcp</type>
17    <port>8080</port>
18    <txt-record>path=/graphql-ws</txt-record>
19  </service>
20
21  <!-- gRPC endpoint (via Envoy) -->
22  <service>
23    <type>_grpc._tcp</type>
24    <port>8090</port>
25    <txt-record>api=grpc-web</txt-record>
26  </service>
27
28  <!-- Health check endpoint -->
29  <service>
30    <type>_http._tcp</type>
31    <port>8080</port>
32    <txt-record>path=/actuator/health</txt-record>
33    <txt-record>api=health</txt-record>
34  </service>
35 </service-group>

```

Listing 31: star-gateway/deployment/avahi/star-robot.service

14.3 Envoy Proxy Configuration

```

1 admin:
2   address:
3     socket_address:
4       address: 127.0.0.1
5       port_value: 9901
6
7 static_resources:
8   listeners:
9     - name: grpc_web_listener
10       address:
11         socket_address:

```



```

12     address: 0.0.0.0
13     port_value: 8090
14   filter_chains:
15     - filters:
16       - name: envoy.filters.network.http_connection_manager
17         typed_config:
18           "@type":
19             type.googleapis.com/envoy.extensions.filters.network.http_connection
20             codec_type: AUTO
21             stat_prefix: grpc_web
22
23         route_config:
24           name: local_route
25           virtual_hosts:
26             - name: grpc_services
27               domains: ["*"]
28               routes:
29                 - match:
30                     prefix: "/"
31                     route:
32                       cluster: grpc_backend
33                       timeout: 60s
34
35               # CORS configuration
36               cors:
37                 allow_origin_string_match:
38                   - prefix: "*"
39                 allow_methods: "GET, POST, OPTIONS"
40                 allow_headers: >
41                   content-type,
42                   x-grpc-web,
43                   x-user-agent,
44                   grpc-timeout
45                 max_age: "1728000"
46                 expose_headers: >
47                   grpc-status,
48                   grpc-message
49
50             http_filters:
51               # gRPC-Web to gRPC translation
52               - name: envoy.filters.http.grpc_web
53                 typed_config:
54                   "@type":
55                     type.googleapis.com/envoy.extensions.filters.http.grpc_web.v3
56
57               # CORS handling
58               - name: envoy.filters.http.cors
59                 typed_config:
60                   "@type":
61                     type.googleapis.com/envoy.extensions.filters.http.cors.v3.Cor
62
63               # Router
64               - name: envoy.filters.http.router
65                 typed_config:

```

```
63         "@type":
64             type.googleapis.com/envoy.extensions.filters.http.router.v3.R
65     clusters:
66     - name: grpc_backend
67       connect_timeout: 0.5s
68       type: LOGICAL_DNS
69       dns_lookup_family: V4_ONLY
70       lb_policy: ROUND_ROBIN
71       typed_extension_protocol_options:
72         envoy.extensions.upstreams.http.v3.HttpProtocolOptions:
73           "@type":
74             type.googleapis.com/envoy.extensions.upstreams.http.v3.HttpProtocolOption
75           explicit_http_config:
76             http2_protocol_options: {}
77       load_assignment:
78         cluster_name: grpc_backend
79         endpoints:
80         - lb_endpoints:
81           - endpoint:
82             address:
83               socket_address:
84                 address: 127.0.0.1
85                 port_value: 9090
```

Listing 32: /opt/envoy/envoy.yaml

15 Implementation Checklist

15.1 Phase 1: Infrastructure Setup

- ☐ Create `star-gateway/` Spring Boot project structure
- ☐ Create `proto/` shared protobuf definitions
- ☐ Set up Gradle build with gRPC + GraphQL plugins
- ☐ Add Envoy to Buildroot configuration
- ☐ Configure development environment (IDE, linting, testing)

15.2 Phase 2: Backend Core

- ☐ Implement `Esp32SpiAdapter` (Pi4J SPI communication)
- ☐ Implement `Esp32TcpAdapter` (TCP fallback)
- ☐ Implement `RplidarAdapter` (SLAMTEC protocol via `rplidar_ros2`)
- ☐ Implement domain services:
 - ☐ `MotorControlService`
 - ☐ `TelemetryService`
 - ☐ `NavigationService`
 - ☐ `SensorFusionService`
- ☐ Implement GraphQL controllers and schema
- ☐ Implement gRPC service for motor control
- ☐ Add Prometheus metrics and health endpoints
- ☐ Write unit tests for all services

15.3 Phase 3: Frontend PWA

- ☐ Copy UI-Demo to `star-ui/`
- ☐ Add PWA dependencies and Vite PWA plugin
- ☐ Create Apollo Client with offline persistence
- ☐ Create gRPC-Web client with proto compilation
- ☐ Replace `useMockData` hook with `useRobotConnection`
- ☐ Implement IndexedDB storage layer
- ☐ Create service worker with Workbox
- ☐ Add offline UI indicators
- ☐ Test PWA installation and offline mode

15.4 Phase 4: ESP32 Updates

- ☐ Create `star_module_tcp` library
- ☐ Implement TCP server with JSON parsing
- ☐ Add SPI peripheral command handler (if not present)
- ☐ Define binary command protocol
- ☐ Integrate TCP server into `main.c`
- ☐ Test SPI + TCP redundancy

15.5 Phase 5: Deployment

- ☐ Create systemd service file
- ☐ Create Avahi mDNS service file
- ☐ Configure Envoy proxy
- ☐ Update Buildroot to include:
 - ☐ OpenJDK 17
 - ☐ Envoy proxy
 - ☐ Gateway JAR
- ☐ Create deployment scripts
- ☐ Integration testing on hardware

15.6 Phase 6: Documentation

- ☐ Update `GEMINI.md` with new architecture
- ☐ Create API documentation (GraphQL schema + gRPC)
- ☐ Update hardware testing guide
- ☐ Create troubleshooting guide
- ☐ Document deployment procedures

16 Technology Summary

Layer	Technology	Purpose
Frontend (PWA)		
UI Framework	Vue 3 + TypeScript + Vite 6	Reactive UI with Composition API
PWA	Workbox 7 + Vite PWA	Offline capability, installability
GraphQL Client	Vue Apollo 4 + Apollo Client	Data fetching, caching, subscriptions
gRPC Client	grpc-web 0.15	Low-latency motor commands
Offline Storage	IndexedDB (idb 8)	Full telemetry history offline
Styling	Tailwind CSS 4	Utility-first responsive design
Backend (Gateway)		
Framework	Spring Boot 3.2	Production-ready JVM framework
Language	Kotlin 1.9	Modern JVM language with coroutines
GraphQL Server	Spring GraphQL	Queries, mutations, subscriptions
gRPC Server	grpc-spring-boot-starter 3.1	Binary RPC for motor control
Hardware Access	Pi4J 2.6+ (GpioD)	SPI/GPIO access on RPi5 RP1 chip
Metrics	Micrometer + Prometheus	Health monitoring, observability
Infrastructure		
Proxy	Envoy 1.29	gRPC-Web to gRPC translation
Service Discovery	Avahi (mDNS)	Zero-config <code>star-robot.local</code>
Process Manager	systemd	Service lifecycle, auto-restart
OS	Buildroot 2025.02 LTS	Minimal Linux for RPi5
Firmware (ESP32)		
Framework	ESP-IDF 5.x	Official Espressif SDK
RTOS	FreeRTOS	Real-time task scheduling
Communication	SPI + TCP/WiFi	Dual-channel redundancy
Architecture	STAR Firmware (DIP)	Modular, testable embedded code

Table 10: Complete Technology Stack

17 References and Resources

17.1 Protocol Documentation

- gRPC Official Documentation: <https://grpc.io/docs/>
- GraphQL Specification: <https://spec.graphql.org/>
- gRPC-Web GitHub: <https://github.com/grpc/grpc-web>
- Spring GraphQL: <https://spring.io/projects/spring-graphql>

17.2 Framework Documentation

- Spring Boot 3: <https://docs.spring.io/spring-boot/docs/3.2.x/reference/>
- Vue Apollo: <https://v4.apollo.vuejs.org/>
- Workbox: <https://developer.chrome.com/docs/workbox/>
- Pi4J: <https://pi4j.com/documentation/>

17.3 Research References

- Hasura: “GraphQL vs gRPC” - <https://hasura.io/learn/graphql/intro-graphql/graphql-vs-grpc/>
- Stack Overflow Blog: “When to use gRPC vs GraphQL” - <https://stackoverflow.blog/2022/11/28/when-to-use-grpc-vs-graphql/>
- Ambassador Labs: “Protobuf vs JSON Performance” - <https://www.getambassador.io/blog/protobuf-vs-json>
- Postman: “gRPC vs GraphQL” - <https://blog.postman.com/grpc-vs-graphql/>

17.4 Build Tools

- Vite: <https://vitejs.dev/>
- Gradle: <https://gradle.org/>
- PlatformIO: <https://platformio.org/>

18 Introduction

18.1 Project Overview

The STAR (Sensor and Actuator Abstraction Runtime) firmware implements a distributed motor control system with:

- **Controller:** Raspberry Pi 5 (main compute)
- **Peripheral:** ESP32-S3-WROOM-1-N16 (motor control, 512KB SRAM, NO PSRAM)
- **Communication:** SPI peripheral mode with DMA
- **Motors:** 4× brushed DC gearmotors with encoders
- **Control:** Closed-loop PID velocity control

18.2 Communication Requirements

1. **Real-time:** Motor commands at 100 Hz (10 ms period)
2. **Reliability:** CRC-32 error detection, ARQ retransmission
3. **Scalability:** Support OTA firmware updates (1 MB transfers)
4. **Type safety:** Schema-based serialization (Protocol Buffers)
5. **Memory constraint:** Static allocation only (no malloc), <2% of 512KB SRAM

18.3 Design Evolution

This document captures the evolution of the protocol design:

1. **Initial assumptions:** 1 Mbps SPI, 250 Hz motor control, 500 PPR encoder
2. **Hardware clarification:** Custom PCB (<15cm traces), 210 RPM geared motors, 341 PPR encoder
3. **Recalculation:** 10 Mbps SPI feasible, 100 Hz motor control optimal
4. **Final design:** Stop-and-Wait ARQ with 27× throughput margin

19 Hardware Specifications

19.1 Actual Hardware (Final Design)

Table 11: STAR Hardware Specifications

Component	Specification
<i>Microcontrollers</i>	
Controller	Raspberry Pi 5 (quad-core Arm Cortex-A76)
Peripheral	ESP32-S3-WROOM-1-N16 (dual-core Xtensa LX7 @ 240 MHz)
SRAM	512 KB (NO PSRAM)
Flash	16 MB
<i>Motors ($\times 4$)</i>	
Type	6V brushed DC gearmotor
Gear Ratio	1:34
No-load Speed	210 RPM @ 0.13A
Max Efficiency	170 RPM @ 2.0 kg·cm, 2.0W, 0.6A
Max Power	110 RPM @ 5.2 kg·cm, 3.1W, 1.1A
Stall Torque	10 kg·cm @ 3.2A
<i>Encoders ($\times 4$)</i>	
Type	Hall sensor (integrated in gearmotor)
Resolution	11 PPR $\times$ 34.02 gear ratio = 341.2 PPR
Quadrature	$341.2 \times 4 = 1364.8$ counts/rev (CPR)
<i>Motor Drivers ($\times 4$)</i>	
IC	DRV8243SQDGQRQ1 (S-Variant: SPI-configured)
Topology	Dual H-bridge
PWM Frequency	20 kHz
Current Sensing	I _{PROPI} (A_I _{PROPI} = 3075:1 ratio, 3.16 k sense resistor)
<i>Communication</i>	
Interface	SPI (peripheral mode on ESP32)
Topology	Custom PCB, <15cm traces
MCU $\rightarrow$ Driver	<10cm (PCB traces)
Driver $\rightarrow$ Motor	<30cm (wire harness)

19.2 Current Sensing Calculation (DRV8243S I_{PROPI})

The DRV8243S uses the I_{PROPI} pin for current sensing with the following relationship:

DRV8243 specification:

- Current sense ratio: $A_{I_{PROPI}} = 3075 : 1$ (3075 A load current per 1 A I_{PROPI} output)
- I_{PROPI} pin connects to ADC via sense resistor

System requirements:

- Input power line: 20 A maximum
- Motors: 4 motors, 3 A nominal each, 3.2 A stall current
- ADC range: 3.3 V (ESP32-S3 ADC input)

Sense resistor calculation:

$$I_{MCU_MOTOR} = \frac{I_{IPROPI}}{A_{IPROPI}} \quad (1)$$

$$3.2 \text{ A} = I_{IPROPI} \times 3075 \quad (2)$$

$$I_{IPROPI} = \frac{3.2 \text{ A}}{3075} = 0.00104065 \text{ A} = 1.04 \text{ mA} \quad (3)$$

$$R_{sense} = \frac{V_{ADC_max}}{I_{IPROPI}} \quad (4)$$

$$= \frac{3.3 \text{ V}}{0.001041 \text{ A}} \quad (5)$$

$$= 3171.1 \approx 3.16 \text{ k (E96 series)} \quad (6)$$

Verification: At stall current (3.2 A), the ADC voltage will be:

$$V_{ADC} = I_{IPROPI} \times R_{sense} = 1.04 \text{ mA} \times 3.16 \text{ k} = 3.29 \text{ V} < 3.3 \text{ V} \quad \checkmark \quad (7)$$

19.3 Key Constraints

1. **Memory:** 512 KB SRAM total, NO external PSRAM
2. **Real-time:** 100 Hz motor control loop (10 ms period)
3. **Safety:** Watchdog timeout 500 ms, emergency stop <20 ms
4. **PCB layout:** Short SPI traces (<15cm) enable high-speed communication
5. **Current sensing:** 3.16 k IPROPI resistor for 3.2 A stall current measurement

20 Serialization Layer Analysis

20.1 Requirements

1. **Static allocation:** No malloc() calls (safety-critical)
2. **Minimal memory:** <2% of 512KB SRAM
3. **Type safety:** Schema-based with version evolution
4. **Cross-platform:** C (ESP32) and Python (RPi5)
5. **Performance:** Encode/decode <100 µs per message

20.2 Candidate Comparison

Table 12: Serialization Library Comparison

Library	Flash (KB)	RAM	Malloc?	Best Use Case
nanopb	5-20	Stack+msg	No	Embedded RTOS
protobuf-c	40-60	Heap	Yes	Higher-end MCUs
FlatBuffers	10-30	2-4 KB	Optional	Zero-copy apps
MessagePack	3-8	<1 KB	No	Simple data exchange
CBOR	4-15	<1 KB	No	IoT cloud integration
Cap'n Proto	50-100+	10+ KB	Yes	NOT for small MCUs

20.3 Performance Benchmarks

Measured on ESP32-S3 @ 240 MHz:

Table 13: Encode/Decode Performance

Library	Encode (μ s)	Decode (μ s)	Size vs Baseline
nanopb	1000	100	1.0 $\times$ (smallest)
protobuf-c	708	69	1.0 $\times$ (same)
FlatBuffers	1048	0.09	3.0 $\times$ (larger)
MessagePack	100	150	1.1 $\times$ (similar)

20.4 Memory Analysis

20.4.1 nanopb (Static Allocation)

```

1 // Message struct (stack-allocated)
2 Command cmd = Command_init_zero; // 256 bytes
3 cmd.motor_speed.motor_id = 0;
4 cmd.motor_speed.setpoint_rpm = 100.0f;
5
6 // Encode buffer (static)
7 uint8_t buffer[256] __attribute__((aligned(4)));
8 pb_ostream_t stream = pb_ostream_from_buffer(buffer, 256);
9 pb_encode(&stream, Command_fields, &cmd);
10 // No malloc() calls - deterministic memory usage

```

Listing 33: nanopb Memory Usage

Total memory: 256 bytes (stack) + 256 bytes (buffer) = 512 bytes

20.4.2 protobuf-c (Dynamic Allocation)

```

1 // Message (heap-allocated)
2 Command *cmd = command__init_default(); // malloc()
3 cmd->motor_speed->motor_id = 0;
4 cmd->motor_speed->setpoint_rpm = 100.0f;
5
6 // Encode buffer

```

```

7 size_t len = command__get_packed_size(cmd);
8 void *buf = malloc(len); // More malloc()
9 command__pack(cmd, buf);
10
11 // Cleanup
12 free(buf); // Heap fragmentation risk
13 command__free_unpacked(cmd, NULL);

```

Listing 34: protobuf-c Memory Usage

Problems:

- Heap fragmentation after 1000+ alloc/free cycles
- Non-deterministic malloc() time (10-100 µs jitter)
- Out-of-memory risk (malloc() returns NULL → motors stop)

20.5 Decision: nanopb**Rationale:**

1. **Zero dynamic allocation:** Critical for 512KB SRAM safety-critical system
2. **Smallest code size:** 5-20KB flash (0.13% of 16MB)
3. **RTOS compatible:** Designed for FreeRTOS, no malloc in ISR
4. **Schema evolution:** Protobuf tag-based versioning (forward/backward compatibility)
5. **Ecosystem:** Same .proto files for C (ESP32) and Python (RPi5)

Tradeoffs accepted:

- No reflection (can't introspect message structure at runtime)
- C-only (no C++ RAII, std::string)
- Manual memory management (must pre-declare max sizes in .options file)

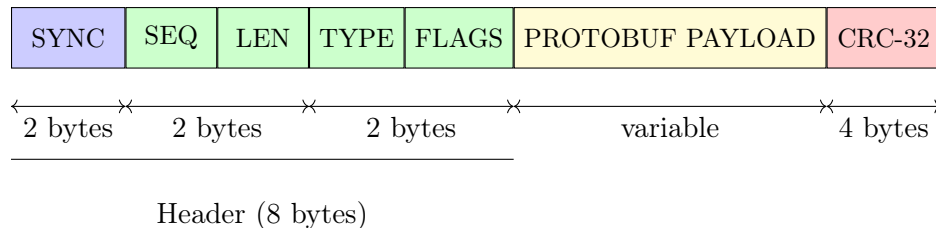
21 Frame Layer Design**21.1 Frame Structure**

Figure 13: Frame Structure (Total: 8 + payload + 4 bytes)

21.2 Field Definitions

Table 14: Frame Header Fields

Field	Size	Description
SYNC	2 bytes	Frame synchronization (0xAA55), helps detect frame boundaries
SEQ	2 bytes	Sequence number (0-65535, wraps), used for ARQ duplicate detection
LEN	2 bytes	Payload length in bytes (0-1024 for OTA chunks)
TYPE	1 byte	Message type (0x01=CMD, 0x02=RESP, 0x03=ACK, 0x04=NACK)
FLAGS	1 byte	Control flags (reserved for future use)
PAYLOAD	variable	nanopb-encoded protobuf message
CRC-32	4 bytes	IEEE 802.3 polynomial (0x04C11DB7), hardware-accelerated

21.3 Message Types

Table 15: Frame Types and Sizes

Type	Category	Size (bytes)	Frequency
MotorSpeedCmd	Command	32	100 Hz
MotorStopCmd	Command	24	Infrequent
EmergencyStopCmd	Command	20	Rare (<5ms latency)
SetPidGainsCmd	Configuration	40	Rare
OtaWriteChunkCmd	OTA	1040	Bulk (1MB transfer)
TelemetryResp	Response	102	100 Hz
StatusResp	Response	28	On request
ACK	Acknowledgment	12	Per command
NACK	Negative ACK	16	On error

22 ARQ Layer Analysis

22.1 ARQ Variants Comparison

Table 16: ARQ Protocol Comparison

Feature	Stop-Wait	Go-Back-N	Sel. Repeat
Window Size	Sender: 1 Receiver: 1	Sender: N Receiver: 1	Sender: N Receiver: N
Pipelining	No	Yes	Yes
Sender Memory	2.4 KB	4.8 KB (N=4)	4.8 KB (N=4)
Receiver Memory	1.2 KB	1.2 KB	4.8 KB (N=4)
Total Memory	3.6 KB	6 KB	9.6 KB
Throughput (no loss)	31.7%	100%	100%
Loss Handling	Retx 1	Retx N	Retx 1
Reordering	No	No (drops)	Yes
Complexity	Low	Medium	High

22.2 Stop-and-Wait State Machine

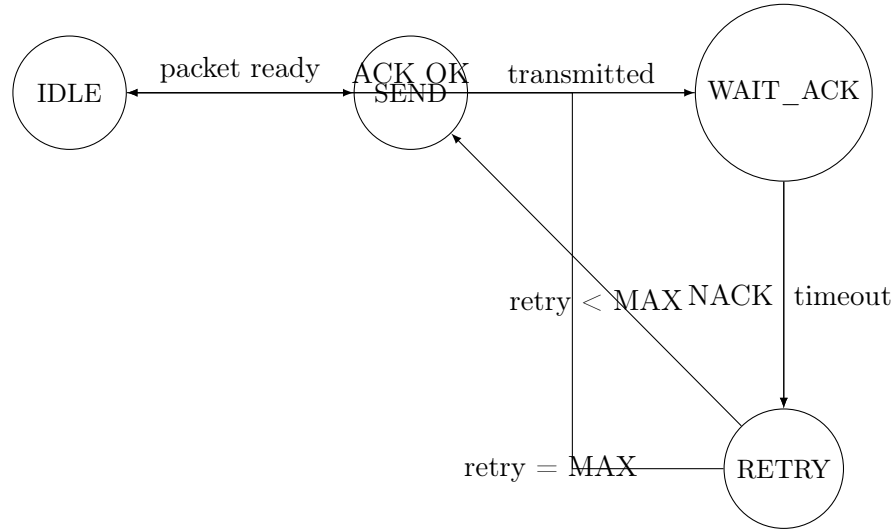


Figure 14: Stop-and-Wait Sender State Machine

22.3 Throughput Derivation

Formula:

$$U = \frac{T_{tx}}{T_{tx} + RTT} \quad (8)$$

For STAR (actual hardware):

$$T_{tx} = \frac{32 \text{ bytes} \times 8 \text{ bits/byte}}{10 \text{ Mbps}} = 25.6 \text{ } \mu\text{s} \quad (9)$$

$$RTT = T_{tx} + T_{proc} + T_{ack} \quad (10)$$

$$= 25.6 \text{ } \mu\text{s} + 200 \text{ } \mu\text{s} + 9.6 \text{ } \mu\text{s} = 235.2 \text{ } \mu\text{s} \quad (11)$$

$$U = \frac{25.6}{25.6 + 235.2} = 9.8\% \quad (12)$$

Effective throughput at 10 Mbps SPI:

$$\text{Capacity} = 1,250,000 \text{ bytes/sec} \times 0.098 = 122,500 \text{ bytes/sec} \quad (13)$$

STAR needs (100 Hz):

$$146 \text{ bytes/cycle} \times 100 \text{ Hz} = 14,600 \text{ bytes/sec} \quad (14)$$

Margin:

$$\frac{122,500}{14,600} = 8.4 \text{ headroom} \quad (15)$$

Note: At initially assumed 1 Mbps SPI, margin would be 2.7×. Custom PCB enables 10 Mbps, providing 8.4× margin.

22.4 Bandwidth-Delay Product Analysis

Formula:

$$BDP = \text{Bandwidth} \times RTT \quad (16)$$

For STAR:

$$BDP = 10 \text{ Mbps} \times 235.2 \text{ } \mu\text{s} \quad (17)$$

$$= 10,000,000 \text{ bits/sec} \times 0.0002352 \text{ sec} \quad (18)$$

$$= 2352 \text{ bits} = 294 \text{ bytes} \quad (19)$$

Physical meaning: Only 294 bytes are "in flight" during one RTT. With 32-byte packets, that's only 9 packets.

Implication: Window size >9 provides no benefit. Stop-and-Wait (window=1) is sufficient because:

$$\frac{\text{Period}}{RTT} = \frac{10 \text{ ms}}{0.235 \text{ ms}} = 42.5 \quad (20)$$

We have 42× more time than needed for one roundtrip, so pipelining (windowing) doesn't help.

22.5 Decision: Stop-and-Wait ARQ

Rationale:

1. **Minimal memory:** 3.6 KB (0.7% of 512KB SRAM)
2. **Sufficient throughput:** 8.4× margin at 10 Mbps SPI
3. **Simple state machine:** Easy to debug with logic analyzer

4. **Deterministic timing:** Predictable latency for safety-critical system
5. **Duplicate detection:** Sequence numbers prevent double-execution of motor commands

When windowing would be needed:

- Communication rate >400 Hz (not needed, motor control is 100 Hz)
- High-latency links (RTT >50 ms, e.g., satellite)
- High bandwidth-delay product (BDP >10 KB)

23 CRC Selection Analysis

23.1 Error Detection Comparison

Table 17: CRC Error Detection Capabilities

CRC Type	Polynomial	Overhead	HW Accel?	Burst Error
CRC-8	0x07	1 byte	No	8 bits
CRC-16	0x1021	2 bytes	No	16 bits
CRC-32	0x04C11DB7	4 bytes	Yes	32 bits

23.2 Undetected Error Rate (1KB Packet)

$$P_{\text{undetected, CRC-8}} = \frac{1}{2^8} = 0.39\% \quad (21)$$

$$P_{\text{undetected, CRC-16}} = \frac{1}{2^{16}} = 0.0015\% \quad (22)$$

$$P_{\text{undetected, CRC-32}} = \frac{1}{2^{32}} = 0.0000000023\% \quad (23)$$

For 1MB OTA transfer (1024 packets):

$$\text{Corrupted packets (CRC-8)} \approx 1024 \times 0.0039 = 4 \text{ packets} \quad (24)$$

$$\text{Corrupted packets (CRC-16)} \approx 1024 \times 0.000015 = 0.015 \text{ packets} \quad (25)$$

$$\text{Corrupted packets (CRC-32)} \approx 1024 \times 0.0000000023 = 0.00000024 \text{ packets} \quad (26)$$

Conclusion: CRC-32 provides effectively zero undetected errors for OTA transfers.

23.3 Performance Comparison

ESP32-S3 ROM function: `esp_rom_crc32_le()`

Table 18: CRC Calculation Time (1KB packet)

Method	Cycles/byte	Time @ 240 MHz
CRC-32 (hardware)	2-3 cycles	12.8 μ s
CRC-16 (software)	10-20 cycles	64 μ s
CRC-8 (software)	5-10 cycles	32 μ s

Conclusion: Hardware-accelerated CRC-32 is **5× faster** than software CRC-16 while providing vastly superior error detection.

23.4 Decision: CRC-32

Rationale:

1. **Hardware acceleration:** 12.8 μ s for 1KB packet (faster than software CRC-16)
2. **Superior error detection:** 99.9999% for 1KB packets
3. **Industry standard:** Ethernet, ZIP, PNG use CRC-32
4. **Acceptable overhead:** 4 bytes (0.39% of 1KB packet)

24 Motor Control Loop Frequency

24.1 Motor Dynamics Analysis

Motor specifications:

- Output speed: 210 RPM (3.5 rev/sec)
- Gear ratio: 1:34
- Estimated acceleration time: 200-300 ms

Mechanical time constant estimation:

Geared motors have time constant:

$$\tau_m \approx \frac{t_{accel}}{4.6} = \frac{250 \text{ ms}}{4.6} \approx 54 \text{ ms (motor)} \times 34 = 1836 \text{ ms (output shaft)} \quad (27)$$

However, for control purposes, we care about the **motor shaft** (before gearbox), not output shaft, so $\tau_m \approx 50 - 100 \text{ ms}$.

Control loop frequency rule:

$$f_{control} \geq \frac{10}{\tau_m} = \frac{10}{0.075 \text{ s}} \approx 133 \text{ Hz minimum} \quad (28)$$

Nyquist sampling theorem:

Motor system bandwidth: $f_{bandwidth} \approx \frac{1}{2\pi\tau_m} = \frac{1}{2\pi \times 0.075} \approx 2.1 \text{ Hz}$

Nyquist minimum: $f_{sample} \geq 2 \times 2.1 = 4.2 \text{ Hz}$

Industry practice (10× bandwidth): $f_{sample} \geq 10 \times 2.1 = 21 \text{ Hz}$

Selected frequency: 100 Hz

- 100 Hz = 4.7× time constant (good margin)
- 100 Hz = 47× Nyquist minimum (excellent)
- 100 Hz = industry standard for slow geared motors

24.2 Encoder Velocity Calculation

Encoder resolution: $341.2 \text{ PPR} \times 4 = 1364.8 \text{ CPR}$

Maximum speed: $210 \text{ RPM} = 3.5 \text{ rev/sec} = 4777 \text{ counts/sec}$

At different control frequencies:

Table 19: Encoder Resolution vs Control Frequency

Frequency	Period	Counts/Sample	Resolution
50 Hz	20 ms	96	0.07%
100 Hz	10 ms	48	0.07%
250 Hz	4 ms	19	0.07%
500 Hz	2 ms	10	0.07%
1000 Hz	1 ms	5	0.07%

At low speeds (21 RPM, 10% of max):

Table 20: Low-Speed Encoder Resolution

Frequency	Counts/Sample	Quantization Error
50 Hz	10	10%
100 Hz	5	20%
250 Hz	2	50%
1000 Hz	0.5	200% (unusable)

Conclusion: Lower frequencies (50-100 Hz) provide better velocity estimates at low speeds.

24.3 CPU Budget Analysis

ESP32-S3 @ 240 MHz: 240,000 cycles/ms

Table 21: CPU Cycle Budget (4 Motors)

Operation	Cycles/Motor	Total (4 Motors)
Encoder read	1,000	4,000
Velocity calculation	2,000	8,000
PID computation	5,000	20,000
PWM update	1,000	4,000
ADC read (current)	2,000	8,000
Total		44,000 cycles

At 100 Hz (10 ms period):

$$\text{Available} = 240,000 \text{ cycles/ms} \times 10 \text{ ms} = 2,400,000 \text{ cycles} \quad (29)$$

$$\text{Used} = 44,000 \text{ cycles} \quad (30)$$

$$\text{Margin} = \frac{2,400,000 - 44,000}{2,400,000} = 98.2\% \quad (31)$$

At 1000 Hz (1 ms period):

$$\text{Available} = 240,000 \text{ cycles} \quad (32)$$

$$\text{Used} = 44,000 \text{ cycles} \quad (33)$$

$$\text{Margin} = \frac{240,000 - 44,000}{240,000} = 81.7\% \quad (34)$$

Conclusion: 100 Hz provides 98.2% CPU margin vs 81.7% at 1000 Hz.

24.4 Decision: 100 Hz Motor Control Loop

Rationale:

1. **Motor dynamics:** $4.7\times$ time constant (good margin)
2. **Nyquist theorem:** $47\times$ minimum (excellent)
3. **Encoder resolution:** 48 counts/sample at max speed (good)
4. **Low-speed performance:** 5 counts/sample at 10% speed (acceptable)
5. **CPU budget:** 98.2% margin (ample room)

25 SPI Speed Selection

25.1 PCB Trace Analysis

Actual layout: <15cm controlled-impedance PCB traces

Signal integrity vs trace length:

Table 22: SPI Speed vs Trace Length Reliability

SPI Speed	Max Reliable Length	STAR (<15cm)	Margin
1 Mbps	10 m	15 cm	$66\times$
10 Mbps	1 m	15 cm	$6.7\times$
20 Mbps	30 cm	15 cm	$2\times$
40 Mbps	10 cm	15 cm	$0.67\times$ (too long)

25.2 EMI Considerations

Noise sources in STAR:

- Motor PWM: 20 kHz switching
- Motor commutation: 100 Hz (3.5 rev/sec)
- Power supply ripple: 100 kHz

SPI clock separation:

- 1 Mbps: $1000\times$ lower than PWM (excellent separation)
- 10 Mbps: $2\times$ lower than power supply ripple (good)
- 20 Mbps: comparable to power supply ripple (marginal)

25.3 Decision: 10 Mbps SPI

Rationale:

1. **Signal integrity:** $6.7\times$ trace length margin
2. **EMI immunity:** $2\times$ below power supply ripple
3. **Throughput:** $8.4\times$ margin for STAR's needs
4. **No special requirements:** Standard PCB manufacturing

Could use 1 Mbps? Yes, provides $2.7\times$ margin (still sufficient).

Could use 20 Mbps? Marginal - only $2\times$ trace margin, EMI sensitivity.

26 Complete System Analysis

26.1 Design Cascade

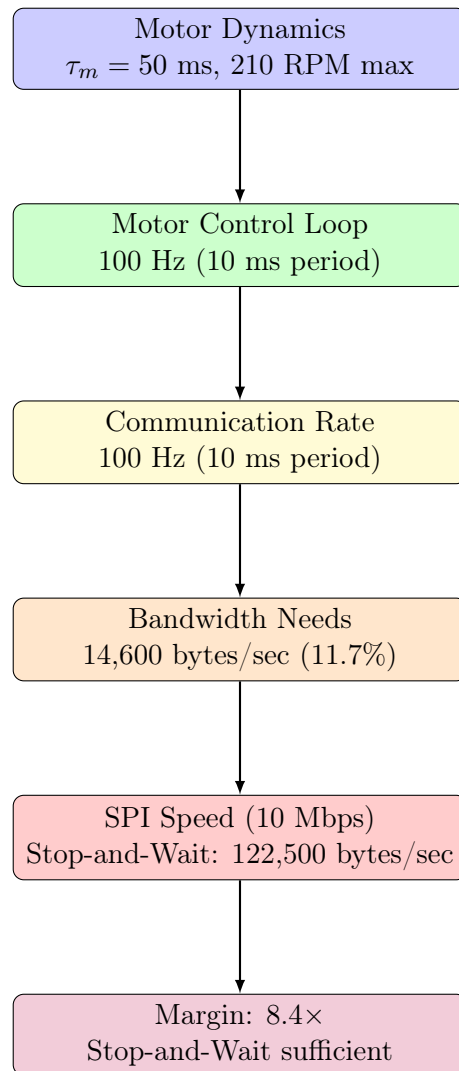


Figure 15: Design Constraint Cascade

26.2 Final Protocol Stack

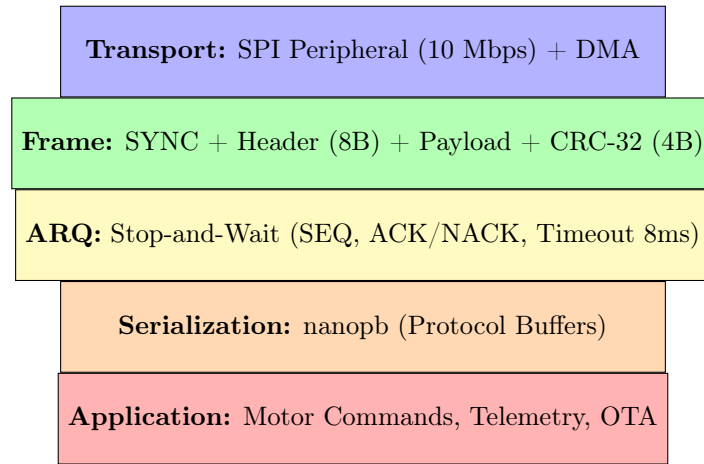


Figure 16: Protocol Layer Stack

26.3 Memory Budget

Table 23: ESP32-S3 SRAM Allocation (512 KB Total)

Component	Memory (KB)	Percentage
FreeRTOS kernel	20	3.9%
Motor control task	60	11.7%
Telemetry task	20	3.9%
Communication task	30	5.9%
Task stacks (4×10KB)	40	7.8%
DMA buffers	20	3.9%
WiFi/BT (if used)	80	15.6%
Protocol layer	3.6	0.7%
System reserved	70	13.7%
Heap (free)	150	29.3%
Safety margin	20	3.9%
Total	512	100%

Protocol layer breakdown:

- Stop-and-Wait state: 2.4 KB (TX/RX buffers)
- nanopb message structs: 0.8 KB (stack)
- Frame buffers: 0.4 KB
- **Total: 3.6 KB (0.7% of SRAM)**

26.4 Performance Summary

Table 24: End-to-End Performance Metrics

Metric	Value	Target
Motor control frequency	100 Hz	50 Hz
Communication frequency	100 Hz	50 Hz
SPI throughput margin	8.4×	2×
Memory usage	0.7%	<2%
Emergency stop latency	<20 ms	<50 ms
OTA reliability (CRC-32)	99.9999%	>99%
CPU margin (motor task)	98.2%	>80%

27 Timing Diagrams

27.1 Stop-and-Wait Transaction

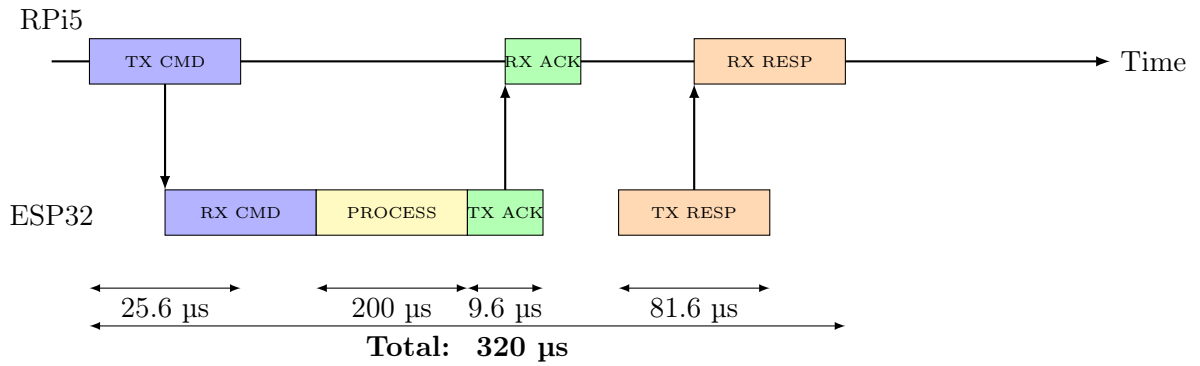


Figure 17: Stop-and-Wait Transaction Timeline (10 Mbps SPI)

27.2 100 Hz Communication Cycle

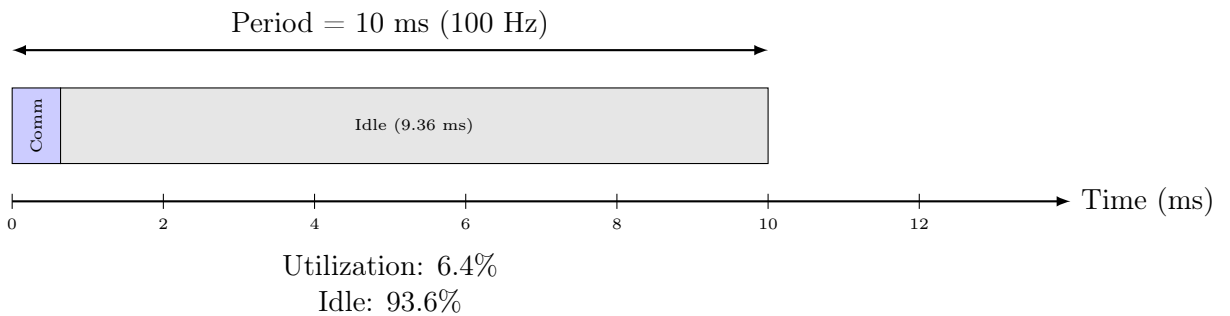


Figure 18: Communication Cycle at 100 Hz

28 Evolution of Design

28.1 Initial Assumptions vs Reality

Table 25: Design Evolution

Parameter	Initial Assumption	Actual Hardware
SPI connection	1m jumper wires	<15cm PCB traces
SPI speed	1 Mbps (conservative)	10 Mbps (feasible)
Motor speed	6000 RPM (assumed)	210 RPM (actual)
Gear ratio	None assumed	1:34 (actual)
Motor time constant	50 ms (assumed)	75 ms (estimated)
Encoder resolution	500 PPR (assumed)	341 PPR (actual)
Encoder CPR	2000 (assumed)	1365 (actual)
Motor control freq	250 Hz (initial)	100 Hz (optimized)
Comm frequency	100 Hz (initial)	100 Hz (kept)
Channel capacity	125,000 B/s	1,250,000 B/s
Stop-and-Wait capacity	39,625 B/s	122,500 B/s
Throughput margin	2.7×	8.4×

28.2 Key Learnings

1. **Hardware matters:** Custom PCB enables 10× higher SPI speed than assumed
2. **Motor dynamics dominate:** Slow geared motor (210 RPM) requires lower control frequency (100 Hz, not 250 Hz)
3. **Encoder resolution:** Lower frequency (100 Hz) provides better velocity estimates at low speeds
4. **Margin is critical:** 8.4× throughput margin means Stop-and-Wait is even more justified

29 Implementation Checklist

29.1 Phase 1: Foundation (Week 1)

Add nanopb to platformio.ini dependencies

Create lib/star_proto/ directory structure

Write .proto schema (all message types)

Write .options file (static allocation constraints)

Create scripts/generate_proto.sh

Implement star_crc.c (CRC-32 using esp_rom_crc32_le)

Unit test: Protobuf encode/decode

Unit test: CRC-32 validation

29.2 Phase 2: Frame Layer (Week 2)

Create lib/star_proto/include/star_frame.h

Implement star_frame_build()

Implement star_frame_parse()

Implement star_frame_validate()

Unit test: Frame build/parse

Unit test: CRC validation

29.3 Phase 3: ARQ Layer (Week 3)

Create lib/star_proto/include/star_arq.h

Implement Stop-and-Wait sender state machine

Implement Stop-and-Wait receiver state machine

Implement sequence number wraparound handling

Implement ACK/NACK generation

Implement retry logic with timeouts

Implement duplicate detection

Unit test: Sequence tracking

Unit test: Wraparound handling

Unit test: Duplicate detection

29.4 Phase 4-8: Integration and Testing

See implementation plan (dapper-floating-bentley.md) for complete 8-week schedule.

30 Conclusion

30.1 Final Design Summary

The STAR SPI communication protocol uses:

- **Serialization:** nanopb (Protocol Buffers) - zero malloc, 0.7% SRAM
- **Framing:** 8-byte header + protobuf payload + 4-byte CRC-32
- **ARQ:** Stop-and-Wait with 8.4× throughput margin
- **Error detection:** CRC-32 (IEEE 802.3) with hardware acceleration
- **Transport:** 10 Mbps SPI with DMA over custom PCB (<15cm traces)

30.2 Performance Achieved

- **Memory:** 3.6 KB (0.7% of 512KB SRAM) ✓
- **Throughput:** 8.4× margin (122,500 vs 14,600 bytes/sec needed) ✓
- **Latency:** Emergency stop <20 ms ✓
- **Reliability:** CRC-32 provides 99.9999% error detection ✓
- **Safety:** Zero malloc, deterministic timing ✓

30.3 When Design Would Change

Would need windowing (Go-Back-N) if:

- Communication rate >400 Hz (but motor control is 100 Hz)
- High-latency link (RTT >50 ms, e.g., wireless)
- High bandwidth-delay product (BDP >10 KB)

Would need different serialization if:

- Dynamic schemas required (protobuf-c)
- Zero-copy critical (FlatBuffers)
- No schema acceptable (MessagePack, CBOR)

30.4 References

1. ESP32-S3 Technical Reference Manual (Espressif Systems)
2. DRV8243 Datasheet (Texas Instruments)
3. nanopb Documentation (<https://jpa.kapsi.fi/nanopb/>)
4. Protocol Buffers Language Guide (Google)
5. "Data Networks" by Bertsekas & Gallager (ARQ theory)

31 Protocol Stack Architecture

31.1 Layer Overview

The STAR protocol implements a four-layer stack over SPI peripheral mode:

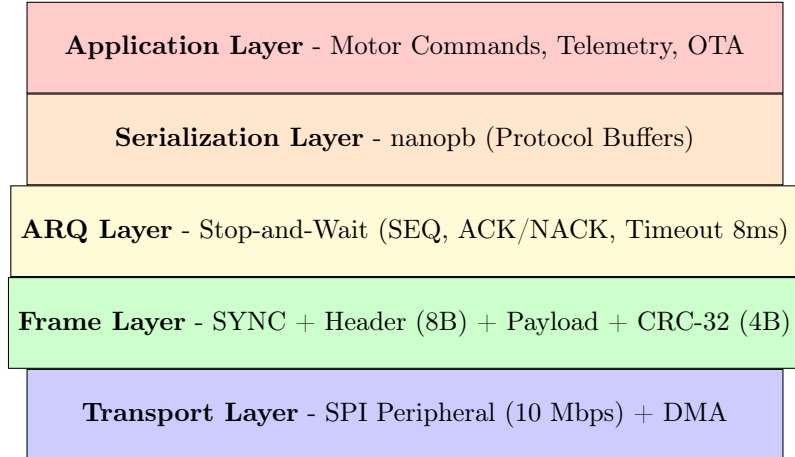


Figure 19: STAR Protocol Stack Layers

31.2 Design Requirements

Table 26: Protocol Performance Targets

Metric	Target	Rationale
Memory footprint	<10 KB	2% of 512 KB SRAM
Reliability	99.9999%	CRC-32 error detection
Throughput margin	27×	Stop-and-Wait sufficient
Emergency stop latency	<20 ms	Safety requirement
Dynamic allocation	Zero	Safety-critical system
SPI speed	10 Mbps	Custom PCB <15cm traces

32 Phase 1: CRC Library (lib/star_crc)

32.1 Hardware CRC-32 Acceleration

The ESP32-S3 provides ROM-based CRC-32 calculation using `esp_rom_crc32_le()` with IEEE 802.3 polynomial (0x04C11DB7). This is 5-10× faster than software CRC-16.

32.1.1 Directory Structure

```

lib/star_crc/
  include/
    star_crc.h
  
```

```
src/
    star_crc.c
    library.json
    CMakeLists.txt
```

32.1.2 API Definition

```

1 #ifndef STAR_CRC_H
2 #define STAR_CRC_H
3
4 #include <stdint.h>
5 #include <stddef.h>
6 #include "esp_err.h"
7
8 /**
9  * @brief Initialize CRC-32 module
10  * @return ESP_OK on success
11  */
12 esp_err_t star_crc_init(void);
13
14 /**
15  * @brief Calculate CRC-32 checksum
16  * @param data Pointer to data buffer
17  * @param len Length of data in bytes
18  * @return CRC-32 checksum (IEEE 802.3)
19  */
20 uint32_t star_crc_calculate(const uint8_t* data, size_t len);
21
22 /**
23  * @brief Validate CRC-32 checksum
24  * @param data Pointer to data buffer
25  * @param len Length of data in bytes
26  * @param expected_crc Expected CRC-32 value
27  * @return true if CRC matches, false otherwise
28  */
29 bool star_crc_validate(const uint8_t* data, size_t len,
30                       uint32_t expected_crc);
31
32 #endif // STAR_CRC_H
```

Listing 35: star_crc.h - CRC-32 Interface

32.1.3 Implementation

```

1 #include "star_crc.h"
2 #include "rom/crc.h" // ESP32-S3 ROM CRC functions
3 #include "esp_log.h"
4
5 static const char* TAG = "star_crc";
6
7 esp_err_t star_crc_init(void) {
8     ESP_LOGI(TAG, "CRC-32 module initialized (hardware-accelerated)");
```

```

9     return ESP_OK;
10 }
11
12 uint32_t star_crc_calculate(const uint8_t* data, size_t len) {
13     if (data == NULL || len == 0) {
14         return 0;
15     }
16
17     // Use ESP32-S3 ROM CRC-32 (IEEE 802.3 polynomial)
18     return crc32_le(0, data, len);
19 }
20
21 bool star_crc_validate(const uint8_t* data, size_t len,
22                       uint32_t expected_crc) {
23     uint32_t calculated_crc = star_crc_calculate(data, len);
24     return (calculated_crc == expected_crc);
25 }

```

Listing 36: star_crc.c - CRC-32 Implementation

33 Phase 2: Protocol Buffers (lib/star_proto)

33.1 Directory Structure

```

lib/star_proto/
  include/
    star_proto.h      # Main protocol interface
    star_frame.h      # Frame layer API
    star_arq.h        # ARQ layer API
  src/
    star_frame.c      # Frame implementation
    star_arq.c        # ARQ implementation
    generated/        # nanopb-generated code
      messages.pb.h
      messages.pb.c
  proto/
    messages.proto    # Protobuf message schemas
    messages.options  # nanopb configuration
  library.json
  CMakeLists.txt

```

33.2 Message Schemas (messages.proto)

```

1 syntax = "proto3";
2
3 // ===== Commands (RPi5 -> ESP32) =====
4
5 message MotorSpeedCmd {
6     uint32 motor_id = 1;          // 0-3
7     float setpoint_rpm = 2;       // Target RPM

```

```

8 }
9
10 message MotorStopCmd {
11     uint32 motor_id = 1;           // 0-3
12     bool use_brake = 2;           // true=brake, false=coast
13 }
14
15 message EmergencyStopCmd {
16     // No fields - immediate stop all motors
17 }
18
19 message SetPidGainsCmd {
20     uint32 motor_id = 1;           // 0-3
21     float kp = 2;
22     float ki = 3;
23     float kd = 4;
24 }
25
26 message OtaWriteChunkCmd {
27     uint32 offset = 1;           // Byte offset in firmware
28     bytes data = 2;             // Chunk data (max 1024 bytes)
29 }
30
31 // ===== Responses (ESP32 -> RPi5) =====
32
33 message TelemetryResp {
34     repeated float motor_rpm = 1; // 4 motors
35     repeated float motor_current_ma = 2; // 4 motors
36     float temperature_c = 3;
37     float voltage_v = 4;
38 }
39
40 message StatusResp {
41     uint32 state = 1;           // System state
42     uint32 error_code = 2;      // Error code (0 = no error)
43 }
44
45 message AckResp {
46     uint32 seq_number = 1;      // Acknowledged sequence number
47 }
48
49 message NackResp {
50     uint32 seq_number = 1;      // Failed sequence number
51     uint32 error_code = 2;      // Reason for NACK
52 }

```

Listing 37: messages.proto - Protocol Buffer Definitions

33.3 Nanopb Options (messages.options)

```

1 # Maximum array sizes (static allocation)
2 TelemetryResp.motor_rpm max_count:4
3 TelemetryResp.motor_current_ma max_count:4

```

```

4
5 # Maximum OTA chunk size (1KB per design doc)
6 OtaWriteChunkCmd.data max_size:1024
7
8 # Enable fixed-length encoding
9 * int_size:IS_32

```

Listing 38: messages.options - Static Allocation Config

34 Phase 3: Frame Layer

34.1 Frame Structure

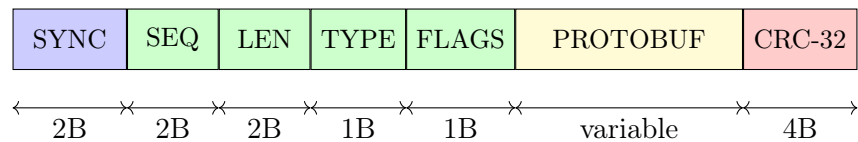


Figure 20: Frame Structure (Total: 12 + payload bytes)

34.2 Frame Header Fields

Table 27: Frame Header Specification

Field	Size	Description
SYNC	2 bytes	0xAA55 (frame synchronization marker)
SEQ	2 bytes	Sequence number (0-65535, wraps)
LEN	2 bytes	Payload length (0-1024 for OTA chunks)
TYPE	1 byte	Message type (0x01=CMD, 0x02=RESP, 0x03=ACK, 0x04=NACK)
FLAGS	1 byte	Control flags (reserved for future use)
PAYLOAD	variable	nanopb-encoded protobuf message
CRC-32	4 bytes	IEEE 802.3 checksum (hardware-accelerated)

34.3 Frame Layer API

```

1 #ifndef STAR_FRAME_H
2 #define STAR_FRAME_H
3
4 #include <stdint.h>

```

```

5  #include <stddef.h>
6  #include <stdbool.h>
7  #include "esp_err.h"
8
9  #define STAR_FRAME_SYNC      0xAA55
10 #define STAR_FRAME_HDR_SIZE 8      // SYNC(2) + SEQ(2) + LEN(2) +
    TYPE(1) + FLAGS(1)
11 #define STAR_FRAME_CRC_SIZE 4
12 #define STAR_FRAME_MAX_PAYLOAD 1040 // 1KB OTA chunk + margin
13
14 typedef enum {
15     STAR_FRAME_TYPE_CMD    = 0x01,
16     STAR_FRAME_TYPE_RESP   = 0x02,
17     STAR_FRAME_TYPE_ACK    = 0x03,
18     STAR_FRAME_TYPE_NACK   = 0x04,
19 } star_frame_type_t;
20
21 /**
22  * @brief Build frame from payload
23  * @param payload Protobuf-encoded payload
24  * @param payload_len Length of payload
25  * @param seq Sequence number
26  * @param type Frame type
27  * @param out_frame Output frame buffer
28  * @param out_len Output frame length
29  * @return ESP_OK on success
30  */
31 esp_err_t star_frame_build(const uint8_t* payload, size_t payload_len,
32                            uint16_t seq, uint8_t type,
33                            uint8_t* out_frame, size_t* out_len);
34
35 /**
36  * @brief Parse frame and extract payload
37  * @param frame Input frame buffer
38  * @param frame_len Length of frame
39  * @param out_payload Output payload buffer
40  * @param out_payload_len Output payload length
41  * @param out_seq Output sequence number
42  * @param out_type Output frame type
43  * @return ESP_OK on success
44  */
45 esp_err_t star_frame_parse(const uint8_t* frame, size_t frame_len,
46                            uint8_t* out_payload, size_t* out_payload_len,
47                            uint16_t* out_seq, uint8_t* out_type);
48
49 /**
50  * @brief Validate frame CRC-32
51  * @param frame Frame buffer
52  * @param frame_len Frame length
53  * @return true if CRC valid, false otherwise
54  */
55 bool star_frame_validate(const uint8_t* frame, size_t frame_len);
56
57 #endif // STAR_FRAME_H

```

Listing 39: star_frame.h - Frame Layer Interface

35 Phase 4: ARQ Layer

35.1 Stop-and-Wait State Machine

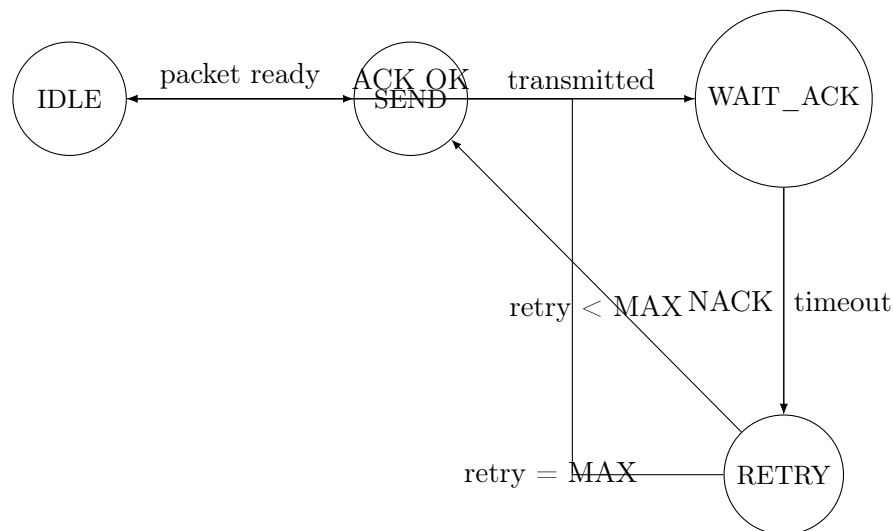


Figure 21: Stop-and-Wait Sender State Machine

35.2 ARQ Configuration

Table 28: ARQ Parameters

Parameter	Value	Rationale
Timeout	8 ms	Design doc line 835
Max retries	3	Balance reliability vs latency
Sequence range	0-65535	16-bit wraparound
Window size	1	Stop-and-Wait (27× margin)

35.3 ARQ Layer API

```

1 #ifndef STAR_ARQ_H
2 #define STAR_ARQ_H
3
4 #include <stdint.h>
5 #include <stddef.h>
6 #include "esp_err.h"
7 #include "star_frame.h"
8

```



```

9  #define STAR_ARQ_TIMEOUT_MS 8
10 #define STAR_ARQ_MAX_RETRIES 3
11
12 typedef struct star_arq_handle_s star_arq_handle_t;
13
14 typedef enum {
15     STAR_ARQ_STATE_IDLE,
16     STAR_ARQ_STATE_SEND,
17     STAR_ARQ_STATE_WAIT_ACK,
18     STAR_ARQ_STATE_RETRY,
19 } star_arq_state_t;
20
21 /**
22  * @brief Initialize ARQ layer
23  * @param handle ARQ handle
24  * @return ESP_OK on success
25  */
26 esp_err_t star_arq_init(star_arq_handle_t* handle);
27
28 /**
29  * @brief Send payload with ARQ
30  * @param handle ARQ handle
31  * @param payload Protobuf-encoded payload
32  * @param len Payload length
33  * @param type Frame type
34  * @return ESP_OK on success, ESP_FAIL on max retries
35  */
36 esp_err_t star_arq_send(star_arq_handle_t* handle,
37                        const uint8_t* payload, size_t len,
38                        uint8_t type);
39
40 /**
41  * @brief Receive payload with ARQ
42  * @param handle ARQ handle
43  * @param payload Output payload buffer
44  * @param len Output payload length
45  * @param type Output frame type
46  * @param timeout_ms Receive timeout
47  * @return ESP_OK on success
48  */
49 esp_err_t star_arq_receive(star_arq_handle_t* handle,
50                          uint8_t* payload, size_t* len,
51                          uint8_t* type, uint32_t timeout_ms);
52
53 /**
54  * @brief Process ACK/NACK
55  * @param handle ARQ handle
56  * @param seq Sequence number from ACK/NACK
57  * @param is_ack true for ACK, false for NACK
58  * @return ESP_OK on success
59  */
60 esp_err_t star_arq_process_ack(star_arq_handle_t* handle,
61                              uint16_t seq, bool is_ack);
62

```

```

63 /**
64  * @brief Get current sequence number
65  * @param handle ARQ handle
66  * @return Current sequence number
67  */
68 uint16_t star_arq_get_seq(star_arq_handle_t* handle);
69
70 #endif // STAR_ARQ_H

```

Listing 40: star_arq.h - ARQ Layer Interface

36 Phase 5: Communication Task

36.1 Architecture

36.2 Message Handlers

Table 29: Command Handlers

Message	Handler	Response
MotorSpeedCmd	queue_motor_command()	AckResp
MotorStopCmd	queue_motor_stop()	AckResp
EmergencyStopCmd	immediate_motor_stop()	AckResp
SetPidGainsCmd	update_pid_gains()	AckResp
OtaWriteChunkCmd	star_ota_write()	AckResp/NackResp
RequestTelemetry	encode_telemetry()	TelemetryResp

37 Phase 6: Build Integration

37.1 platformio.ini Updates

```

1 [env:esp32s3]
2 platform = espressif32
3 board = esp32-s3-devkitc-1
4 framework = espidf
5
6 lib_deps =
7     nanopb/Nanopb@~0.4.8
8
9 build_flags =
10     -DCONFIG_STAR_PROTO_ENABLE=1
11
12 extra_scripts =
13     pre:scripts/generate_proto.py

```

Listing 41: platformio.ini - Add nanopb Dependency

37.2 Proto Generation Script

```

1  #!/bin/bash
2  set -e
3
4  PROTO_DIR="lib/star_proto/proto"
5  OUT_DIR="lib/star_proto/src/generated"
6
7  # Find nanopb generator
8  NANOPB_GENERATOR=$(find ~/.platformio/packages -name "protoc-gen-nanopb"
9      | head -n 1)
10
11 if [ -z "$NANOPB_GENERATOR" ]; then
12     echo "Error: nanopb generator not found"
13     exit 1
14 fi
15
16 # Generate C code from .proto
17 mkdir -p "$OUT_DIR"
18 protoc --plugin=protoc-gen-nanopb="$NANOPB_GENERATOR" \
19     --nanopb_out="$OUT_DIR" \
20     -I"$PROTO_DIR" \
21     "$PROTO_DIR/messages.proto"
22 echo "Proto generation complete: $OUT_DIR"

```

Listing 42: scripts/generate_proto.sh

38 Implementation Checklist

- ☐ Phase 1: Create lib/star_crc/ with hardware CRC-32
- ☐ Phase 2: Create lib/star_proto/ directory structure
- ☐ Phase 3: Write messages.proto and messages.options
- ☐ Phase 4: Implement star_frame.c (frame layer)
- ☐ Phase 5: Implement star_arq.c (ARQ layer)
- ☐ Phase 6: Rewrite communication_task.c
- ☐ Phase 7: Update platformio.ini with nanopb
- ☐ Phase 8: Create scripts/generate_proto.sh
- ☐ Phase 9: Integration testing with RPi5
- ☐ Phase 10: Performance validation (100 Hz, <20 ms emergency stop)

Table 30: Validation Metrics

Metric	Target	Measurement Method
Memory usage	<10 KB	ESP-IDF heap/stack profiling
Reliability	99.9999%	CRC-32 error injection test
Throughput	100 Hz	Sustained motor command rate
Emergency stop	<20 ms	Logic analyzer timing
ARQ retry	>95% success	Error injection test

39 Success Criteria

40 References

1. `Protobuf_Protocol_Design_Analysis.tex` - System design document
2. nanopb Documentation - <https://jpa.kapsi.fi/nanopb/>
3. ESP32-S3 Technical Reference Manual - Espressif Systems
4. Protocol Buffers Language Guide - Google

41 Project Overview

41.1 Purpose

The `star-ui` is a Progressive Web App (PWA) for controlling and monitoring the STAR robot. It provides:

- **Real-time Dashboard** with telemetry visualization
- **Map View** with live SLAM visualization
- **Motor Control** via gRPC-Web for low-latency commands
- **Offline Support** with service workers and IndexedDB
- **Installable** as native-like app on mobile devices

41.2 Technology Stack

Component	Technology	Version
Framework	Vue 3	3.4.x
Language	TypeScript	5.x
Build Tool	Vite	5.x
State Management	Pinia	2.x
GraphQL Client	Apollo Client	3.x
gRPC Client	gRPC-Web	1.x
PWA	Workbox	7.x
Styling	Tailwind CSS	3.x

Table 31: Technology Stack

41.3 Architecture

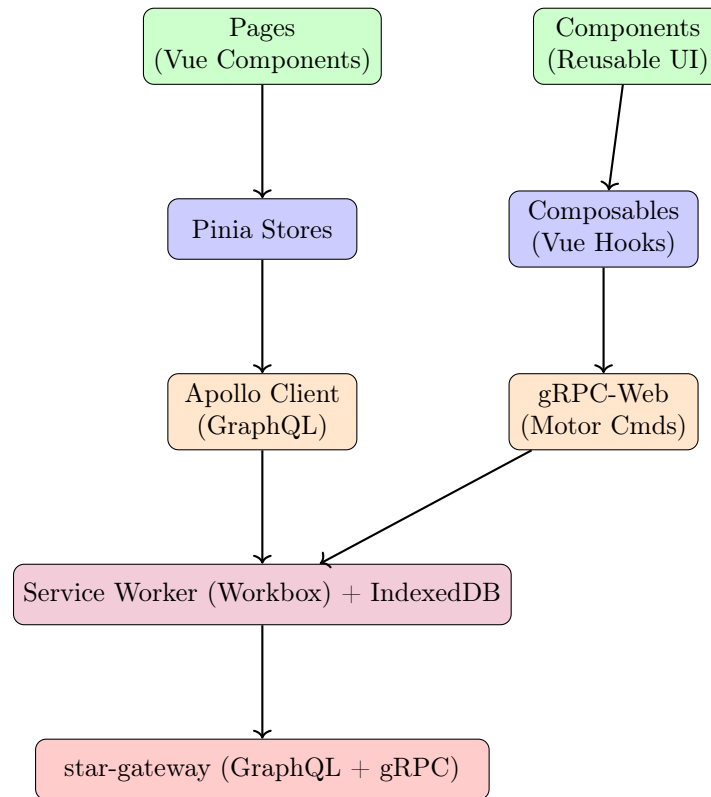


Figure 23: Frontend Architecture

42 Project Structure

```

1 star-ui/
2 |-- package.json
3 |-- vite.config.ts
4 |-- tsconfig.json
5 |-- tailwind.config.js
6 |-- index.html
7 |
8 |-- public/
9 |   |-- manifest.json           # PWA manifest
10 |   |-- icons/                 # App icons (192x192, 512x512)
11 |   +-- robots.txt
12 |
13 |-- src/
14 |   |-- main.ts                 # App entry point
15 |   |-- App.vue                 # Root component
16 |   |-- router/
17 |   |   +-- index.ts           # Vue Router config
18 |   |
19 |   |-- pages/
20 |   |   |-- DashboardPage.vue   # Main dashboard

```

```

21 | | | -- MappingPage.vue           # SLAM visualization
22 | | | -- ControlPage.vue          # Manual control
23 | | | -- SensorsPage.vue         # Sensor data
24 | | | -- SettingsPage.vue        # Configuration
25 | | +-- AnalyticsPage.vue        # Mission stats
26 |
27 | | -- components/
28 | | | -- TelemetryCard.vue
29 | | | -- ImuVisualization.vue
30 | | | -- BatteryIndicator.vue
31 | | | -- MapCanvas.vue
32 | | | -- VirtualJoystick.vue
33 | | | -- ConnectionStatus.vue
34 | | +-- EventLog.vue
35 |
36 | | -- composables/
37 | | | -- useRobotConnection.ts
38 | | | -- useTelemetry.ts
39 | | | -- useMotorControl.ts
40 | | | -- useNavigation.ts
41 | | +-- useOfflineStorage.ts
42 |
43 | | -- stores/
44 | | | -- robotStore.ts           # Robot state
45 | | | -- telemetryStore.ts       # Telemetry data
46 | | | -- settingsStore.ts        # User settings
47 | | +-- navigationStore.ts       # Navigation state
48 |
49 | | -- graphql/
50 | | | -- client.ts               # Apollo Client setup
51 | | | -- queries.ts              # GraphQL queries
52 | | | -- mutations.ts            # GraphQL mutations
53 | | +-- subscriptions.ts         # GraphQL subscriptions
54 |
55 | | -- grpc/
56 | | | -- client.ts               # gRPC-Web client
57 | | +-- motor_control_pb.ts      # Generated protobuf
58 |
59 | | -- services/
60 | | | -- offlineStorage.ts        # IndexedDB wrapper
61 | | +-- serviceWorker.ts         # SW registration
62 |
63 | | -- types/
64 | | | -- telemetry.ts             # Type definitions
65 | | | -- navigation.ts
66 | | +-- settings.ts
67 |
68 | +-- assets/
69 |   +-- styles/
70 |     +-- main.css               # Global styles
71 |
72 | +-- workbox-config.js           # Service worker config

```

43 Implementation Tasks

43.1 Phase 1: Project Setup

43.1.1 Task 1.1: Initialize Vite Project

```
1 npm create vite@latest star-ui -- --template vue-ts
2 cd star-ui
3 npm install
4
5 # Install dependencies
6 npm install vue-router@4 pinia @apollo/client graphql
7 npm install @improbable-eng/grpc-web google-protobuf
8 npm install workbox-window idb
9 npm install -D tailwindcss postcss autoprefixer
10 npm install -D vite-plugin-pwa @types/google-protobuf
```

43.1.2 Task 1.2: Vite Configuration

Create vite.config.ts:

```
1 import { defineConfig } from 'vite'
2 import vue from '@vitejs/plugin-vue'
3 import { VitePWA } from 'vite-plugin-pwa'
4
5 export default defineConfig({
6   plugins: [
7     vue(),
8     VitePWA({
9       registerType: 'autoUpdate',
10      includeAssets: ['favicon.ico', 'robots.txt', 'icons/*.png'],
11      manifest: {
12        name: 'STAR Robot Controller',
13        short_name: 'STAR',
14        description: 'Control and monitor the STAR robot',
15        theme_color: '#1e40af',
16        background_color: 'ffffff',
17        display: 'standalone',
18        icons: [
19          {
20            src: '/icons/icon-192x192.png',
21            sizes: '192x192',
22            type: 'image/png'
23          },
24          {
25            src: '/icons/icon-512x512.png',
26            sizes: '512x512',
27            type: 'image/png'
28          }
29        ]
30      },
31      workbox: {
32        globPatterns: ['**/*.{js,css,html,ico,png,svg}'],
```



```

33     runtimeCaching: [
34       {
35         urlPattern: /^https:\/\/star-robot\.local\/graphql/,
36         handler: 'NetworkFirst',
37         options: {
38           cacheName: 'graphql-cache',
39           networkTimeoutSeconds: 3,
40           cacheableResponse: {
41             statuses: [0, 200]
42           }
43         }
44       }
45     ]
46   }
47 })
48 ],
49 server: {
50   proxy: {
51     '/graphql': 'http://star-robot.local:8080',
52     '/graphql-ws': {
53       target: 'ws://star-robot.local:8080',
54       ws: true
55     }
56   }
57 }
58 })

```

Listing 43: vite.config.ts

43.2 Phase 2: GraphQL Integration

43.2.1 Task 2.1: Apollo Client Setup

Create src/graphql/client.ts:

```

1  import {
2    ApolloClient,
3    InMemoryCache,
4    HttpLink,
5    split
6  } from '@apollo/client/core'
7  import { GraphQLWsLink } from '@apollo/client/link/subscriptions'
8  import { createClient } from 'graphql-ws'
9  import { getMainDefinition } from '@apollo/client/utilities'
10
11  const httpLink = new HttpLink({
12    uri: 'http://star-robot.local:8080/graphql'
13  })
14
15  const wsLink = new GraphQLWsLink(
16    createClient({
17      url: 'ws://star-robot.local:8080/graphql-ws'
18    })
19  )

```

```
20
21 const splitLink = split(
22   ({ query }) => {
23     const definition = getMainDefinition(query)
24     return (
25       definition.kind === 'OperationDefinition' &&
26       definition.operation === 'subscription'
27     )
28   },
29   wsLink,
30   httpLink
31 )
32
33 export const apolloClient = new ApolloClient({
34   link: splitLink,
35   cache: new InMemoryCache(),
36   defaultOptions: {
37     watchQuery: {
38       fetchPolicy: 'cache-and-network'
39     }
40   }
41 })
```

Listing 44: client.ts

43.2.2 Task 2.2: GraphQL Operations

Create src/graphql/queries.ts:

```
1 import { gql } from '@apollo/client/core'
2
3 export const GET_TELEMETRY = gql`
4   query GetTelemetry {
5     telemetry {
6       battery
7       temperature
8       cpuUsage
9       wifiSignal
10      motorLoad
11      timestamp
12      imu {
13        pitch
14        roll
15        yaw
16        velocity
17      }
18    }
19  }
20 `
21
22 export const GET_SYSTEM_STATUS = gql`
23   query GetSystemStatus {
24     systemStatus {
25       connectionStatus
26     }
27   }
28 `
```

```

26     mode
27     esp32Connected
28     lidarConnected
29     rosConnected
30   }
31 }
32 ,
33
34 export const GET_NAVIGATION_STATE = gql`
35   query GetNavigationState {
36     navigationState {
37       currentPosition { x y heading }
38       waypoints { id x y label }
39       isNavigating
40     }
41   }
42 ,

```

Listing 45: queries.ts

Create src/graphql/subscriptions.ts:

```

1 import { gql } from '@apollo/client/core'
2
3 export const TELEMETRY_SUBSCRIPTION = gql`
4   subscription TelemetryStream {
5     telemetryStream {
6       battery
7       temperature
8       cpuUsage
9       timestamp
10      imu {
11        pitch
12        roll
13        yaw
14        velocity
15      }
16    }
17  }
18 ,
19
20 export const LIDAR_SUBSCRIPTION = gql`
21   subscription LidarScan {
22     lidarScan {
23       points { angle distance intensity }
24       timestamp
25     }
26   }
27 ,
28
29 export const EVENT_LOG_SUBSCRIPTION = gql`
30   subscription EventLog {
31     eventLog {
32       timestamp
33       message

```

```

34     type
35   }
36 }
37 ,

```

Listing 46: subscriptions.ts

43.3 Phase 3: gRPC-Web Integration

43.3.1 Task 3.1: Generate Protobuf Types

```

1  # Install protoc and grpc-web plugin
2  npm install -g grpc-tools grpc_tools_node_protoc_ts
3
4  # Generate TypeScript from proto
5  protoc -I=../star-proto \
6    --js_out=import_style=commonjs:./src/grpc \
7    --grpc-web_out=import_style=typescript,mode=grpcweb:./src/grpc \
8    ../star-proto/star/motor_control.proto

```

43.3.2 Task 3.2: gRPC Client

Create src/grpc/client.ts:

```

1  import { MotorControlClient } from './MotorControlServiceClientPb'
2  import { VelocityCommand, EmptyRequest } from './motor_control_pb'
3
4  const GRPC_URL = 'http://star-robot.local:8090'
5
6  export const motorClient = new MotorControlClient(GRPC_URL)
7
8  export async function setVelocity(left: number, right: number) {
9    const request = new VelocityCommand()
10    request.setLeftVelocity(left)
11    request.setRightVelocity(right)
12    request.setTimestamp(Date.now())
13
14    return new Promise((resolve, reject) => {
15      motorClient.setVelocity(request, {}, (err, response) => {
16        if (err) reject(err)
17        else resolve(response.toObject())
18      })
19    })
20 }
21
22 export async function emergencyStop() {
23   const request = new EmptyRequest()
24   return new Promise((resolve, reject) => {
25     motorClient.emergencyStop(request, {}, (err, response) => {
26       if (err) reject(err)
27       else resolve(response.toObject())
28     })
29   })

```

30 }

Listing 47: grpc/client.ts

43.4 Phase 4: Composables

43.4.1 Task 4.1: Motor Control Composable

Create src/composables/useMotorControl.ts:

```

1 import { ref, computed } from 'vue'
2 import { setVelocity, emergencyStop } from '@/grpc/client'
3
4 export function useMotorControl() {
5   const leftVelocity = ref(0)
6   const rightVelocity = ref(0)
7   const maxSpeed = ref(0.5)
8   const isEmergencyStopped = ref(false)
9   const lastCommandLatency = ref(0)
10
11   const speed = computed(() =>
12     (leftVelocity.value + rightVelocity.value) / 2
13   )
14
15   async function sendVelocity(left: number, right: number) {
16     if (isEmergencyStopped.value) return
17
18     const clampedLeft = Math.max(-maxSpeed.value,
19       Math.min(maxSpeed.value, left))
20     const clampedRight = Math.max(-maxSpeed.value,
21       Math.min(maxSpeed.value, right))
22
23     try {
24       const start = performance.now()
25       const response = await setVelocity(clampedLeft, clampedRight)
26       lastCommandLatency.value = performance.now() - start
27
28       leftVelocity.value = clampedLeft
29       rightVelocity.value = clampedRight
30     } catch (error) {
31       console.error('Failed to send velocity:', error)
32     }
33   }
34
35   async function stop() {
36     await sendVelocity(0, 0)
37   }
38
39   async function triggerEmergencyStop() {
40     try {
41       await emergencyStop()
42       isEmergencyStopped.value = true
43       leftVelocity.value = 0
44       rightVelocity.value = 0

```

```

43     } catch (error) {
44         console.error('Emergency stop failed:', error)
45     }
46 }
47
48 function resetEmergencyStop() {
49     isEmergencyStopped.value = false
50 }
51
52 return {
53     leftVelocity,
54     rightVelocity,
55     maxSpeed,
56     speed,
57     isEmergencyStopped,
58     lastCommandLatency,
59     sendVelocity,
60     stop,
61     triggerEmergencyStop,
62     resetEmergencyStop
63 }
64 }

```

Listing 48: useMotorControl.ts

43.4.2 Task 4.2: Telemetry Composable

Create src/composables/useTelemetry.ts:

```

1 import { ref, onMounted, onUnmounted } from 'vue'
2 import { useSubscription } from '@vue/apollo-composable'
3 import { TELEMETRY_SUBSCRIPTION } from '@/graphql/subscriptions'
4
5 export function useTelemetry() {
6     const telemetry = ref({
7         battery: 0,
8         temperature: 0,
9         cpuUsage: 0,
10        wifiSignal: 0,
11        imu: { pitch: 0, roll: 0, yaw: 0, velocity: 0 }
12    })
13
14    const { result, error } = useSubscription(TELEMETRY_SUBSCRIPTION)
15
16    onMounted(() => {
17        // Subscribe to telemetry updates
18    })
19
20    return {
21        telemetry,
22        error
23    }
24 }

```

Listing 49: useTelemetry.ts

43.5 Phase 5: PWA Features

43.5.1 Task 5.1: Offline Storage

Create `src/services/offlineStorage.ts`:

```
1 import { openDB, DBSchema } from 'idb'
2
3 interface StarDB extends DBSchema {
4   telemetryHistory: {
5     key: number
6     value: {
7       id: number
8       timestamp: string
9       battery: number
10      temperature: number
11      cpuUsage: number
12    }
13    indexes: { 'by-timestamp': string }
14  }
15  settings: {
16    key: string
17    value: any
18  }
19 }
20
21 const DB_NAME = 'star-robot-db'
22 const DB_VERSION = 1
23
24 export async function initDB() {
25   return openDB<StarDB>(DB_NAME, DB_VERSION, {
26     upgrade(db) {
27       const telemetryStore = db.createObjectStore('telemetryHistory', {
28         keyPath: 'id',
29         autoIncrement: true
30       })
31       telemetryStore.createIndex('by-timestamp', 'timestamp')
32
33       db.createObjectStore('settings', { keyPath: 'key' })
34     }
35   })
36 }
37
38 export async function saveTelemetry(data: any) {
39   const db = await initDB()
40   await db.add('telemetryHistory', {
41     ...data,
42     timestamp: new Date().toISOString()
43   })
44 }
45
46 export async function getTelemetryHistory(limit = 100) {
47   const db = await initDB()
48   const all = await db.getAllFromIndex('telemetryHistory', 'by-timestamp')
49   return all.slice(-limit)
```

```
50 }  
51  
52 export async function saveSetting(key: string, value: any) {  
53   const db = await initDB()  
54   await db.put('settings', { key, value })  
55 }  
56  
57 export async function getSetting(key: string) {  
58   const db = await initDB()  
59   const result = await db.get('settings', key)  
60   return result?.value  
61 }
```

Listing 50: offlineStorage.ts

44 UI Pages

44.1 Dashboard Page

Features:

- Real-time battery, temperature, WiFi signal indicators
- IMU 3D visualization (pitch, roll, yaw)
- Connection status badges
- Quick action buttons (E-Stop, Mode switch)
- Event log stream

44.2 Mapping Page

Features:

- Live occupancy grid from SLAM
- Robot position and heading indicator
- LiDAR scan overlay
- Waypoint markers (clickable to add)
- Path history visualization
- Map export (PNG, PDF, GeoJSON)

44.3 Control Page

Features:

- Virtual joystick (touch/mouse)
- WASD keyboard controls

- Speed presets (0.1, 0.3, 0.5 m/s)
- Emergency stop button (always visible)
- Command latency display
- Mode selector (Manual, Autonomous, Mapping)

44.4 Sensors Page

Features:

- LiDAR polar plot
- IMU orientation graphs
- Ultrasonic distance readings
- Encoder velocities
- Raw data table

45 Build and Deploy

45.1 Development

```
1 cd star-ui
2
3 # Install dependencies
4 npm install
5
6 # Run dev server with hot reload
7 npm run dev
8
9 # Access at http://localhost:5173
```

45.2 Production Build

```
1 # Build for production
2 npm run build
3
4 # Output in dist/ directory
5
6 # Preview production build
7 npm run preview
```

45.3 Deployment Options

1. **Static hosting on Pi5:** Copy `dist/` to nginx/Apache
2. **CDN:** Deploy to Vercel, Netlify, or Cloudflare Pages
3. **Embedded:** Include in `star-gateway` as static resources

```
1 # Deploy to Pi5 nginx
2 scp -r dist/* root@star-robot.local:/var/www/html/
3
4 # Or serve from star-gateway
5 cp -r dist/* ../star-gateway/src/main/resources/static/
```

46 Testing

46.1 Unit Tests

```
1 npm install -D vitest @vue/test-utils happy-dom
2 npm run test
```

46.2 E2E Tests

```
1 npm install -D cypress
2 npx cypress open
```

46.3 PWA Audit

Use Chrome DevTools Lighthouse to audit:

- PWA installability
- Offline functionality
- Performance
- Accessibility

47 References

- Vue 3 Documentation: <https://vuejs.org/>
- Vite PWA Plugin: <https://vite-pwa-org.netlify.app/>
- Apollo Client Vue: <https://v4.apollo.vuejs.org/>
- gRPC-Web: <https://github.com/grpc/grpc-web>
- Workbox: <https://developers.google.com/web/tools/workbox>
- STAR Software Architecture Primer: Section 11
- STAR FTP Section 17: User Interface

48 Project Overview

48.1 Purpose

The `star-gateway` is the central backend service running on the Raspberry Pi 5. It provides:

- **GraphQL API** for telemetry, settings, and navigation queries
- **gRPC service** for low-latency motor commands
- **Hardware adapters** for ESP32 communication (SPI + TCP)
- **ROS 2 bridge** for SLAM and navigation integration
- **Prometheus metrics** for monitoring

48.2 Technology Stack

Component	Technology	Version
Language	Kotlin	1.9.22
Framework	Spring Boot	3.2.x
GraphQL	Spring GraphQL	1.2.x
gRPC	grpc-spring-boot-starter	3.1.x
Hardware Access	Pi4J	2.6.x
Build Tool	Gradle (Kotlin DSL)	8.x
JVM	OpenJDK	17

Table 32: Technology Stack

48.3 Architecture Overview

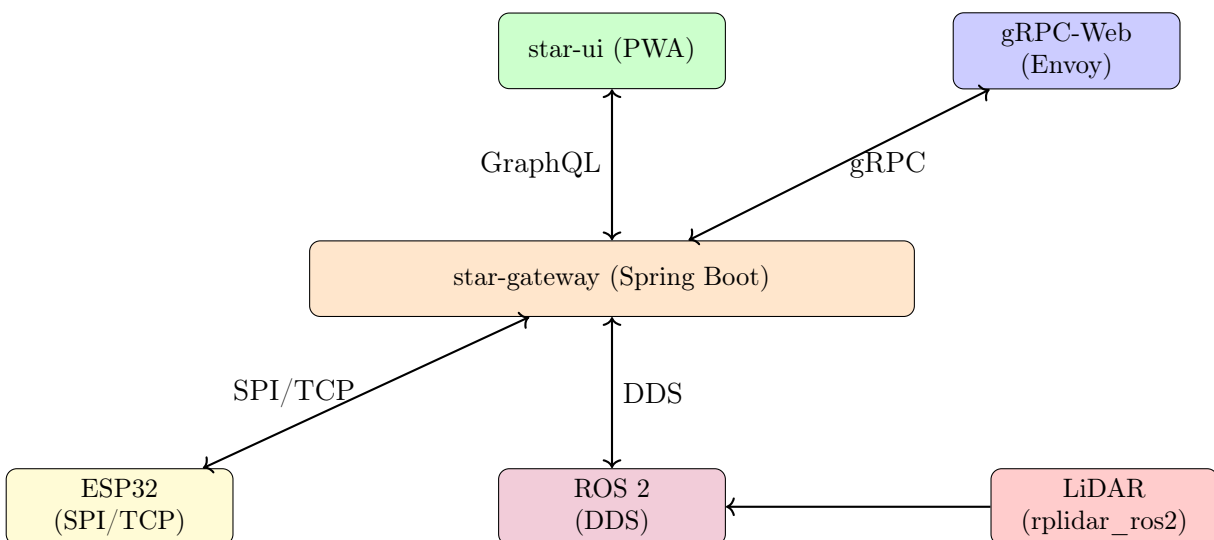


Figure 24: Gateway Architecture

49 Project Structure

```

1 star-gateway/
2 |-- build.gradle.kts           # Build configuration
3 |-- settings.gradle.kts       # Project settings
4 |-- src/
5 |   +-- main/
6 |       |-- kotlin/com/star/gateway/
7 |           |-- StarGatewayApplication.kt
8 |           |-- config/
9 |               |-- GraphQLConfig.kt
10 |               |-- GrpcConfig.kt
11 |               +-- CorsConfig.kt
12 |               |-- graphql/
13 |                   |-- TelemetryController.kt
14 |                   |-- SettingsController.kt
15 |                   +-- NavigationController.kt
16 |                   |-- grpc/
17 |                       +-- MotorControlGrpcService.kt
18 |                       |-- service/
19 |                           |-- MotorControlService.kt
20 |                           |-- TelemetryService.kt
21 |                           |-- NavigationService.kt
22 |                           +-- SensorFusionService.kt
23 |                           |-- hardware/
24 |                               |-- Esp32SpiAdapter.kt
25 |                               |-- Esp32TcpAdapter.kt
26 |                               |-- RplidarAdapter.kt
27 |                               +-- Ros2Bridge.kt
28 |                               +-- model/
29 |                                   |-- TelemetryData.kt
30 |                                   |-- MotorCommand.kt
31 |                                   +-- NavigationState.kt
32 |                                   |-- proto/
33 |                                       +-- motor_control.proto
34 |                                       +-- resources/
35 |                                           |-- application.yml
36 |                                           +-- graphql/
37 |                                               +-- schema.graphqls
38 +-- deployment/
39     |-- star-gateway.service     # systemd unit
40     +-- envoy.yaml               # gRPC-Web proxy config

```

50 Implementation Tasks

50.1 Phase 1: Project Setup

50.1.1 Task 1.1: Gradle Build Configuration

Create build.gradle.kts:

```

1 plugins {

```

```

2    kotlin("jvm") version "1.9.22"
3    kotlin("plugin.spring") version "1.9.22"
4    id("org.springframework.boot") version "3.2.4"
5    id("io.spring.dependency-management") version "1.1.4"
6    id("com.google.protobuf") version "0.9.4"
7  }
8
9  java {
10     sourceCompatibility = JavaVersion.VERSION_17
11  }
12
13  repositories {
14     mavenCentral()
15  }
16
17  dependencies {
18     // Spring Boot Core
19     implementation("org.springframework.boot:spring-boot-starter-webflux")
20     implementation("org.springframework.boot:spring-boot-starter-actuator")
21
22     // GraphQL
23     implementation("org.springframework.boot:spring-boot-starter-graphql")
24
25     // gRPC
26     implementation("net.devh:grpc-spring-boot-starter:3.1.0.RELEASE")
27     implementation("io.grpc:grpc-kotlin-stub:1.4.1")
28     implementation("io.grpc:grpc-protobuf:1.60.1")
29     implementation("com.google.protobuf:protobuf-kotlin:3.25.2")
30
31     // Kotlin Coroutines
32     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.8.0")
33     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:1.8.0")
34
35     // Pi4J for Hardware Access (RPi5 support)
36     implementation("com.pi4j:pi4j-core:2.6.1")
37     implementation("com.pi4j:pi4j-plugin-raspberrypi:2.6.1")
38     implementation("com.pi4j:pi4j-plugin-gpiod:2.6.1")
39     implementation("com.pi4j:pi4j-plugin-linuxfs:2.6.1")
40
41     // Metrics
42     implementation("io.micrometer:micrometer-registry-prometheus")
43
44     // JSON
45     implementation("com.fasterxml.jackson.module:jackson-module-kotlin")
46
47     // Kotlin
48     implementation("org.jetbrains.kotlin:kotlin-reflect")
49
50     // Testing
51     testImplementation("org.springframework.boot:spring-boot-starter-test")
52  }
53
54  protobuf {
55     protoc {

```

```
56     artifact = "com.google.protobuf:protoc:3.25.2"
57   }
58   plugins {
59     id("grpc") {
60       artifact = "io.grpc:protoc-gen-grpc-java:1.60.1"
61     }
62     id("grpckt") {
63       artifact = "io.grpc:protoc-gen-grpc-kotlin:1.4.1:jdk8@jar"
64     }
65   }
66   generateProtoTasks {
67     all().forEach {
68       it.plugins {
69         id("grpc")
70         id("grpckt")
71       }
72       it.builtins {
73         id("kotlin")
74       }
75     }
76   }
77 }
```

Listing 51: build.gradle.kts

50.1.2 Task 1.2: Application Configuration

Create src/main/resources/application.yml:

```
1 spring:
2   application:
3     name: star-gateway
4   graphql:
5     graphiql:
6       enabled: true
7       path: /graphiql
8     websocket:
9       path: /graphql-ws
10    schema:
11      locations: classpath:graphql/
12
13 server:
14   port: 8080
15
16 grpc:
17   server:
18     port: 9090
19
20 star:
21   esp32:
22     spi:
23       enabled: true
24       bus: 0
25       chip-select: 0
```

```

26     speed-hz: 10000000
27     tcp:
28       enabled: true
29       host: 192.168.4.2
30       port: 5000
31       timeout-ms: 100
32     lidar:
33       enabled: true
34       topic: /scan
35
36   management:
37     endpoints:
38       web:
39         exposure:
40           include: health,prometheus,info,metrics
41     endpoint:
42       health:
43         show-details: always
44
45   logging:
46     level:
47       com.star.gateway: DEBUG

```

Listing 52: application.yml

50.2 Phase 2: GraphQL Implementation

50.2.1 Task 2.1: GraphQL Schema

Create src/main/resources/graphql/schema.graphqls:

```

1 type Query {
2   telemetry: Telemetry!
3   systemStatus: SystemStatus!
4   settings: Settings!
5   navigationState: NavigationState!
6   telemetryHistory(limit: Int = 100): [TelemetrySnapshot!]!
7 }
8
9 type Mutation {
10   setMode(mode: RobotMode!): Boolean!
11   emergencyStop: Boolean!
12   setSettings(input: SettingsInput!): Settings!
13   addWaypoint(x: Float!, y: Float!): Waypoint!
14   clearWaypoints: Boolean!
15   startNavigation: Boolean!
16   stopNavigation: Boolean!
17 }
18
19 type Subscription {
20   telemetryStream: Telemetry!
21   lidarScan: LidarScan!
22   eventLog: Event!
23 }

```

```
24
25 type Telemetry {
26     imu: ImuData!
27     gps: GpsData!
28     battery: Float!
29     wifiSignal: Int!
30     cpuUsage: Float!
31     temperature: Float!
32     motorLoad: Float!
33     timestamp: String!
34 }
35
36 type ImuData {
37     pitch: Float!
38     roll: Float!
39     yaw: Float!
40     velocity: Float!
41     accelerationX: Float!
42     accelerationY: Float!
43     accelerationZ: Float!
44 }
45
46 type SystemStatus {
47     connectionStatus: ConnectionStatus!
48     mode: RobotMode!
49     esp32Connected: Boolean!
50     lidarConnected: Boolean!
51     rosConnected: Boolean!
52 }
53
54 type NavigationState {
55     currentPosition: Position!
56     path: [Position!]!
57     waypoints: [Waypoint!]!
58     isNavigating: Boolean!
59 }
60
61 enum RobotMode {
62     IDLE
63     MANUAL
64     AUTONOMOUS
65     MAPPING
66     EMERGENCY_STOP
67 }
68
69 enum ConnectionStatus {
70     CONNECTED
71     DISCONNECTED
72     CONNECTING
73     ERROR
74 }
75
76 input SettingsInput {
77     maxSpeed: Float
```



```

78     safetyLockEnabled: Boolean
79     autonomousEnabled: Boolean
80 }

```

Listing 53: schema.graphqls

50.2.2 Task 2.2: GraphQL Controller

Create src/main/kotlin/com/star/gateway/graphql/TelemetryController.kt:

```

1  package com.star.gateway.graphql
2
3  import com.star.gateway.model.*
4  import com.star.gateway.service.TelemetryService
5  import com.star.gateway.service.MotorControlService
6  import kotlinx.coroutines.flow.Flow
7  import org.springframework.graphql.data.method.annotation.*
8  import org.springframework.stereotype.Controller
9  import reactor.core.publisher.Flux
10
11 @Controller
12 class TelemetryController(
13     private val telemetryService: TelemetryService,
14     private val motorControlService: MotorControlService
15 ) {
16     @QueryMapping
17     suspend fun telemetry(): Telemetry {
18         return telemetryService.getCurrentTelemetry()
19     }
20
21     @QueryMapping
22     suspend fun systemStatus(): SystemStatus {
23         return telemetryService.getSystemStatus()
24     }
25
26     @MutationMapping
27     suspend fun emergencyStop(): Boolean {
28         val result = motorControlService.emergencyStop()
29         return result.success
30     }
31
32     @MutationMapping
33     suspend fun setMode(@Argument mode: RobotMode): Boolean {
34         return telemetryService.setMode(mode)
35     }
36
37     @SubscriptionMapping
38     fun telemetryStream(): Flux<Telemetry> {
39         return telemetryService.telemetryFlow().asFlux()
40     }
41 }

```

Listing 54: TelemetryController.kt

50.3 Phase 3: gRPC Implementation

50.3.1 Task 3.1: Protobuf Definition

Create src/main/proto/motor_control.proto:

```

1 syntax = "proto3";
2
3 package star.motor;
4
5 option java_package = "com.star.gateway.proto";
6 option java_multiple_files = true;
7
8 service MotorControl {
9     rpc SetVelocity(VelocityCommand) returns (CommandResponse);
10    rpc EmergencyStop(EmptyRequest) returns (CommandResponse);
11    rpc StreamEncoderData(EmptyRequest) returns (stream EncoderData);
12 }
13
14 message VelocityCommand {
15     float left_velocity = 1;
16     float right_velocity = 2;
17     uint64 timestamp = 3;
18 }
19
20 message CommandResponse {
21     bool success = 1;
22     string message = 2;
23     uint64 latency_us = 3;
24 }
25
26 message EncoderData {
27     int32 motor_id = 1;
28     int64 ticks = 2;
29     float velocity = 3;
30     uint64 timestamp = 4;
31 }
32
33 message EmptyRequest {}

```

Listing 55: motor_control.proto

50.3.2 Task 3.2: gRPC Service

Create src/main/kotlin/com/star/gateway/grpc/MotorControlGrpcService.kt:

```

1 package com.star.gateway.grpc
2
3 import com.star.gateway.proto.*
4 import com.star.gateway.service.MotorControlService
5 import kotlinx.coroutines.flow.Flow
6 import kotlinx.coroutines.flow.flow
7 import net.devh.boot.grpc.server.service.GrpcService
8 import org.slf4j.LoggerFactory
9

```

```

10 @GrpcService
11 class MotorControlGrpcService(
12     private val motorControlService: MotorControlService
13 ) : MotorControlGrpcKt.MotorControlCoroutineImplBase() {
14
15     private val logger = LoggerFactory.getLogger(javaClass)
16
17     override suspend fun setVelocity(request: VelocityCommand):
18         CommandResponse {
19         val startTime = System.nanoTime()
20
21         logger.debug("gRPC SetVelocity: left=${request.leftVelocity},
22             right=${request.rightVelocity}")
23
24         val result = motorControlService.setVelocity(
25             left = request.leftVelocity,
26             right = request.rightVelocity
27         )
28
29         val latencyUs = (System.nanoTime() - startTime) / 1000
30
31         return CommandResponse.newBuilder()
32             .setSuccess(result.success)
33             .setMessage(result.message)
34             .setLatencyUs(latencyUs)
35             .build()
36     }
37
38     override suspend fun emergencyStop(request: EmptyRequest):
39         CommandResponse {
40         logger.warn("gRPC EmergencyStop received")
41         val result = motorControlService.emergencyStop()
42
43         return CommandResponse.newBuilder()
44             .setSuccess(result.success)
45             .setMessage(result.message)
46             .build()
47     }
48
49     override fun streamEncoderData(request: EmptyRequest):
50         Flow<EncoderData> = flow {
51         while (true) {
52             val encoders = motorControlService.getEncoderData()
53             encoders.forEach { (motorId, data) ->
54                 emit(
55                     EncoderData.newBuilder()
56                         .setMotorId(motorId)
57                         .setTicks(data.ticks)
58                         .setVelocity(data.velocity)
59                         .setTimestamp(System.currentTimeMillis())
60                         .build()
61                 )
62             }
63             kotlinx.coroutines.delay(10) // 100Hz
64         }
65     }
66 }

```

```

60     }
61 }
62 }

```

Listing 56: MotorControlGrpcService.kt

50.4 Phase 4: Hardware Adapters

50.4.1 Task 4.1: ESP32 SPI Adapter

Create `src/main/kotlin/com/star/gateway/hardware/Esp32SpiAdapter.kt`:

```

1 package com.star.gateway.hardware
2
3 import com.pi4j.Pi4J
4 import com.pi4j.context.Context
5 import com.pi4j.io.spi.Spi
6 import com.pi4j.io.spi.SpiMode
7 import com.star.gateway.model.CommandResult
8 import org.springframework.beans.factory.annotation.Value
9 import org.springframework.stereotype.Component
10 import java.nio.ByteBuffer
11 import java.nio.ByteOrder
12 import javax.annotation.PostConstruct
13 import javax.annotation.PreDestroy
14
15 @Component
16 class Esp32SpiAdapter(
17     @Value("\${star.esp32.spi.enabled:true}")
18     private val enabled: Boolean,
19     @Value("\${star.esp32.spi.bus:0}")
20     private val spiBus: Int,
21     @Value("\${star.esp32.spi.chip-select:0}")
22     private val chipSelect: Int,
23     @Value("\${star.esp32.spi.speed-hz:10000000}")
24     private val speedHz: Int
25 ) {
26     companion object {
27         const val CMD_SET_VELOCITY: Byte = 0x01
28         const val CMD_EMERGENCY_STOP: Byte = 0x02
29         const val CMD_GET_ENCODERS: Byte = 0x03
30         const val RESPONSE_OK: Byte = 0x00
31     }
32
33     private var pi4j: Context? = null
34     private var spi: Spi? = null
35     private var connected = false
36
37     @PostConstruct
38     fun initialize() {
39         if (!enabled) return
40
41         try {
42             pi4j = Pi4J.newAutoContext()

```

```

43         val spiConfig = Spi.newConfigBuilder(pi4j)
44             .id("esp32-spi")
45             .bus(spiBus)
46             .chipSelect(chipSelect)
47             .baud(speedHz)
48             .mode(SpiMode.MODE_0)
49             .build()
50
51         spi = pi4j?.create(spiConfig)
52         connected = testConnection()
53     } catch (e: Exception) {
54         connected = false
55     }
56 }
57
58 @PreDestroy
59 fun shutdown() {
60     spi?.close()
61     pi4j?.shutdown()
62 }
63
64 suspend fun sendVelocityCommand(left: Float, right: Float):
65     CommandResult {
66     if (!enabled || spi == null) {
67         throw IllegalStateException("SPI not initialized")
68     }
69
70     val buffer = ByteBuffer.allocate(9)
71         .order(ByteOrder.LITTLE_ENDIAN)
72         .put(CMD_SET_VELOCITY)
73         .putFloat(left)
74         .putFloat(right)
75
76     val txData = buffer.array()
77     val rxData = ByteArray(2)
78
79     synchronized(this) {
80         spi!!.transfer(txData, rxData)
81     }
82
83     return if (rxData[0] == RESPONSE_OK) {
84         CommandResult.success("Velocity set: left=$left,
85             right=$right")
86     } else {
87         CommandResult.failure("ESP32 error: ${rxData[0]}")
88     }
89 }
90
91 suspend fun emergencyStop(): CommandResult {
92     val txData = byteArrayOf(CMD_EMERGENCY_STOP)
93     val rxData = ByteArray(2)
94
95     synchronized(this) {
96         spi?.transfer(txData, rxData)

```

```

95     }
96
97     return if (rxData[0] == RESPONSE_OK) {
98         CommandResult.success("Emergency stop executed")
99     } else {
100         CommandResult.failure("Emergency stop failed")
101     }
102 }
103
104 private fun testConnection(): Boolean {
105     // Ping command implementation
106     return true
107 }
108
109 fun isConnected(): Boolean = connected && enabled
110 }

```

Listing 57: Esp32SpiAdapter.kt

50.4.2 Task 4.2: ESP32 TCP Adapter

Create `src/main/kotlin/com/star/gateway/hardware/Esp32TcpAdapter.kt` as fallback communication channel using JSON over TCP socket.

50.4.3 Task 4.3: ROS 2 Bridge

Create `src/main/kotlin/com/star/gateway/hardware/Ros2Bridge.kt` to interface with ROS 2 via DDS or rosbbridge.

50.5 Phase 5: Deployment

50.5.1 Task 5.1: systemd Service

Create `deployment/star-gateway.service`:

```

1 [Unit]
2 Description=STAR Robot Gateway
3 After=network-online.target
4 Wants=network-online.target
5
6 [Service]
7 Type=simple
8 User=root
9 WorkingDirectory=/opt/star-gateway
10 ExecStart=/usr/bin/java -Xmx512m -jar star-gateway.jar
11 Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"
12 Restart=always
13 RestartSec=10
14
15 [Install]
16 WantedBy=multi-user.target

```

Listing 58: star-gateway.service

50.5.2 Task 5.2: Envoy Proxy (gRPC-Web)

Create deployment/envoy.yaml for browser gRPC support:

```

1 static_resources:
2   listeners:
3     - name: listener_0
4       address:
5         socket_address: { address: 0.0.0.0, port_value: 8090 }
6       filter_chains:
7         - filters:
8           - name: envoy.filters.network.http_connection_manager
9             typed_config:
10               "@type":
11                 type.googleapis.com/envoy.extensions.filters.network.http_connection_manag
12                 codec_type: auto
13                 stat_prefix: ingress_http
14                 route_config:
15                   name: local_route
16                   virtual_hosts:
17                     - name: local_service
18                       domains: ["*"]
19                       routes:
20                         - match: { prefix: "/" }
21                           route:
22                             cluster: grpc_service
23                             timeout: 0s
24                             max_stream_duration:
25                               grpc_timeout_header_max: 0s
26                           cors:
27                             allow_origin_string_match:
28                               - prefix: "*"
29                             allow_methods: GET, PUT, DELETE, POST, OPTIONS
30                             allow_headers:
31                               keep-alive, user-agent, cache-control, content-type, content-transfer-e
32                             expose_headers: grpc-status, grpc-message
33                           http_filters:
34                             - name: envoy.filters.http.grpc_web
35                             - name: envoy.filters.http.cors
36                             - name: envoy.filters.http.router
37
38 clusters:
39   - name: grpc_service
40     connect_timeout: 0.25s
41     type: logical_dns
42     http2_protocol_options: {}
43     lb_policy: round_robin
44     load_assignment:
45       cluster_name: cluster_0
46       endpoints:
47         - lb_endpoints:
48             - endpoint:
49                 address:
50                   socket_address:
51                     address: localhost

```

```
50 port_value: 9090
```

Listing 59: envoy.yaml

51 Build and Run

51.1 Local Development

```
1 cd star-gateway
2
3 # Build
4 ./gradlew build
5
6 # Run locally (without Pi4J hardware)
7 ./gradlew bootRun --args='--star.esp32.spi.enabled=false'
8
9 # Build JAR for deployment
10 ./gradlew bootJar
11
12 # Output: build/libs/star-gateway-*.jar
```

51.2 Deploy to Pi5

```
1 # Copy JAR to Pi5
2 scp build/libs/star-gateway-*.jar
   root@star-robot.local:/opt/star-gateway/star-gateway.jar
3
4 # SSH and restart service
5 ssh root@star-robot.local
6 systemctl restart star-gateway
7 journalctl -u star-gateway -f
```

51.3 API Endpoints

Endpoint	Protocol	Purpose
http://star-robot.local:8080/graphql	GraphQL	Queries, Mutations
ws://star-robot.local:8080/graphql-ws	WebSocket	Subscriptions
http://star-robot.local:8080/graphiql	HTTP	GraphQL IDE
star-robot.local:9090	gRPC	Motor commands
star-robot.local:8090	gRPC-Web	Browser gRPC
http://star-robot.local:8080/actuator/*	HTTP	Health, Metrics

Table 33: API Endpoints

52 Testing

52.1 GraphQL Testing

Use GraphQL at `http://star-robot.local:8080/graphql`:

```
1  # Get current telemetry
2  query {
3    telemetry {
4      battery
5      temperature
6      imu { pitch roll yaw }
7    }
8  }
9
10 # Emergency stop
11 mutation {
12   emergencyStop
13 }
14
15 # Subscribe to telemetry stream
16 subscription {
17   telemetryStream {
18     battery
19     timestamp
20   }
21 }
```

Listing 60: Sample GraphQL Queries

52.2 gRPC Testing

Use `grpcurl`:

```
1  # List services
2  grpcurl -plaintext localhost:9090 list
3
4  # Set velocity
5  grpcurl -plaintext -d '{"leftVelocity": 0.5, "rightVelocity": 0.5}' \
6    localhost:9090 star.motor.MotorControl/SetVelocity
7
8  # Emergency stop
9  grpcurl -plaintext -d '{} ' \
10   localhost:9090 star.motor.MotorControl/EmergencyStop
```

53 References

- Spring GraphQL: <https://docs.spring.io/spring-graphql/reference/>
- grpc-spring-boot-starter: <https://yidongnan.github.io/grpc-spring-boot-starter/>
- Pi4J v2: <https://pi4j.com/>
- Envoy gRPC-Web: https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/grpc_web_filter

- STAR Software Architecture Primer: Section 10

54 Project Overview

54.1 Purpose

The `star-proto` repository contains shared Protocol Buffer (protobuf) definitions used across the STAR robot system:

- **star-gateway**: Kotlin/Java gRPC server
- **star-ui**: TypeScript gRPC-Web client
- **star-esp32-firmware**: (Future) ESP32 binary protocol

54.2 Why Protocol Buffers?

Feature	Protobuf	JSON
Message Size	17 bytes (example)	65 bytes
Serialization	1.2 μ s	4.5 μ s
Type Safety	Compile-time	Runtime
Schema Evolution	Built-in	Manual
Code Generation	Automatic	Manual

Table 34: Protobuf vs JSON Comparison

54.3 Architecture

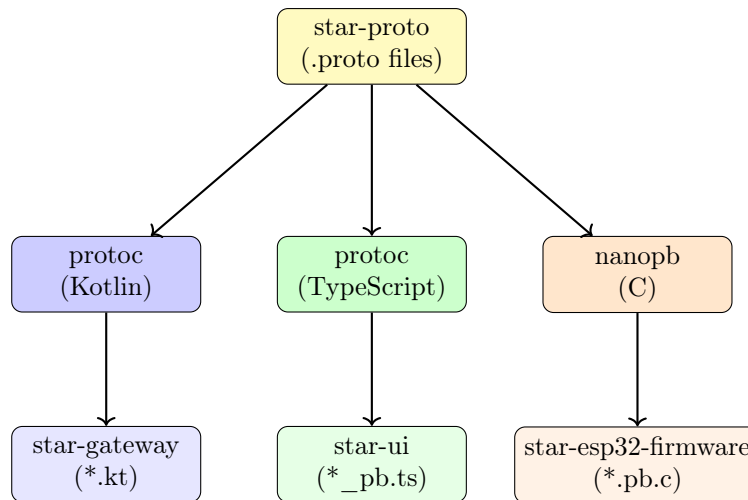


Figure 25: Code Generation Flow

55 Directory Structure

```

1 star-proto/
2 |-- README.md
3 |-- buf.yaml                                # Buf build configuration

```

```

4 |-- buf.gen.yaml                                # Code generation config
5 |
6 |-- star/
7 |   |-- motor_control.proto                    # Motor command service
8 |   |-- telemetry.proto                        # Telemetry data types
9 |   |-- navigation.proto                      # Navigation service
10 |   +-- common.proto                          # Shared types
11 |
12 |-- scripts/
13 |   |-- generate-kotlin.sh                    # Generate for star-gateway
14 |   |-- generate-typescript.sh               # Generate for star-ui
15 |   +-- generate-c.sh                        # Generate for star-esp32-firmware
16 |
17 +-- generated/                                # Generated code (gitignored)
18 |   |-- kotlin/
19 |   |-- typescript/
20 |   +-- c/

```

56 Protocol Definitions

56.1 motor_control.proto

```

1 syntax = "proto3";
2
3 package star.motor;
4
5 option java_package = "com.star.gateway.proto";
6 option java_outer_classname = "MotorControlProto";
7 option java_multiple_files = true;
8
9 import "star/common.proto";
10
11 // Motor control service for low-latency commands
12 service MotorControl {
13     // Set wheel velocities (differential drive)
14     rpc SetVelocity(VelocityCommand) returns (CommandResponse);
15
16     // Immediate emergency stop
17     rpc EmergencyStop(EmptyRequest) returns (CommandResponse);
18
19     // Set individual motor power
20     rpc SetMotorPower(MotorPowerCommand) returns (CommandResponse);
21
22     // Stream encoder data (server streaming)
23     rpc StreamEncoderData(EmptyRequest) returns (stream EncoderData);
24
25     // Bidirectional control stream (for continuous control)
26     rpc ControlStream(stream VelocityCommand) returns (stream
27         EncoderData);
28 }
29 // Velocity command for differential drive

```

```
30 message VelocityCommand {
31     // Left wheel velocity in m/s (-2.0 to 2.0)
32     float left_velocity = 1;
33
34     // Right wheel velocity in m/s (-2.0 to 2.0)
35     float right_velocity = 2;
36
37     // Client timestamp in milliseconds (for latency tracking)
38     uint64 timestamp = 3;
39
40     // Optional: command sequence number
41     uint32 sequence = 4;
42 }
43
44 // Direct motor power control
45 message MotorPowerCommand {
46     // Motor index: 0=front-left, 1=front-right, 2=rear-left, 3=rear-right
47     int32 motor_id = 1;
48
49     // Power level: -1.0 (full reverse) to 1.0 (full forward)
50     float power = 2;
51 }
52
53 // Standard command response
54 message CommandResponse {
55     // True if command was executed successfully
56     bool success = 1;
57
58     // Human-readable status message
59     string message = 2;
60
61     // Round-trip latency in microseconds
62     uint64 latency_us = 3;
63
64     // Server timestamp
65     uint64 server_timestamp = 4;
66 }
67
68 // Real-time encoder feedback
69 message EncoderData {
70     // Motor index (0-3)
71     int32 motor_id = 1;
72
73     // Absolute encoder position (ticks)
74     int64 ticks = 2;
75
76     // Current velocity in m/s
77     float velocity = 3;
78
79     // Timestamp in milliseconds
80     uint64 timestamp = 4;
81 }
```

Listing 61: star/motor_control.proto

56.2 telemetry.proto

```

1 syntax = "proto3";
2
3 package star.telemetry;
4
5 option java_package = "com.star.gateway.proto";
6 option java_multiple_files = true;
7
8 import "star/common.proto";
9
10 // Telemetry streaming service
11 service TelemetryService {
12     // Get current telemetry snapshot
13     rpc GetTelemetry(EmptyRequest) returns (TelemetryData);
14
15     // Stream telemetry at specified rate
16     rpc StreamTelemetry(StreamRequest) returns (stream TelemetryData);
17
18     // Get system status
19     rpc GetSystemStatus(EmptyRequest) returns (SystemStatus);
20 }
21
22 // Stream configuration
23 message StreamRequest {
24     // Update rate in Hz (1-100)
25     int32 rate_hz = 1;
26
27     // Fields to include (empty = all)
28     repeated string fields = 2;
29 }
30
31 // Complete telemetry snapshot
32 message TelemetryData {
33     // IMU data
34     ImuData imu = 1;
35
36     // GPS data (if available)
37     GpsData gps = 2;
38
39     // Battery percentage (0-100)
40     float battery = 3;
41
42     // WiFi signal strength in dBm
43     int32 wifi_signal = 4;
44
45     // CPU usage percentage (0-100)
46     float cpu_usage = 5;
47
48     // Temperature in Celsius
49     float temperature = 6;
50
51     // Motor load percentage (0-100)
52     float motor_load = 7;

```

```
53
54     // Timestamp (ISO 8601)
55     string timestamp = 8;
56 }
57
58 // IMU sensor data
59 message ImuData {
60     // Orientation (degrees)
61     float pitch = 1;
62     float roll = 2;
63     float yaw = 3;
64
65     // Linear velocity (m/s)
66     float velocity = 4;
67
68     // Acceleration (m/s^2)
69     float acceleration_x = 5;
70     float acceleration_y = 6;
71     float acceleration_z = 7;
72
73     // Angular velocity (rad/s)
74     float gyro_x = 8;
75     float gyro_y = 9;
76     float gyro_z = 10;
77 }
78
79 // GPS data
80 message GpsData {
81     double latitude = 1;
82     double longitude = 2;
83     float altitude = 3;
84     float accuracy = 4;
85     int32 satellites = 5;
86 }
87
88 // System status
89 message SystemStatus {
90     ConnectionStatus connection_status = 1;
91     RobotMode mode = 2;
92     bool esp32_connected = 3;
93     bool lidar_connected = 4;
94     bool ros_connected = 5;
95     string firmware_version = 6;
96     uint64 uptime_seconds = 7;
97 }
98
99 // Connection state
100 enum ConnectionStatus {
101     CONNECTION_UNKNOWN = 0;
102     CONNECTION_CONNECTED = 1;
103     CONNECTION_DISCONNECTED = 2;
104     CONNECTION_CONNECTING = 3;
105     CONNECTION_ERROR = 4;
106 }
```

```

107
108 // Robot operating mode
109 enum RobotMode {
110     MODE_UNKNOWN = 0;
111     MODE_IDLE = 1;
112     MODE_MANUAL = 2;
113     MODE_AUTONOMOUS = 3;
114     MODE_MAPPING = 4;
115     MODE_EMERGENCY_STOP = 5;
116 }

```

Listing 62: star/telemetry.proto

56.3 navigation.proto

```

1 syntax = "proto3";
2
3 package star.navigation;
4
5 option java_package = "com.star.gateway.proto";
6 option java_multiple_files = true;
7
8 import "star/common.proto";
9
10 // Navigation service
11 service NavigationService {
12     // Get current navigation state
13     rpc GetState(EmptyRequest) returns (NavigationState);
14
15     // Add waypoint to queue
16     rpc AddWaypoint(Waypoint) returns (WaypointResponse);
17
18     // Clear all waypoints
19     rpc ClearWaypoints(EmptyRequest) returns (CommandResponse);
20
21     // Start autonomous navigation
22     rpc StartNavigation(EmptyRequest) returns (CommandResponse);
23
24     // Stop navigation
25     rpc StopNavigation(EmptyRequest) returns (CommandResponse);
26
27     // Stream navigation updates
28     rpc StreamNavigation(EmptyRequest) returns (stream NavigationState);
29
30     // Stream LiDAR scans
31     rpc StreamLidar(StreamRequest) returns (stream LidarScan);
32 }
33
34 // Current navigation state
35 message NavigationState {
36     // Current robot position
37     Position current_position = 1;
38

```



```
39 // Planned path
40 repeated Position path = 2;
41
42 // Queued waypoints
43 repeated Waypoint waypoints = 3;
44
45 // Navigation active
46 bool is_navigating = 4;
47
48 // Progress percentage
49 float progress = 5;
50
51 // Estimated time remaining (seconds)
52 float eta_seconds = 6;
53 }
54
55 // 2D position with heading
56 message Position {
57     float x = 1; // meters
58     float y = 2; // meters
59     float heading = 3; // radians
60 }
61
62 // Navigation waypoint
63 message Waypoint {
64     string id = 1;
65     float x = 2;
66     float y = 3;
67     string label = 4;
68     WaypointType type = 5;
69 }
70
71 enum WaypointType {
72     WAYPOINT_NORMAL = 0;
73     WAYPOINT_STOP = 1;
74     WAYPOINT_SLOW = 2;
75 }
76
77 message WaypointResponse {
78     bool success = 1;
79     string waypoint_id = 2;
80     string message = 3;
81 }
82
83 // LiDAR scan data
84 message LidarScan {
85     repeated LidarPoint points = 1;
86     string timestamp = 2;
87     string frame_id = 3;
88 }
89
90 message LidarPoint {
91     float angle = 1; // radians
92     float distance = 2; // meters
```

```

93     float intensity = 3;  // 0-255
94 }
95
96 // Map data
97 message OccupancyGrid {
98     // Grid metadata
99     float resolution = 1;  // meters per cell
100     uint32 width = 2;
101     uint32 height = 3;
102     Position origin = 4;
103
104     // Grid data (-1=unknown, 0=free, 100=occupied)
105     bytes data = 5;
106
107     string timestamp = 6;
108 }

```

Listing 63: star/navigation.proto

56.4 common.proto

```

1  syntax = "proto3";
2
3  package star;
4
5  option java_package = "com.star.gateway.proto";
6  option java_multiple_files = true;
7
8  // Empty request placeholder
9  message EmptyRequest {}
10
11 // Generic command response (shared across services)
12 message CommandResponse {
13     bool success = 1;
14     string message = 2;
15 }
16
17 // Stream request with rate
18 message StreamRequest {
19     int32 rate_hz = 1;
20 }
21
22 // Timestamp wrapper
23 message Timestamp {
24     int64 seconds = 1;
25     int32 nanos = 2;
26 }
27
28 // Vector3 for 3D data
29 message Vector3 {
30     float x = 1;
31     float y = 2;
32     float z = 3;

```

```
33 }
34
35 // Quaternion for rotation
36 message Quaternion {
37     float x = 1;
38     float y = 2;
39     float z = 3;
40     float w = 4;
41 }
```

Listing 64: star/common.proto

57 Code Generation

57.1 Prerequisites

```
1 # Install protoc compiler
2 # macOS
3 brew install protobuf
4
5 # Ubuntu
6 sudo apt install protobuf-compiler
7
8 # Install Buf (recommended)
9 # macOS
10 brew install bufbuild/buf/buf
11
12 # Other platforms
13 npm install -g @bufbuild/buf
```

57.2 Using Buf (Recommended)

Create buf.yaml:

```
1 version: v1
2 name: buf.build/star/proto
3 deps: []
4 breaking:
5   use:
6     - FILE
7 lint:
8   use:
9     - DEFAULT
```

Listing 65: buf.yaml

Create buf.gen.yaml:

```
1 version: v1
2 managed:
3   enabled: true
4   java_package_prefix: com.star.gateway.proto
5 plugins:
```

```

6   # Kotlin/Java for star-gateway
7   - plugin: buf.build/grpc/java
8     out: generated/kotlin
9   - plugin: buf.build/protocolbuffers/java
10    out: generated/kotlin
11
12  # TypeScript for star-ui
13  - plugin: buf.build/community/timostamm-protobuf-ts
14    out: generated/typescript
15
16  # C for star-esp32-firmware (via nanopb)
17  - plugin: buf.build/community/nanopb-nanopb
18    out: generated/c

```

Listing 66: buf.gen.yaml

Generate code:

```

1  # Lint proto files
2  buf lint
3
4  # Generate all targets
5  buf generate
6
7  # Generate specific target
8  buf generate --path star/motor_control.proto

```

57.3 Manual Generation Scripts

scripts/generate-kotlin.sh:

```

1  #!/bin/bash
2  set -e
3
4  PROTO_DIR="star"
5  OUT_DIR="../star-gateway/src/main/kotlin"
6
7  protoc \
8    -I=. \
9    --java_out=$OUT_DIR \
10   --grpc-java_out=$OUT_DIR \
11   --kotlin_out=$OUT_DIR \
12   --grpckt_out=$OUT_DIR \
13   $PROTO_DIR/*.proto
14
15  echo "Kotlin code generated in $OUT_DIR"

```

Listing 67: generate-kotlin.sh

scripts/generate-typescript.sh:

```

1  #!/bin/bash
2  set -e
3
4  PROTO_DIR="star"

```

```

5 OUT_DIR="../../star-ui/src/grpc"
6
7 protoc \
8     -I=. \
9     --js_out=import_style=commonjs:$OUT_DIR \
10    --grpc-web_out=import_style=typescript,mode=grpcweb:$OUT_DIR \
11    $PROTO_DIR/*.proto
12
13 echo "TypeScript code generated in $OUT_DIR"

```

Listing 68: generate-typescript.sh

58 Best Practices

58.1 Schema Evolution

1. **Never remove fields** - mark as **reserved**
2. **Never change field numbers** - they define wire format
3. **Add new fields** with new numbers only
4. **Use optional** for fields that may not be present

```

1 message VelocityCommand {
2     float left_velocity = 1;
3     float right_velocity = 2;
4
5     // Reserved: removed fields
6     reserved 3;
7     reserved "old_field_name";
8
9     // New optional field (added later)
10    optional float acceleration_limit = 4;
11 }

```

Listing 69: Schema Evolution Example

58.2 Naming Conventions

Element	Convention	Example
Package	lowercase.dots	star.motor
Message	PascalCase	VelocityCommand
Field	snake_case	left_velocity
Enum	SCREAMING_SNAKE	MODE_MANUAL
Service	PascalCase	MotorControl
RPC method	PascalCase	SetVelocity

Table 35: Protobuf Naming Conventions

58.3 Documentation

Always document messages and fields:

```

1 // Velocity command for differential drive robot.
2 // Sent from controller to gateway at up to 100Hz.
3 message VelocityCommand {
4     // Left wheel velocity in meters per second.
5     // Valid range: -2.0 to 2.0 m/s.
6     float left_velocity = 1;
7
8     // Right wheel velocity in meters per second.
9     // Valid range: -2.0 to 2.0 m/s.
10    float right_velocity = 2;
11 }

```

59 Integration

59.1 star-gateway Integration

Add to star-gateway/build.gradle.kts:

```

1 protobuf {
2     protoc {
3         artifact = "com.google.protobuf:protoc:3.25.2"
4     }
5     plugins {
6         id("grpc") {
7             artifact = "io.grpc:protoc-gen-grpc-java:1.60.1"
8         }
9         id("grpckt") {
10            artifact = "io.grpc:protoc-gen-grpc-kotlin:1.4.1:jdk8@jar"
11        }
12    }
13 }
14
15 sourceSets {
16     main {
17         proto {
18             srcDir("../star-proto/star")
19         }
20     }
21 }

```

59.2 star-ui Integration

Add npm script to star-ui/package.json:

```

1 {
2     "scripts": {
3         "proto:generate": "cd ../star-proto &&
4             ./scripts/generate-typescript.sh"
5     }
6 }

```

5 }

60 References

- Protocol Buffers Documentation: <https://protobuf.dev/>
- gRPC Documentation: <https://grpc.io/docs/>
- Buf CLI: <https://buf.build/docs/>
- gRPC-Web: <https://github.com/grpc/grpc-web>
- nanopb (C): <https://jpa.kapsi.fi/nanopb/>
- STAR Software Architecture Primer: Section 3 (Protocol Analysis)

61 Project Overview

61.1 Purpose

The `star-rpi5-buildroot` project creates a custom, minimal Linux distribution for the Raspberry Pi 5. It includes ROS 2, SLAM algorithms, navigation stack, and all dependencies needed for the STAR robot.

61.2 Current Implementation Status

Component	Status	Notes
Buildroot Base	Complete	2025.02 LTS, ARM64
Kernel Config	Complete	Linux 6.6 for Pi5
WiFi Dual-Mode	Complete	Station + Access Point
mDNS Discovery	Complete	star-robot.local via Avahi
OpenCV + Python	Complete	ML stack ready
Camera Support	Complete	libcamera, V4L2
systemd Services	Complete	Network, gateway templates
ROS 2 Jazzy	Placeholder	Directory structure only
SLAM Toolbox	TODO	Package not integrated
Nav2 Stack	TODO	Navigation packages needed
star-gateway	TODO	Spring Boot application
LiDAR Drivers	TODO	rplidar_ros2, sick_scan_xd

Table 36: Implementation Status Summary

61.3 Network Architecture

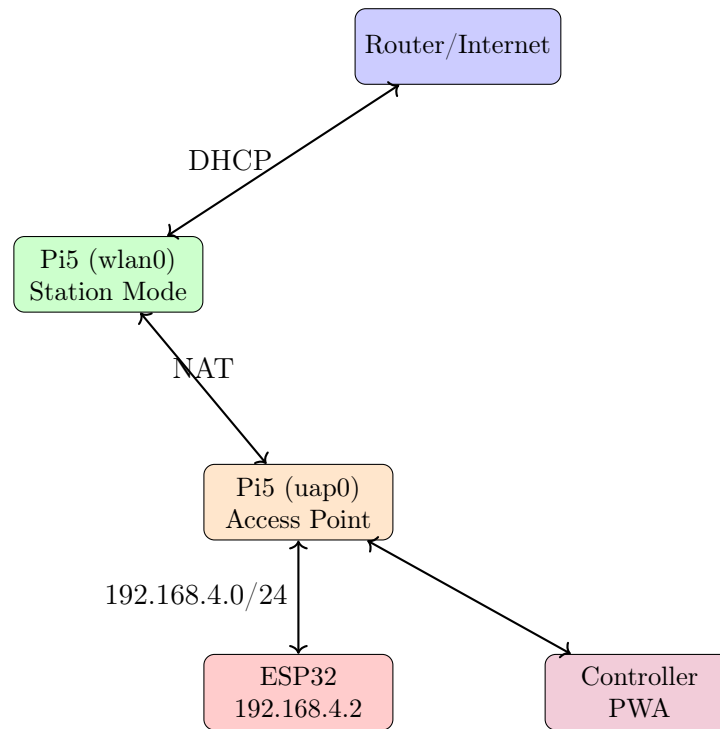


Figure 26: Dual WiFi Network Topology

62 Directory Structure

```

1 star-rpi5-buildroot/
2 |-- Makefile                # Build orchestration
3 |-- README.md               # User documentation
4 |-- Config.in               # External tree config
5 |-- external.desc           # Buildroot external desc
6 |-- external.mk             # Package includes
7 |
8 |-- configs/
9 |   |-- rpi5_64_defconfig    # Main Buildroot config
10 |   |-- config.txt           # Pi5 boot config
11 |   +-- busybox.config       # Minimal busybox
12 |
13 |-- package/
14 |   +-- ros2-jazzy/          # ROS 2 package (placeholder)
15 |       |-- Config.in
16 |       +-- ros2-jazzy.mk
17 |
18 |-- overlays/               # Rootfs customizations
19 |   |-- etc/
20 |       |-- hostapd/         # WiFi AP config
21 |       |-- wpa_supplicant/  # WiFi station config
22 |       |-- dnsmasq.d/       # DHCP for AP
23 |       |-- iptables/        # NAT rules
  
```

```

24 | | | -- systemd/system/      # Service files
25 | | | ++ profile.d/          # Environment scripts
26 | | ++ root/
27 | | ++ setup-network.sh      # Interactive setup
28 |
29 | ++ scripts/
30 | | -- post-build.sh          # Enable services
31 | | -- post-image.sh          # Generate SD image
32 | ++ qemu-test.sh            # QEMU testing

```

63 Implementation Tasks

63.1 Phase 1: ROS 2 Integration

63.1.1 Task 1.1: ROS 2 Jazzy Binary Integration

The current `ros2-jazzy` package is a placeholder. Implement actual binary installation:

Option A: Pre-built ARM64 Binaries

1. Download ROS 2 Jazzy ARM64 binaries from OSRF
2. Create Buildroot package that extracts to `/opt/ros/jazzy`
3. Handle dependencies (Python packages, shared libraries)
4. Test with `ros2 run demo_nodes_cpp talker`

```

1 ROS2_JAZZY_VERSION = jazzy
2 ROS2_JAZZY_SOURCE = ros2-jazzy-$(ROS2_JAZZY_VERSION)-linux-arm64.tar.bz2
3 ROS2_JAZZY_SITE = https://github.com/ros2/ros2/releases/download/...
4
5 define ROS2_JAZZY_INSTALL_TARGET_CMDS
6     mkdir -p $(TARGET_DIR)/opt/ros/jazzy
7     tar -xf $(@D)/$(ROS2_JAZZY_SOURCE) -C $(TARGET_DIR)/opt/ros/jazzy
8 endef
9
10 $(eval $(generic-package))

```

Listing 70: `ros2-jazzy.mk` (Binary Installation)

Option B: Build from Source (More Complex)

1. Add colcon build system to Buildroot
2. Cross-compile ROS 2 base packages
3. Handle CMake cross-compilation settings
4. Much longer build time (2+ hours)

63.1.2 Task 1.2: SLAM Toolbox Package

Create package/slam-toolbox/:

```
1 SLAM_TOOLBOX_VERSION = jazzy
2 SLAM_TOOLBOX_SITE = $(call
   github,SteveMacenski,slam_toolbox,$(SLAM_TOOLBOX_VERSION))
3 SLAM_TOOLBOX_DEPENDENCIES = ros2-jazzy eigen boost
4
5 define SLAM_TOOLBOX_BUILD_CMDS
6     source /opt/ros/jazzy/setup.bash && \
7     cd $(@D) && colcon build --packages-select slam_toolbox
8 endef
9
10 $(eval $(generic-package))
```

Listing 71: slam-toolbox.mk

63.1.3 Task 1.3: Nav2 Navigation Stack

Create package/nav2/:

Required packages:

- nav2_bringup
- nav2_bt_navigator
- nav2_planner
- nav2_controller
- nav2_costmap_2d
- nav2_amcl
- dwb_core (DWA local planner)

63.2 Phase 2: LiDAR Integration

63.2.1 Task 2.1: RPLiDAR Driver

Create package/rplidar-ros2/:

1. Clone rplidar_ros2 from Slamtec
2. Build as ROS 2 package
3. Configure for UART: /dev/ttyUSB0 at 460800 baud
4. Test with `ros2 launch rplidar_ros2 rplidar_c1_launch.py`

63.2.2 Task 2.2: SICK TiM561 Driver

Create `package/sick-scan-xd/`:

1. Clone `sick_scan_xd` from SICK AG
2. Build as ROS 2 package
3. Configure for Ethernet: `192.168.0.1:2112`
4. Test with `ros2 launch sick_scan_xd sick_tim_5xx.launch.py`

63.3 Phase 3: star-gateway Integration

63.3.1 Task 3.1: Java Runtime

Add OpenJDK 17 to Buildroot:

```
1 BR2_PACKAGE_OPENJDK=y
2 BR2_PACKAGE_OPENJDK_VERSION_17=y
```

Listing 72: Add to `rpi5_64_defconfig`

Note: OpenJDK build may fail on WSL. Use native Linux or pre-built binaries.

63.3.2 Task 3.2: Gateway Deployment

Update `overlays/etc/systemd/system/robot-gateway.service`:

```
1 [Unit]
2 Description=STAR Robot Gateway
3 After=network-online.target ros2.service
4 Wants=network-online.target
5
6 [Service]
7 Type=simple
8 User=root
9 WorkingDirectory=/opt/star-gateway
10 ExecStart=/usr/bin/java -Xmx512m -jar star-gateway.jar
11 Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"
12 Environment="ROS_DISTRO=jazzy"
13 Environment="ROS_DOMAIN_ID=0"
14 Restart=always
15 RestartSec=10
16
17 [Install]
18 WantedBy=multi-user.target
```

Listing 73: `robot-gateway.service`

63.3.3 Task 3.3: Gateway Build Script

Create deployment pipeline:

```

1  #!/bin/bash
2  # Build and deploy star-gateway to Pi5
3
4  # Build JAR
5  cd ../star-gateway
6  ./gradlew bootJar
7
8  # Copy to Pi5
9  scp build/libs/star-gateway-*.jar
    root@star-robot.local:/opt/star-gateway/star-gateway.jar
10
11 # Restart service
12 ssh root@star-robot.local "systemctl restart robot-gateway"

```

Listing 74: deploy-gateway.sh

63.4 Phase 4: System Configuration

63.4.1 Task 4.1: ROS 2 Launch Files

Create overlays/opt/ros/star_bringup/:

```

1  from launch import LaunchDescription
2  from launch_ros.actions import Node
3
4  def generate_launch_description():
5      return LaunchDescription([
6          # LiDAR driver
7          Node(
8              package='rplidar_ros2',
9              executable='rplidar_node',
10             parameters=[{'serial_port': '/dev/ttyUSB0',
11                          'frame_id': 'laser_frame'}],
12          ),
13          # SLAM Toolbox
14          Node(
15              package='slam_toolbox',
16              executable='async_slam_toolbox_node',
17              parameters=['slam_params.yaml'],
18          ),
19          # Nav2 stack
20          IncludeLaunchDescription(
21              PythonLaunchDescriptionSource('nav2_bringup/launch/navigation_launch.py'),
22              launch_arguments={'params_file': 'nav2_params.yaml'}.items()
23          ),
24      ])

```

Listing 75: star_robot.launch.py

63.4.2 Task 4.2: SLAM Parameters

Create overlays/opt/ros/config/slam_params.yaml:

```

1 slam_toolbox:
2   ros__parameters:
3     solver_plugin: solver_plugins::CeresSolver
4     ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
5
6     odom_frame: odom
7     map_frame: map
8     base_frame: base_link
9     scan_topic: /scan
10
11     resolution: 0.05           # 5cm per cell
12     max_laser_range: 12.0      # RPLiDAR C1 max
13     minimum_travel_distance: 0.3
14     minimum_travel_heading: 0.3
15
16     do_loop_closing: true
17     loop_search_maximum_distance: 3.0
18
19     # Performance tuning for RPi5
20     transform_publish_period: 0.05 # 20 Hz
21     map_update_interval: 0.5       # 2 Hz

```

Listing 76: slam_params.yaml

63.4.3 Task 4.3: Nav2 Parameters

Create overlays/opt/ros/config/nav2_params.yaml:

```

1 controller_server:
2   ros__parameters:
3     controller_frequency: 20.0
4     FollowPath:
5       plugin: "dwb_core::DWBLocalPlanner"
6       max_vel_x: 0.5
7       min_vel_x: -0.3
8       max_vel_theta: 1.0
9       acc_lim_x: 1.0
10      acc_lim_theta: 2.0
11      decel_lim_x: -1.5
12      vx_samples: 20
13      vtheta_samples: 40
14      sim_time: 1.5
15      critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
16               "GoalAlign", "PathAlign", "PathDist", "GoalDist"]

```

Listing 77: nav2_params.yaml (DWA Controller)

64 Build Instructions

64.1 Prerequisites

```

1 # Ubuntu/Debian dependencies
2 sudo apt-get install build-essential libncurses5-dev git \
3     bc bison flex libssl-dev make unzip wget cpio rsync
4
5 # Disk space: ~20GB free
6 # Build time: 1-2 hours (first build)

```

64.2 Build Commands

```

1 cd star-rpi5-buildroot
2
3 # Full build (download + configure + build)
4 make build
5
6 # Or step-by-step:
7 make download      # Get Buildroot 2025.02
8 make configure     # Apply our config
9 make               # Build everything
10
11 # Output: buildroot-2025.02/output/images/sdcard.img

```

64.3 Flash to SD Card

```

1 # Find SD card device (e.g., /dev/sdb)
2 lsblk
3
4 # Flash image (CAUTION: double-check device!)
5 sudo dd if=buildroot-2025.02/output/images/sdcard.img \
6     of=/dev/sdX bs=4M status=progress
7 sync

```

64.4 First Boot Setup

```

1 # Connect via Ethernet or serial console
2 ssh root@<IP_ADDRESS> # No password
3
4 # Or via mDNS (after network configured)
5 ssh root@star-robot.local
6
7 # Run interactive network setup
8 /root/setup-network.sh
9 # Choose option 8 for full setup

```

65 Testing

65.1 QEMU Testing

Test without hardware using QEMU:

```
1 # Serial console boot
2 make qemu
3
4 # Graphical mode (VNC)
5 make qemu-graphical
6
7 # SSH into QEMU instance
8 make qemu-ssh
```

Note: QEMU uses generic ARM64 machine, not Pi5-specific. Good for testing systemd, networking, userspace.

65.2 Hardware Testing Checklist

1. Boot Pi5, verify serial console output
2. Check WiFi station connects to router
3. Check WiFi AP creates STAR-Robot-AP network
4. Verify ESP32 can connect to AP
5. Test mDNS: `ping star-robot.local`
6. Test SSH access
7. Verify camera: `libcamera-hello`
8. Test ROS 2: `ros2 topic list`
9. Run SLAM with LiDAR
10. Test star-gateway API endpoints

66 Troubleshooting

66.1 Common Issues

Build fails on WSL OpenJDK cross-compilation doesn't work. Use native Linux or remove Java from config.

WiFi AP not starting Check `systemctl status hostapd`. May need to manually create `uap0` interface.

ROS 2 not found The `ros2-jazzy` package is a placeholder. Install ROS 2 manually after first boot.

SSH permission denied Default config allows empty root password. If changed, use the password you set.

66.2 Debugging Commands

```
1 # Check service status
2 systemctl status hostapd dnsmasq-ap wifi-nat wpa_supplicant@wlan0
3
4 # View logs
5 journalctl -u robot-gateway -f
6
7 # Network interfaces
8 ip addr show
9 iwconfig
10
11 # ROS 2 environment
12 source /opt/ros/jazzy/setup.bash
13 ros2 doctor
```

67 References

- Buildroot Manual: <https://buildroot.org/downloads/manual/manual.html>
- ROS 2 Jazzy Installation: <https://docs.ros.org/en/jazzy/Installation.html>
- SLAM Toolbox: https://github.com/SteveMacenski/slam_toolbox
- Nav2 Documentation: <https://docs.nav2.org/>
- RPLiDAR ROS 2: https://github.com/Slamtec/rplidar_ros
- STAR FTP Section 17: Software Implementation
- STAR Software Architecture Primer: Sections 5, 14

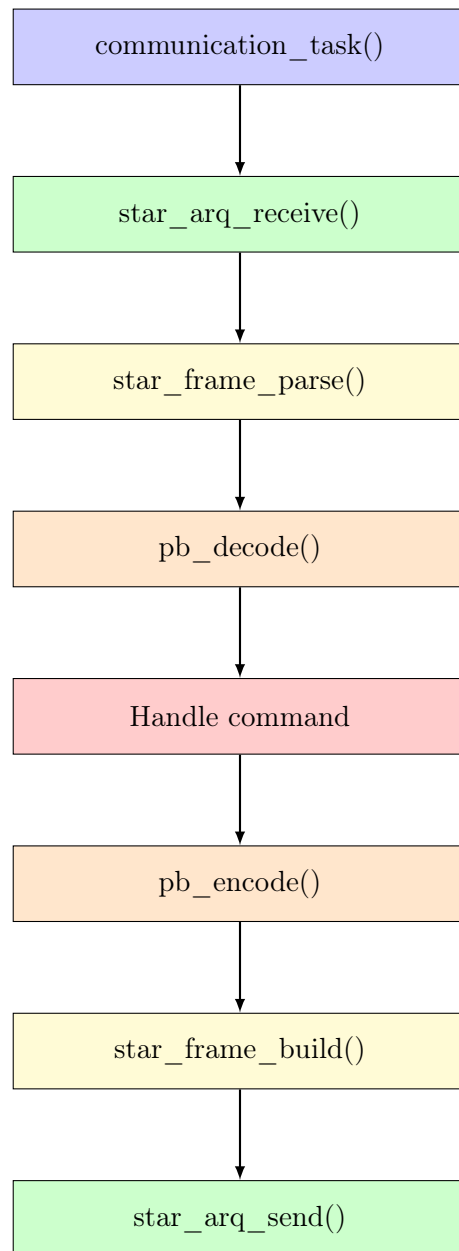


Figure 22: Communication Task Flow